



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escuela Técnica Superior de Ingeniería Informática
Universidad Politécnica de Valencia

Libreta de Estudio Redes Industriales

TODO: Foto de portada

REDES INDUSTRIALES

Grado en Informática Industrial y Robótica

Autores:

Francisco Nortes Novikov

Mario Martínez Carneros

Profesor: Guillermo Lenin Lemus Zúñiga

Índice general

Índice general	III
Índice de figuras	IX
Índice de tablas	X
Índice de códigos	XI

1 Unidad 1: Comunicación Serie	1
1.1 ¿Por qué comunicamos dispositivos?	1
1.2 Transmisión paralela vs. transmisión en serie	2
1.2.1 Transferencia en paralelo	2
1.2.2 Transferencia en serie	3
1.3 Tipos de codificación en comunicaciones serie	4
1.3.1 NRZ (Non-Return-to-Zero)	4
1.3.2 NRZ bipolar (NRZ-I)	5
1.3.3 Manchester	5
1.3.4 RZ (Return-to-Zero)	5
1.3.5 AMI (Alternate Mark Inversion)	6
1.4 Conceptos fundamentales de las comunicaciones serie	7
1.4.1 Topologías de conexión	7
1.4.2 Configuraciones balanceadas vs. no balanceadas	8
1.4.3 Medios de transmisión físicos	9
1.5 El reloj: cómo se sincronizan emisor y receptor	10
1.5.1 Transmisión asíncrona: relojes independientes	10
1.5.2 Transmisión síncrona: reloj en línea separada	11
1.5.3 Recuperación de reloj desde los datos (CDR)	11
1.6 Formatos de trama y protocolos	12
1.6.1 Transmisión asíncrona vs. síncrona de tramas	13
1.7 Detección y corrección de errores	13
1.7.1 Bit de paridad	13
1.7.2 Código de verificación de bloques (BCC / BCS)	13
1.7.3 Verificación de redundancia cíclica (CRC)	13
1.7.4 Corrección de errores de reenvío (FEC)	14
1.8 Métodos de acceso al medio	14
1.8.1 Maestro-Esclavo	14
1.8.2 CSMA (Carrier Sense Multiple Access)	15
1.8.3 TDMA (Time Division Multiple Access)	16
1.8.4 Dúplex (Duplexing)	17
1.9 El estándar RS-232: comunicación serie industrial	17
1.9.1 ¿Qué es RS-232?	17
1.9.2 Niveles de tensión: la gran diferencia con UART/TTL	18
1.9.3 Conectores y pines	19

1.9.4	Velocidad de subida y limitaciones	19
1.9.5	Ventajas del RS-232 frente a otros protocolos	19
1.10	El protocolo USB: bus serie moderno	20
1.10.1	Introducción	20
1.10.2	Características principales	20
1.10.3	Interfaz física y niveles eléctricos	20
1.10.4	Codificación y sincronización	21
1.10.5	Terminología USB	21
1.11	Práctica 1: Comunicación UART con ESP32-S3	22
1.11.1	Objetivos	22
1.11.2	Material utilizado	22
1.11.3	Marco teórico: protocolo UART	23
1.11.4	Configuración de pines UART en ESP32-S3	25
1.11.5	Desarrollo de la práctica	26
1.11.6	Relación teoría-práctica	28
1.12	Práctica 2: Comunicación RS-232 entre ESP32-S3	28
1.12.1	Objetivos	28
1.12.2	Material utilizado	29
1.12.3	Desarrollo de la práctica	29
1.12.4	Relación teoría-práctica: comparación UART vs. RS-232 . . .	32
1.13	Resumen de la Unidad 1	32
2	Redes Inalámbricas y Comunicaciones IoT	35
2.1	Comunicaciones inalámbricas: ¿por qué eliminar los cables?	35
2.2	Bluetooth vs. Wi-Fi: dos filosofías, una misma banda	36
2.3	Bluetooth Low Energy (BLE)	37
2.3.1	¿Qué es BLE?	37
2.3.2	Arquitectura GATT: servicios y características	37
2.3.3	Advertising y escaneo	38
2.3.4	Notificaciones y lecturas	38
2.4	Wi-Fi en sistemas embebidos	38
2.4.1	Modos de operación	38
2.4.2	Protocolo HTTP para IoT	39
2.4.3	Redes corporativas y seguridad	39
2.5	Sensor de temperatura y humedad DHT11	39
2.6	Práctica 3: Comunicaciones Inalámbricas	40
2.6.1	Objetivos	40
2.6.2	Material utilizado	40
2.6.3	Desarrollo de la práctica	40
2.6.4	Problemas encontrados	45
2.6.5	Relacion teoria-practica: BLE vs. Wi-Fi en la practica	45
2.7	Resumen de la Unidad 2	45
3	Redes de Area Local y Encaminamiento	47
3.1	Redes de computadores: mas alla de dos dispositivos	47
3.1.1	Medios de transmision	48
3.2	El modelo TCP/IP	48
3.2.1	TCP frente a UDP	48
3.3	Direccionamiento IP	49
3.3.1	Estructura de una direccion IPv4	49

3.3.2	Clases de direcciones IP	49
3.3.3	Gateway (puerta de enlace)	50
3.4	Dispositivos de interconexion: switch y router	50
3.4.1	Switch (conmutador)	50
3.4.2	Router (encaminador)	50
3.4.3	Tabla de encaminamiento	51
3.5	Encaminamiento: como los paquetes encuentran su camino	51
3.5.1	Tipos de encaminamiento	51
3.5.2	Encaminamiento en la practica: de una LAN a otra	52
3.6	Practica 4: Cisco Packet Tracer	52
3.6.1	Objetivos	52
3.6.2	Material utilizado	52
3.6.3	Desarrollo de la practica	53
3.6.4	Analisis de la comunicacion entre redes	55
3.6.5	Relacion teoria-practica	55
3.7	Resumen de la Unidad 3	55
4	Servicios de Red	57
4.1	Definicion de servicios de red	57
4.2	Clasificacion de los servicios de red	57
4.2.1	Servicios de infraestructura	57
4.2.2	Servicios de internet y comunicacion	58
4.2.3	Servicios de acceso y gestion	58
4.2.4	Servicios de aplicacion	58
4.3	DNS (Domain Name System)	59
4.3.1	El problema: los humanos usan nombres, las maquinas usan numeros	59
4.3.2	El nombre en una red: FQDN	59
4.3.3	Estructura jerarquica del DNS	59
4.3.4	Tipos de consultas DNS	60
4.3.5	Tipos de registros DNS	60
4.3.6	DNS internos vs. DNS externos (autoritativos)	60
4.3.7	Consulta manual con nslookup	61
4.4	DHCP (Dynamic Host Configuration Protocol)	61
4.4.1	Definicion	61
4.4.2	Beneficios clave de DHCP	61
4.4.3	Como funciona: el proceso DORA	61
4.4.4	Mensajes DHCP adicionales	62
4.4.5	Renovacion de la concesion	62
4.4.6	Ambitos (scopes) y exclusiones	63
4.4.7	Agentes de retransmision (DHCP Relay)	63
4.4.8	BOOTP: el predecesor de DHCP	63
4.5	Otros servicios de red relevantes	63
4.5.1	HTTP/HTTPS (World Wide Web)	63
4.5.2	SSH (Secure Shell)	63
4.5.3	Correo electronico: SMTP, POP3, IMAP	64
4.5.4	FTP/SFTP	64
4.6	Resumen de la Unidad 4	64
5	Introduccion a las Redes Industriales	65

5.1	Sistemas industriales distribuidos	65
5.1.1	Definicion	65
5.1.2	Caracteristicas principales	66
5.1.3	Aplicaciones y ventajas	66
5.2	Redes industriales: conceptos fundamentales	66
5.2.1	Definicion	66
5.2.2	Como funcionan	67
5.2.3	Convergencia de TI y TO (OT)	67
5.2.4	Desafios	67
5.3	Clasificacion de las redes industriales	67
5.3.1	Clasificacion por funcionalidad	68
5.3.2	Tabla comparativa de tipos de redes industriales	68
5.4	Principales protocolos de redes industriales	69
5.4.1	PROFINET	69
5.4.2	PROFIBUS	69
5.4.3	EtherCAT	69
5.4.4	ControlNet	69
5.4.5	Ethernet en la industria: importancia en la Industria 4.0 . . .	70
5.5	Resumen de la Unidad 5	70
6	Encaminadores y Protocolos de Routing	71
6.1	Conceptos fundamentales de encaminamiento	71
6.1.1	Tipos de encaminamiento	71
6.2	Grafos y encaminamiento	72
6.2.1	Tipos de grafos	72
6.2.2	Concatenaciones de enlaces	72
6.2.3	Grafo conectado	73
6.2.4	Representacion de grafos	73
6.2.5	Costes en los enlaces	73
6.3	Algoritmos de camino mas corto	74
6.3.1	Algoritmos con una unica fuente	74
6.3.2	Algoritmos para toda la red	74
6.3.3	Arbol de expansion minimo (Minimum Spanning Tree) . . .	74
6.4	Protocolos de encaminamiento	74
6.4.1	Clasificacion de protocolos	75
6.4.2	Encaminamiento por vector de distancia	75
6.4.3	Encaminamiento por estado de enlace	76
6.4.4	Encaminamiento jerarquico: Sistemas Autonomos (AS) . . .	76
6.5	Protocolo RIP (Routing Information Protocol)	77
6.5.1	Caracteristicas generales	77
6.5.2	Temporizadores de RIP	77
6.5.3	Mensajes RIP	77
6.5.4	RIPng para IPv6	78
6.6	Practica 5: Encaminamiento en Internet con RIP	78
6.6.1	Objetivos	78
6.6.2	Material utilizado	78
6.6.3	Desarrollo de la practica	79
6.6.4	Relacion teoria-practica	82
6.7	Resumen de la Unidad 6	82

7	Protocolos de Comunicacion y Redes CAN	85
7.1	I2C: Inter-Integrated Circuit	85
7.1.1	Que es I2C	85
7.1.2	Senalizacion	86
7.1.3	Transferencia de datos	86
7.1.4	Ejemplo practico: sensor brujula digital CMPS03	87
7.2	SPI: Serial Peripheral Interface	88
7.2.1	Que es SPI	88
7.2.2	Senales del bus SPI	88
7.2.3	Funcionamiento	88
7.2.4	Comparacion: SPI vs I2C vs UART	89
7.3	RS-485: Comunicacion diferencial robusta	90
7.3.1	Introduccion	90
7.3.2	Caracteristicas principales	90
7.3.3	Niveles de senal y cableado	90
7.3.4	Distancia vs velocidad	91
7.3.5	RS-232 vs RS-485	91
7.4	Modbus: protocolo de comunicacion industrial	91
7.4.1	Que es Modbus	91
7.4.2	Variantes de Modbus	92
7.4.3	Modbus RTU en detalle	92
7.4.4	RS-232 vs RS-485 en Modbus	93
7.5	CAN: Controller Area Network	94
7.5.1	Introduccion	94
7.5.2	Caracteristicas principales	94
7.5.3	Tipos de tramas CAN	94
7.5.4	Formato de trama de datos	95
7.5.5	Arbitraje de bus: dominante y recesivo	95
7.5.6	Supervision de bits y relleno de bits	96
7.5.7	Errores y confinamiento de fallos	97
7.5.8	Topologia y cableado	97
7.5.9	Controladores CAN: Basic vs Full CAN	98
7.5.10	Resumen: SPI, I2C, RS-485, Modbus y CAN	99
7.6	Resumen de la Unidad 7	99

Bibliografía	101
---------------------	------------

Apéndices

A	Manual de Instalación	103
A.1	Requisitos previos	103
A.2	Procedimiento de instalación	103
A.3	Resolución de problemas frecuentes	103
B	Manual de Funcionamiento	105
B.1	Puesta en marcha	105
B.2	Operación básica	105
B.3	Mantenimiento y buenas prácticas	105
C	Otra documentación adicional	107
C.1	Especificaciones técnicas detalladas	107
C.2	Manuales y referencias complementarias	107

Índice de figuras

1.1	Modelo conceptual de cualquier comunicación: origen, medio y destino	1
1.2	Comparación visual: transmisión paralela (8 líneas) vs. transmisión en serie (1 línea)	3
1.3	Ejemplo de codificación NRZ para una secuencia de bits	4
1.4	Comparación entre codificación NRZ y NRZ-I	5
1.5	Ejemplo de codificación Manchester	5
1.6	Ejemplo de codificación RZ unipolar y bipolar	6
1.7	Ejemplo de codificación AMI	6
1.8	Las cuatro topologías básicas en comunicaciones serie	7
1.9	Cancelación del ruido en una conexión balanceada (diferencial) . .	9
1.10	Medios de transmisión físicos más comunes en comunicaciones serie	10
1.11	Formato típico de una transmisión asíncrona byte a byte (ejemplo UART)	11
1.12	Estructura genérica de una trama de datos en comunicaciones serie	12
1.13	Topología maestro-esclavo en un bus compartido	15
1.14	Ejemplo de CSMA/CD con detección de colisión y retransmisión .	16
1.15	Ejemplo de ciclo TDMA con cuatro nodos	17
1.16	Modelo DTE-DCE en una comunicación RS-232	18
1.17	Distribución de pines en un conector DB-9 para RS-232	19
1.18	Conector USB estándar y asignación de pines	21
1.19	Conexión cruzada entre dos dispositivos UART: TX de A con RX de B, y viceversa	23
1.20	Formato de trama UART para la configuración 8N1 (8 bits, sin paridad, 1 bit de parada)	24
1.21	Captura de osciloscopio de una trama UART real (ejemplo del carácter 'A')	24
1.22	Secuencia de control de flujo por hardware (RTS/CTS) en UART . .	25
1.23	Placa ESP32-S3-DevKitC-1 con indicación de los pines UART utilizados	25
1.24	Código de ejemplo para envío de datos por UART (PC o ESP32) . .	27
1.25	Señal UART del carácter 'A' capturada en osciloscopio	27
1.26	Montaje físico para comunicación bidireccional PC ↔ ESP32 por UART	28
1.27	Resultado de la comunicación bidireccional PC ↔ ESP32	28
1.28	Montaje físico de dos ESP32-S3 conectadas en comunicación serie directa	30
1.29	Señal del carácter 'S' en osciloscopio (niveles TTL)	31
1.30	Medida de amplitud de la señal TTL en osciloscopio	31
1.31	Montaje con adaptadores TTL-RS-232 y cable null-modem	32

1.32	Mapa conceptual resumen de la Unidad 1	33
2.1	Montaje físico del sensor DHT11 con la ESP32-S3	41
2.2	Monitor serie del cliente BLE recibiendo temperatura	43
2.3	Monitor serie del cliente Wi-Fi recibiendo temperatura via HTTP	45
3.1	Topología de la primera LAN (192.168.10.0)	53
3.2	Topología de la segunda LAN (192.168.20.0)	54
3.3	Topología completa: interconexión de dos LAN mediante un router	55
4.1	Proceso DORA de DHCP	62
6.1	Topología de la red con 4 routers y encaminamiento RIP	79
6.2	Resultado de traceroute mostrando el camino de 3 saltos	82
7.1	Esquema básico del bus I2C con resistencias de pull-up.	86
7.2	Esquema del bus SPI con un maestro y dos esclavos.	88
7.3	Topología típica de un bus RS-485 con terminaciones.	91
7.4	Arbitraje no destructivo en el bus CAN.	96
7.5	Topología de bus CAN según la norma ISO 11898.	98

Índice de tablas

1.1	Comparativa: transmisión paralela vs. en serie	4
1.2	Resumen comparativo de codificaciones de línea	7
1.3	Los tres métodos de sincronización en comunicaciones serie	12
1.4	Comparativa de métodos de detección de errores	14
1.5	Comparativa de niveles de tensión: TTL vs. RS-232	18
1.6	Comparativa entre RS-232 y USB	22
1.7	Interfaces seriales más comunes en sistemas embebidos e industriales	26
1.8	Mapeo de pines UART en ESP32-S3	26
1.9	Conexión entre conceptos teóricos y elementos de la Práctica 1	29
1.10	Comparación entre Práctica 1 (UART/TTL) y Práctica 2 (RS-232)	32
2.1	Comparación entre Bluetooth y Wi-Fi	36
2.2	Conexión entre conceptos teóricos y elementos de la Práctica 3	46
3.1	Las cuatro capas del modelo TCP/IP	48
3.2	Comparación entre TCP y UDP	49
3.3	Clases de direcciones IPv4	50
3.4	Conexión entre conceptos teóricos y elementos de la Práctica 4	56
4.1	Tipos de registros DNS más comunes	60
5.1	Niveles de las redes industriales	68
5.2	Comparación de tipos de redes industriales	68

6.1	Representacion de grafos	73
6.2	Clasificacion de protocolos de encaminamiento	75
6.3	Versiones de RIP	77
6.4	Conexion entre conceptos teoricos y elementos de la Practica 5	82
7.1	Comparativa de protocolos de comunicacion serie	89
7.2	Relacion entre distancia y velocidad en RS-485	91
7.3	Comparativa entre RS-232 y RS-485	91
7.4	Variantes del protocolo Modbus	92
7.5	Formato de trama Modbus RTU	93
7.6	Campos de una trama de datos CAN (formato estandar)	95
7.7	Longitud de trama CAN con relleno de bits	97
7.8	Asignacion de pines en conector DB9 para CAN	98
7.9	Resumen comparativo de protocolos industriales	99

Índice de códigos

1.1	Código ESP32 para transmisión/recepción por Serial2	29
2.1	Lectura del sensor DHT11	40
2.2	Servidor BLE de temperatura (extracto)	41
2.3	Envio de temperatura por BLE	42
2.4	Cliente BLE de temperatura (extracto)	42
2.5	Servidor Wi-Fi de temperatura	43
2.6	Cliente Wi-Fi de temperatura	44
3.1	Verificacion de conectividad en la LAN 1	53
3.2	Verificacion de conectividad entre redes	54
4.1	Consulta DNS con nslookup	61
6.1	Configuracion de RIP en Router 0	79
6.2	Configuracion de RIP en Router 1	80
6.3	Configuracion de RIP en Router 2	80
6.4	Configuracion de RIP en Router 3	80
6.5	Verificacion de la tabla de encaminamiento en Router 0	81
6.6	Ping desde LAN 192.168.10.0 hacia LAN 192.168.20.0	81
6.7	Traceroute desde Router 0 hacia 192.168.20.10	81

Resumen

TODO: Escribir el resumen del nuevo trabajo académico.

Palabras clave: TODO: palabra clave 1, palabra clave 2, palabra clave 3

CAPÍTULO 1

Unidad 1: Comunicación Serie

1.1 ¿Por qué comunicamos dispositivos?

En cualquier sistema industrial moderno, los componentes necesitan intercambiar información: un sensor debe enviar una medida a un controlador, un PLC debe ordenar un movimiento a un motor, o un ordenador debe supervisar el estado de toda una planta. Esta necesidad de compartir datos entre dispositivos electrónicos es el origen de las comunicaciones digitales.

A lo largo de la historia, los ingenieros han buscado formas cada vez más eficientes de transmitir información. Desde el telégrafo de Samuel Morse (1844), que enviaba puntos y rayas por un único cable, hasta las redes industriales actuales que mueven gigabytes por segundo, el principio fundamental es el mismo: codificar información, transportarla por un medio físico y decodificarla en el destino.



Figura 1.1: Modelo conceptual de cualquier comunicación: origen, medio y destino

Hoy en día, la decisión más básica que enfrenta un diseñador de sistemas embebidos es: ¿transmito los datos en paralelo o en serie?

1.2 Transmisión paralela vs. transmisión en serie

1.2.1. Transferencia en paralelo

En una transmisión paralela, todos los bits de una palabra de datos se envían simultáneamente por múltiples líneas independientes. Por ejemplo, para transmitir un byte completo (8 bits), se necesitan 8 cables de datos, más las líneas de control y tierra.

*La **transferencia de datos en paralelo** es un método de comunicación en el que todos los bits de una palabra de datos se transmiten simultáneamente a través de múltiples canales físicos independientes.*

Las ventajas parecen claras: es rápida, ya que un byte completo llega en un solo ciclo. Sin embargo, esta aparente ventaja desaparece cuando se analizan los problemas físicos:

- **Coste hardware elevado:** cada bit requiere su propio conductor, conector y circuito de E/S.
- **Diafonía:** a alta velocidad, las señales eléctricas de un cable se acoplan a los adyacentes por efecto capacitivo e inductivo, generando ruido.
- **Sesgo de tiempo (skew):** los cables no tienen exactamente la misma longitud ni impedancia, por lo que los bits llegan ligeramente desfasados. A velocidades altas, este desfase puede causar errores.
- **Limitación de distancia:** a frecuencias de decenas de Mb/s, los buses paralelos solo funcionan en trayectos de pocos centímetros. A Gb/s, apenas unos milímetros dentro de un chip.

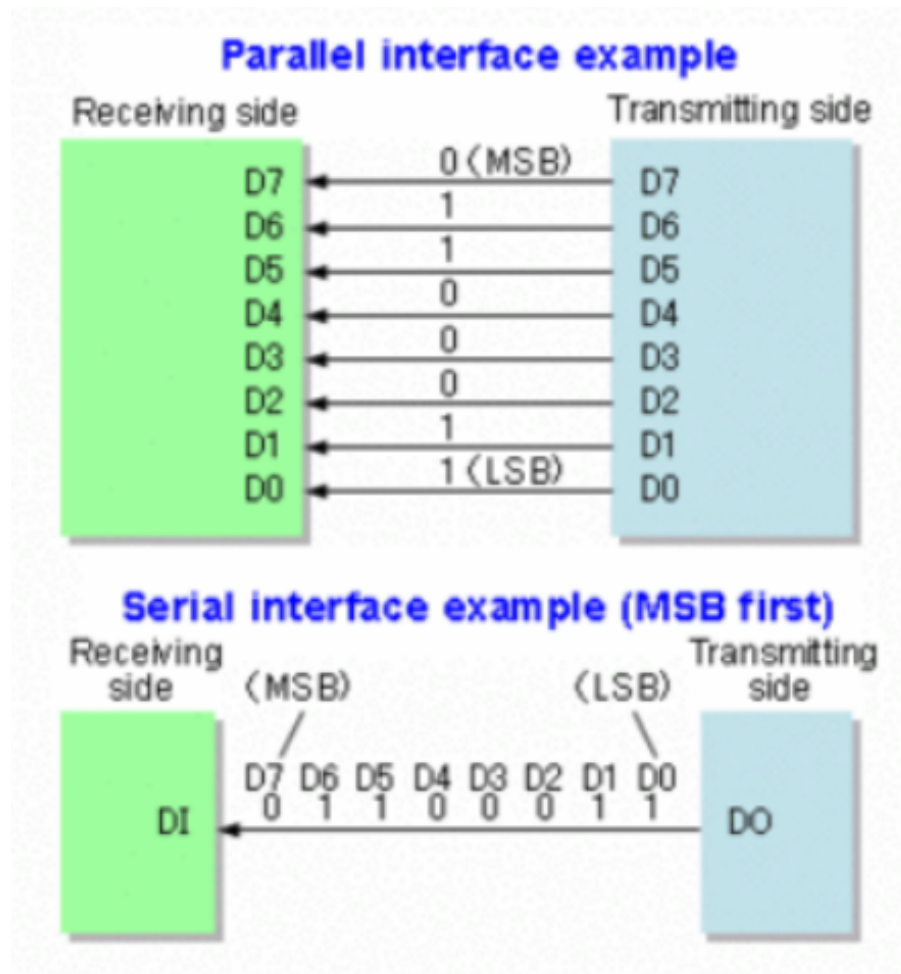


Figura 1.2: Comparación visual: transmisión paralela (8 líneas) vs. transmisión en serie (1 línea)

1.2.2. Transferencia en serie

En una transmisión en serie, los bits se envían uno tras otro por una única línea de datos (más tierra). El byte completo tarda más tiempo en llegar, pero el hardware es mucho más sencillo.

*La **transferencia de datos en serie** es un método de comunicación en el que los bits de una palabra se transmiten secuencialmente, uno tras otro, a través de un único canal físico.*

Las ventajas de la transmisión en serie son:

- **Menor coste:** un solo cable de datos, conectores más pequeños y menos pines en los circuitos integrados.
- **Mayor inmunidad al ruido:** con técnicas de transmisión diferencial (ver sección 1.4.2), el ruido se cancela.
- **Distancias mayores:** algunas interfaces serie alcanzan cientos de metros (RS-485) o kilómetros (fibra óptica).

- **Velocidades elevadas:** aunque los bits llegan de uno en uno, las frecuencias de reloj pueden ser muy altas (Gb/s en USB, SATA, fibra óptica).

La principal desventaja es temporal: transmitir un byte en serie tarda más que en paralelo. Sin embargo, en la mayoría de aplicaciones industriales y de comunicaciones, esta diferencia es irrelevante frente a los beneficios de simplicidad y fiabilidad.

Tabla 1.1: Comparativa: transmisión paralela vs. en serie

Característica	Paralelo	Serie
Número de líneas de datos	N (una por bit)	1
Velocidad por transferencia	Alta (1 byte/ciclo)	Más lenta (1 bit/ciclo)
Coste hardware	Elevado (muchos pines, cables)	Bajo (pocos pines, cables)
Diafonía	Problema grave a alta velocidad	Prácticamente nulo
Distancia máxima	Centímetros a metros	Metros a kilómetros
Aplicaciones típicas	Buses internos de PCB, RAM	USB, Ethernet, UART, SATA

1.3 Tipos de codificación en comunicaciones serie

Antes de que los bits viajen por el medio físico, deben convertirse en señales eléctricas (o luminosas) que el receptor pueda interpretar. El proceso de convertir un bit lógico (0 o 1) en una forma de onda física se denomina **codificación de línea**. La elección de la codificación afecta directamente a la inmunidad al ruido, la capacidad de recuperación de reloj y la eficiencia espectral del canal.

1.3.1. NRZ (Non-Return-to-Zero)

En la codificación NRZ, el nivel de tensión se mantiene constante durante todo el periodo de bit: un nivel alto representa un “1” y un nivel bajo representa un “0”. Es el método más sencillo, pero presenta dos inconvenientes principales: no garantiza transiciones cuando se transmiten largas secuencias del mismo bit, lo que dificulta la recuperación de reloj, y tiene una componente continua (DC) que no es adecuada para algunos medios acoplados con transformador.

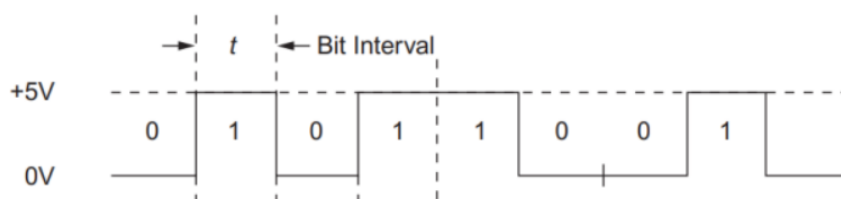


Figura 1.3: Ejemplo de codificación NRZ para una secuencia de bits

1.3.2. NRZ bipolar (NRZ-I)

Una variante del NRZ es NRZ-I (inverso), en el que la señal **no cambia** para un "0" y **invierte** su polaridad para un "1". De esta forma, una secuencia de unos produce transiciones continuas, mientras que una secuencia de ceros mantiene el nivel. NRZ-I se utiliza en interfaces como USB (junto con bit stuffing) para garantizar suficientes transiciones y permitir la recuperación de reloj.

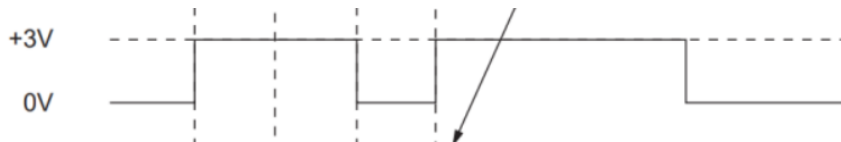


Figura 1.4: Comparación entre codificación NRZ y NRZ-I

1.3.3. Manchester

La codificación Manchester resuelve el problema de la recuperación de reloj generando **siempre una transición en el centro de cada bit**: una transición de bajo a alto representa un "1", y de alto a bajo representa un "0". Garantiza una transición por bit, lo que facilita enormemente la sincronización, pero duplica el ancho de banda necesario porque la frecuencia de la señal es el doble de la tasa de bits. Se empleó históricamente en Ethernet 10BASE-T.

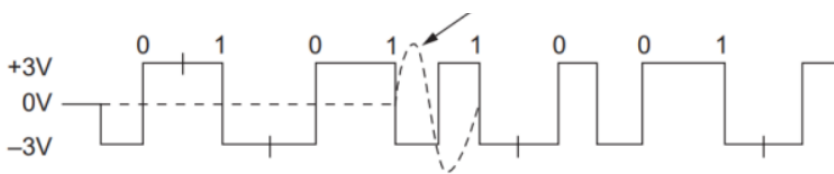


Figura 1.5: Ejemplo de codificación Manchester

1.3.4. RZ (Return-to-Zero)

En RZ, la señal vuelve a cero (o al nivel de referencia) en la **mitad** del periodo de bit, independientemente del valor transmitido. Por ejemplo, un "1" se representa como un pulso positivo seguido de un retorno a cero, y un "0" como un pulso negativo seguido de retorno a cero (en RZ bipolar). Aunque simplifica la sincronización, consume más ancho de banda y requiere tres niveles de tensión, lo que complica los circuitos.

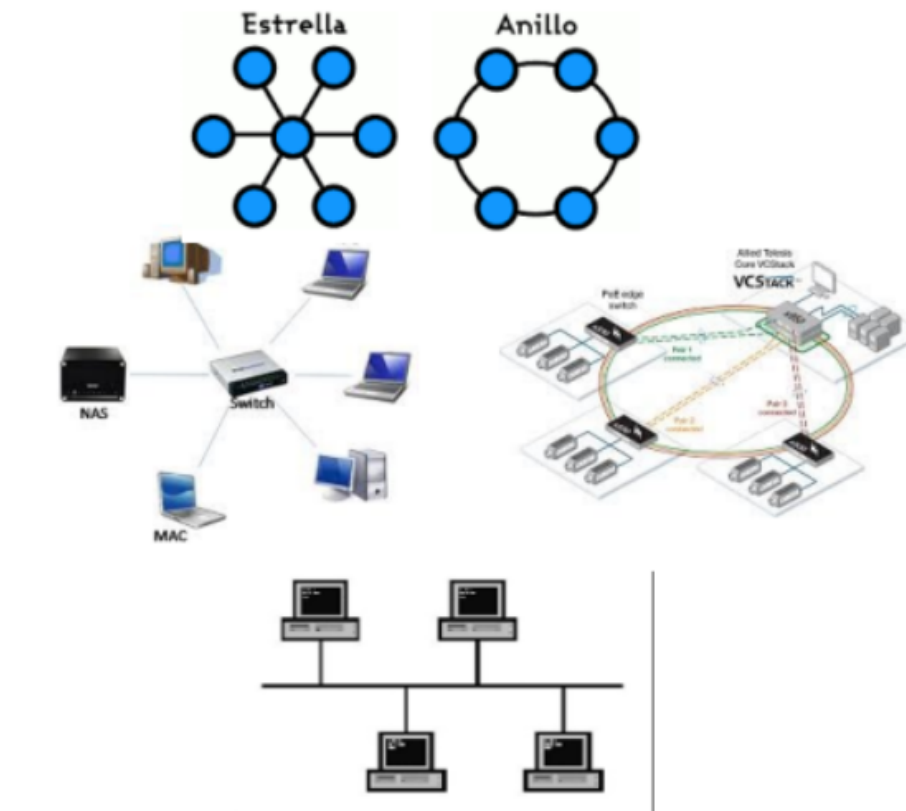


Figura 1.6: Ejemplo de codificación RZ unipolar y bipolar

1.3.5. AMI (Alternate Mark Inversion)

AMI es una codificación de tres niveles en la que los "0" se transmiten como nivel cero, mientras que los "1" (marcas) alternan entre polaridad positiva y negativa. Esta alternancia elimina la componente continua y permite detectar errores de transmisión: si el receptor observa dos marcas consecutivas del mismo signo, sabe que ha ocurrido un error. AMI se utiliza en sistemas de telecomunicaciones tradicionales (T1/E1).

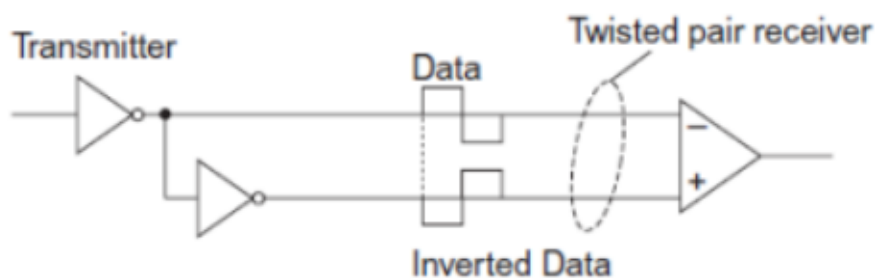


Figura 1.7: Ejemplo de codificación AMI

Tabla 1.2: Resumen comparativo de codificaciones de línea

Codificación	Transiciones	Componente DC	Uso típico
NRZ	No garantizadas	Sí	UART, SPI (interno)
NRZ-I	Sólo en unos (con bit stuffing)	Sí	USB
Manchester	Garantizadas (1/bit)	No	Ethernet 10BASE-T
RZ	Garantizadas (retorno a cero)	No (bipolar)	Canales de fibra óptica antiguos
AMI	En marcas (alternadas)	No	T1/E1, telecomunicaciones

1.4 Conceptos fundamentales de las comunicaciones serie

1.4.1. Topologías de conexión

La topología describe cómo se conectan físicamente los nodos de una red:

- **Punto a punto (peer-to-peer):** la conexión más simple, con solo dos nodos. Ejemplo: un PC conectado a una impresora, o un microcontrolador a un sensor.
- **Bus:** todos los nodos comparten un mismo medio de transmisión. Ejemplo: RS-485, CAN bus.
- **Estrella:** todos los nodos se conectan a un nodo central. Ejemplo: Ethernet con switch, USB con hub.
- **Anillo:** cada nodo se conecta a dos vecinos, formando un círculo. Ejemplo: redes industriales antiguas, anillos ópticos.

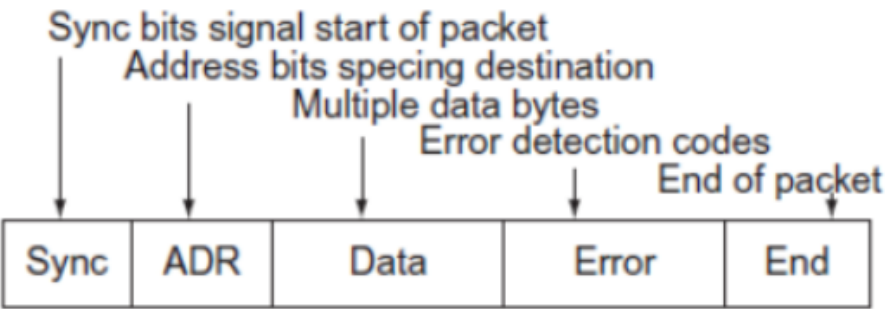


Figura 1.8: Las cuatro topologías básicas en comunicaciones serie

1.4.2. Configuraciones balanceadas vs. no balanceadas

Esta distinción es fundamental para entender la inmunidad al ruido de una comunicación.

Conexión no balanceada (single-ended)

En una conexión no balanceada, la señal se transmite por un único conductor y se toma como referencia la tierra (GND). El voltaje de la señal se mide respecto a tierra.

*Una conexión **no balanceada** (o desbalanceada) es aquella en la que el voltaje de señal se referencia a un punto de tierra común, utilizando un único conductor para transportar la información.*

Es sencilla y económica, pero tiene un problema grave: cualquier ruido inducido en el cable o diferencia de potencial entre las tierras de los dos dispositivos se suma directamente a la señal, pudiendo causar errores.

Conexión balanceada (diferencial)

En una conexión balanceada, la señal se transmite por dos conductores, ninguno de los cuales está conectado a tierra. Los voltajes en ambos conductores son de polaridad opuesta respecto a tierra. El receptor no mide el voltaje respecto a tierra, sino la **diferencia** entre ambos conductores.

*Una conexión **balanceada** (o diferencial) es aquella en la que la señal se transmite mediante dos conductores con voltajes de polaridad opuesta, y el receptor recupera la información calculando la diferencia entre ambos voltajes.*

¿Por qué cancela el ruido? Si un ruido externo se acopla a la línea, afecta por igual a los dos conductores. Como el receptor resta los voltajes, la componente de ruido común ($V_{\text{ruido}} - V_{\text{ruido}} = 0$) desaparece. Esto se conoce como **rechazo al modo común (CMRR)**.

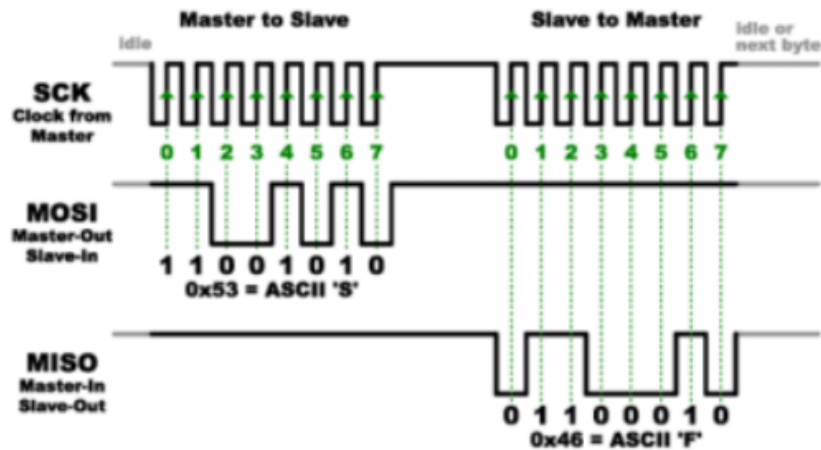


Figura 1.9: Cancelación del ruido en una conexión balanceada (diferencial)

1.4.3. Medios de transmisión físicos

Los datos en serie pueden viajar por diferentes medios. Los más comunes son:

Líneas en placa de circuito impreso (PCB)

Son las trazas de cobre que conectan chips dentro de una misma placa. Son cortas (centímetros) y pueden ser desbalanceadas (traza + tierra) o balanceadas (dos trazas paralelas). Su uso se limita a comunicaciones internas de alta velocidad entre circuitos integrados vecinos.

Cable coaxial

Un conductor central rodeado por una malla metálica (blindaje) y una cubierta exterior. Es una línea desbalanceada con impedancia característica típica de $50\ \Omega$. Aunque la atenuación es considerable, puede transportar señales de varios Gb/s en distancias cortas. Hoy se usa principalmente en RF y en algunas redes de comunicaciones.

Par trenzado (UTP/STP)

Dos cables aislados retorcidos entre sí, sin blindaje (UTP) o con blindaje (STP). Es el medio más utilizado en redes modernas:

- El retorcido reduce la diafonía entre pares adyacentes.
- El blindaje (en STP) protege en entornos industriales ruidosos.
- Ejemplos: cableado telefónico, Ethernet CAT5/CAT6.

Fibra óptica

Un núcleo de vidrio o plástico rodeado de revestimiento y cubierta. Los datos se convierten en pulsos de luz infrarroja generados por LEDs o láseres. En el receptor, un fotodiodo (PIN o avalancha) convierte la luz de nuevo en señal eléctrica.

Sus ventajas son excepcionales:

- Velocidades superiores a 100 Gb/s.
- Inmunidad total al ruido eléctrico y a la diafonía.
- Distancias de kilómetros sin regeneración.

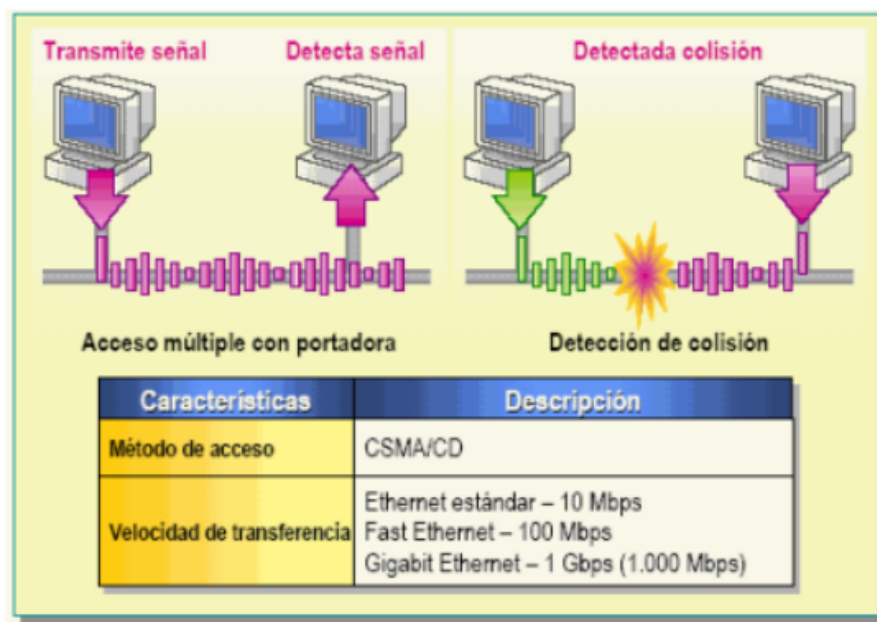


Figura 1.10: Medios de transmisión físicos más comunes en comunicaciones serie

1.5 El reloj: cómo se sincronizan emisor y receptor

En cualquier comunicación digital, el receptor debe saber **cuándo** leer cada bit. Si muestrea demasiado pronto o demasiado tarde, leerá un valor incorrecto. La sincronización se resuelve mediante una señal de reloj. En comunicaciones serie existen tres estrategias fundamentales.

1.5.1. Transmisión asíncrona: relojes independientes

Cada dispositivo tiene su propio reloj interno. Los relojes no están sincronizados, pero deben coincidir aproximadamente en frecuencia. Para que el receptor sepa cuándo empieza un nuevo dato, el transmisor añade un **bit de inicio** (start bit) antes de los datos propiamente dichos. Al detectar esta transición, el receptor recalibra su muestreo para el byte siguiente.

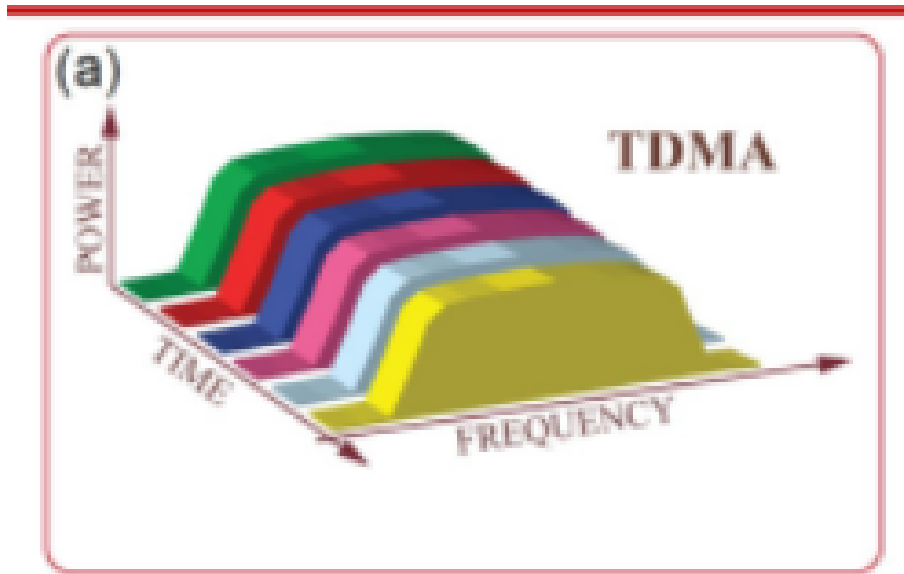


Figura 1.11: Formato típico de una transmisión asíncrona byte a byte (ejemplo UART)

Ventaja: solo se necesita una línea de datos (más tierra). **Limitación:** las frecuencias de reloj deben ser muy similares, y se desperdicia algo de tiempo en los bits de sincronización de cada byte.

1.5.2. Transmisión síncrona: reloj en línea separada

El transmisor envía una señal de reloj por un cable adicional, junto con la línea de datos. El receptor muestrea los datos exactamente en los flancos de este reloj externo.

Ventaja: sincronización perfecta, sin margen de error por desajuste de relojes. **Desventaja:** requiere una línea adicional (dos líneas de señal + tierra), lo que encarece el cableado.

1.5.3. Recuperación de reloj desde los datos (CDR)

En las comunicaciones más rápidas, no hay línea de reloj separada ni bits de inicio/parada. El receptor extrae el reloj directamente de las transiciones de la propia señal de datos. Para que esto funcione, los datos deben codificarse de forma que haya suficientes transiciones ($0 \rightarrow 1$ y $1 \rightarrow 0$), evitando secuencias largas del mismo nivel.

*La técnica de **reloj y recuperación de datos (CDR, Clock and Data Recovery)** consiste en reconstruir la señal de reloj a partir de las transiciones de la propia señal de datos recibida, permitiendo la sincronización sin necesidad de una línea de reloj separada.*

Este método se usa en interfaces de alta velocidad como USB 3.0, SATA, PCIe y Ethernet de fibra óptica.

Tabla 1.3: Los tres métodos de sincronización en comunicaciones serie

Método	Característica	Ejemplo
Asíncrona	Relojes independientes, bits de inicio/parada	UART, RS-232
Síncrona (reloj separado)	Línea de reloj adicional, perfecta sincronización	SPI, I2C
CDR	Reloj extraído de los datos, codificación especial	USB 3.0, SATA, Ethernet

1.6 Formatos de trama y protocolos

Un **protocolo** es el conjunto de reglas que definen cómo se comunican dos dispositivos: el formato de los datos, el orden de los bits, la detección de errores, el control de flujo, etc.

La mayoría de los datos se envían en bloques estructurados llamados **tramas** o **frames**. Aunque cada protocolo tiene su propio formato, la estructura general sigue un patrón común:

1. **Delimitador de inicio:** una secuencia especial de bits que indica “aquí empieza una trama”.
2. **Dirección:** identifica el nodo destino (y a veces el origen).
3. **Datos:** el contenido propiamente dicho, normalmente un bloque de bytes.
4. **Código de control de errores:** un valor calculado a partir de los datos (paridad, CRC, BCC) para detectar o corregir errores.
5. **Delimitador de fin:** señala el final de la trama.

D - 25	D - 9	FUNCION	NOMBRE	DIRECCION
1	-	Masa	GND	-
2	3	Transmit Data	TD	SALIDA
3	2	ReceiveData	RD	ENTRADA
4	7	RequestTo Send	RTS	SALIDA
5	8	ClearTo Send	CTS	ENTRADA
6	6	DataSet Ready	DSR	ENTRADA
7	5	MasaChasis	GND	-
8	1	DataCarrier Detect	DCD	ENTRADA
20	4	DataTerminalReady	DTR	SALIDA
22	9	RingIndicator	RI	ENTRADA

Figura 1.12: Estructura genérica de una trama de datos en comunicaciones serie

1.6.1. Transmisión asíncrona vs. síncrona de tramas

- **Asíncrona:** cada byte se envía de forma independiente con sus propios bits de inicio y parada. No hay una estructura de trama compleja; se transmiten bytes sueltos.
- **Síncrona:** los datos se agrupan en tramas largas. No hay bits de inicio/parada por cada byte; en su lugar, la trama comienza con un delimitador especial y el receptor usa CDR o una línea de reloj para muestrear continuamente.

1.7 Detección y corrección de errores

El ruido electromagnético, las interferencias y las imperfecciones del medio pueden alterar los bits durante la transmisión. Por eso, los protocolos serie incorporan mecanismos para detectar o incluso corregir errores.

1.7.1. Bit de paridad

Es el método más sencillo. Consiste en añadir un bit extra a cada byte transmitido, de forma que el número total de bits a 1 sea par (paridad par) o impar (paridad impar).

*El **bit de paridad** es un bit adicional que se añade a una palabra de datos para que el número total de bits con valor 1 sea par (paridad par) o impar (paridad impar). El receptor recuenta los bits y compara con el bit de paridad recibido; si hay discrepancia, se detecta un error.*

Limitación: solo detecta errores cuando cambia un número impar de bits. Si dos bits se invierten simultáneamente, la paridad no lo detecta. Además, no indica qué bit está mal, solo que ha ocurrido un error.

1.7.2. Código de verificación de bloques (BCC / BCS)

En lugar de verificar byte a byte, se puede calcular un código de verificación para un bloque completo de datos. Un método sencillo es el **BCC (Block Check Character)**: se suman (o se hace XOR) todos los bytes del bloque, y el resultado final se envía como un carácter de verificación al final de la trama. El receptor repite el cálculo y compara.

Este método permite detectar errores en múltiples bits del bloque, pero no identificar la posición exacta del error.

1.7.3. Verificación de redundancia cíclica (CRC)

El CRC es el método más robusto de detección de errores en comunicaciones serie industriales. Se trata los bits de datos como una secuencia binaria y se divide

por un polinomio generador predefinido mediante aritmética modular. El resto de esta división (el CRC) se adjunta a la trama y se verifica en el receptor.

El CRC detecta eficazmente las **ráfagas de errores** (varios bits consecutivos alterados), siempre que la longitud de la ráfaga no supere el grado del polinomio generador.

Tabla 1.4: Comparativa de métodos de detección de errores

Método	Complejidad	Eficacia	Uso típico
Bit de paridad	Muy baja	Detecta 1 bit impar	UART, RS-232
BCC/BCS	Baja	Detecta errores simples	Protocolos antiguos
CRC	Media	Detecta ráfagas de errores	Ethernet, CAN, USB, industrial

1.7.4. Corrección de errores de reenvío (FEC)

Además de detectar errores, algunos protocolos de comunicación serie son capaces de **corregirlos** sin necesidad de retransmisión. La técnica se conoce como **Forward Error Correction (FEC)** o corrección de errores de reenvío.

Los métodos FEC añaden información redundante calculada a partir de los datos originales. Si la señal recibida se deteriora por ruido o interferencias, el receptor utiliza esa redundancia para reconstruir los bits erróneos. Esto mejora la fiabilidad de la transmisión y reduce la tasa de errores de bits (BER).

Una ventaja notable de los sistemas FEC es que, en presencia de ruido, actúan como si el sistema produjera **ganancia de codificación**. Esta ganancia compensa el ruido en el canal y permite alcanzar velocidades de datos más altas sin aumentar la potencia de transmisión. Por ello, el FEC es fundamental en interfaces seriales de alta velocidad, comunicaciones por satélite y sistemas de almacenamiento masivo.

1.8 Métodos de acceso al medio

Cuando más de dos dispositivos comparten un mismo medio de transmisión, es necesario establecer un mecanismo que determine quién puede transmitir y cuándo. Estos mecanismos se denominan **métodos de acceso al medio** y son la base de cualquier red multipunto o bus compartido.

1.8.1. Maestro–Esclavo

En un sistema de acceso maestro–esclavo, un único dispositivo **maestro** controla dos o más dispositivos **esclavos**. El maestro decide qué esclavo debe transmitir o recibir en cada momento, evitando colisiones en el medio compartido. Los protocolos de interfaz utilizan comandos específicos para instruir a los esclavos, y todas las operaciones están determinadas por el maestro.

Una técnica común es el **sondeo** (polling): el maestro consulta a cada esclavo en secuencia, enviando o recibiendo datos según sea necesario. Si un esclavo no tiene información, responde con una indicación de “no hay datos” y el maestro pasa al siguiente. Este método es sencillo, determinista y ampliamente utilizado en buses industriales como SPI y I2C.

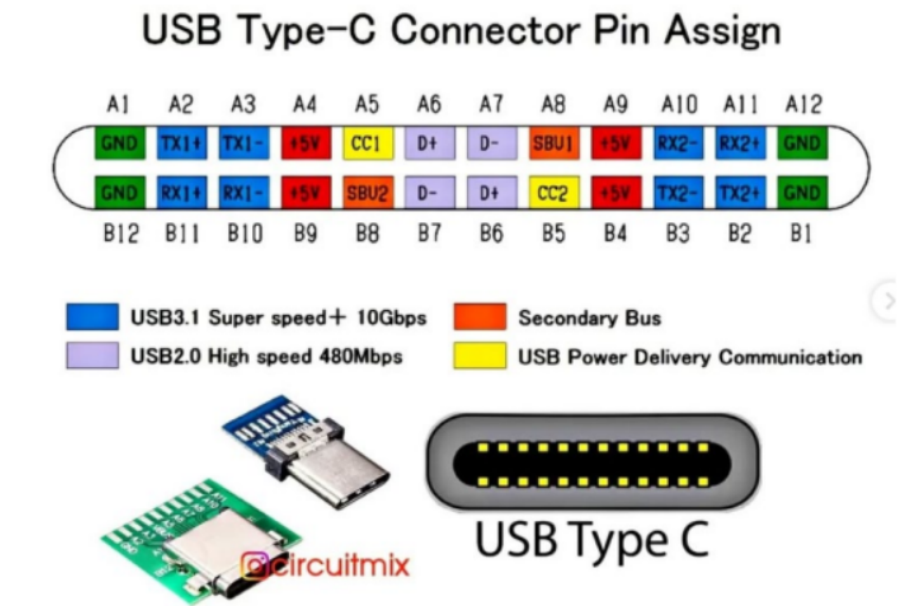


Figura 1.13: Topología maestro-esclavo en un bus compartido

1.8.2. CSMA (Carrier Sense Multiple Access)

El **acceso múltiple por detección de portadora (CSMA)** es un sistema en el que todos los nodos de un bus “escuchan” el medio antes de transmitir. Si detectan que ya hay datos en tránsito (un “portador” presente), esperan para evitar colisiones.

La mayoría de los sistemas CSMA incorporan también **detección de colisiones (CD)**. Si dos dispositivos transmiten simultáneamente, sus señales se mezclan y generan errores. CSMA/CD detecta la colisión, hace que ambos nodos dejen de transmitir y esperen un tiempo aleatorio antes de reintentarlo. Aunque este método maneja automáticamente la contención, la competencia por el bus reduce el rendimiento global y aumenta la latencia, especialmente cuando hay muchos nodos activos.

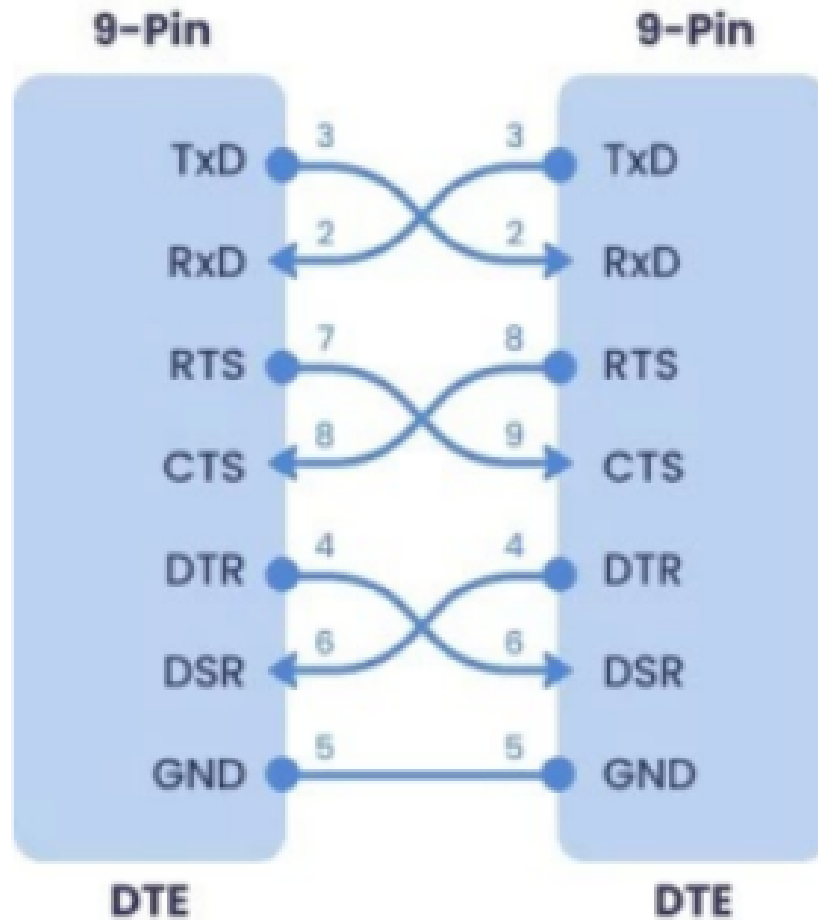


Figura 1.14: Ejemplo de CSMA/CD con detección de colisión y retransmisión

1.8.3. TDMA (Time Division Multiple Access)

El **acceso múltiple por división de tiempo (TDMA)** asigna a cada nodo una **franja horaria** (time slot) dentro de un ciclo periódico. El sistema define una unidad de tiempo para un ciclo completo y la divide en intervalos, uno para cada nodo del medio compartido. Cuando un dispositivo necesita transmitir, espera su intervalo asignado y envía los datos durante ese tiempo.

Las franjas horarias suelen ser de una palabra o un byte, aunque pueden ser más largas según el protocolo. Aunque la velocidad de transmisión instantánea de cada nodo se ve limitada por este método, en la mayoría de sistemas industriales la velocidad global es tan alta que el tiempo añadido no supone una desventaja significativa. TDMA es muy común en redes inalámbricas, sistemas GSM y buses industriales deterministas.

TODO: Añadir diagrama de un ciclo TDMA con cuatro nodos y sus respectivas franjas horarias (slots).

Figura 1.15: Ejemplo de ciclo TDMA con cuatro nodos

1.8.4. Dúplex (Duplexing)

El término **dúplex** se refiere a la direccionalidad de las comunicaciones entre dos dispositivos:

- **Simplex:** comunicación unidireccional. Solo un dispositivo transmite y el otro recibe; no hay respuesta. Ejemplo: un sensor que envía lecturas continuamente a un display.
- **Semidúplex (half-duplex):** ambos dispositivos pueden transmitir y recibir, pero **no simultáneamente**. Comparten un único canal y alternan turnos. Ejemplo: walkie-talkies, RS-485 en modo semidúplex.
- **Dúplex completo (full-duplex):** transmisión y recepción **simultáneas**. En sistemas cableados se requieren dos canales físicos (uno para cada dirección). Ejemplo: UART, RS-232, Ethernet.

En sistemas inalámbricos o con un solo par de conductores, el dúplex completo se puede lograr mediante técnicas de multiplexación como TDMA, donde los slots de transmisión y recepción se alternan a gran velocidad, dando la apariencia de comunicación simultánea.

1.9 El estándar RS-232: comunicación serie industrial

1.9.1. ¿Qué es RS-232?

El estándar **RS-232** (Recommended Standard 232) es una de las interfaces de comunicación serie más antiguas y duraderas de la industria electrónica. Aunque hoy coexisten con USB, Wi-Fi y Ethernet, el RS-232 sigue siendo ampliamente utilizado en entornos industriales, de automatización y de pruebas de equipos.

*La interfaz **RS-232** es un estándar de comunicación serie asíncrona que define la transmisión de datos binarios entre un equipo **DTE** (Data Terminal Equipment, equipo terminal de datos) y un equipo **DCE** (Data Circuit-terminating Equipment, equipo de*

comunicación de datos). Especifica las características eléctricas, mecánicas y funcionales de la conexión.

Ejemplos clásicos de dispositivos:

- **DTE:** ordenadores personales, terminales, PLCs, microcontroladores como el ESP32.
- **DCE:** módems, convertidores de interfaz, dispositivos de medición.

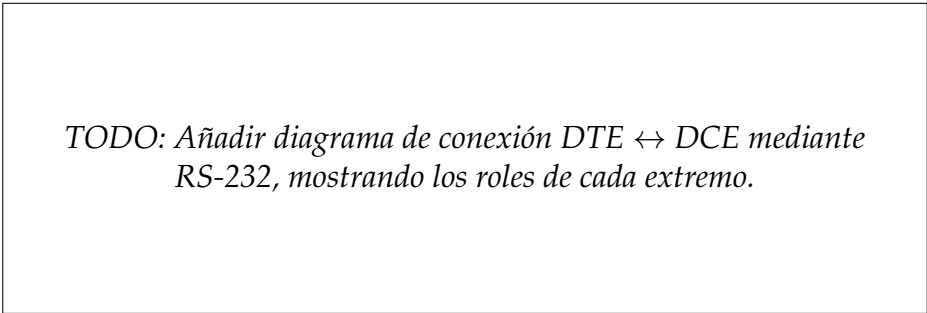


Figura 1.16: Modelo DTE-DCE en una comunicación RS-232

1.9.2. Niveles de tensión: la gran diferencia con UART/TTL

La diferencia más importante entre UART (a nivel TTL) y RS-232 es el **nivel de tensión**:

- **UART/TTL** (0V / 3.3V o 5V): señales de bajo voltaje, idóneas para comunicaciones internas en PCB o cortas distancias.
- **RS-232** ($\pm 3V$ a $\pm 15V$, típicamente $\pm 12V$): señales de mayor amplitud, mucho más inmunes al ruido electromagnético y capaces de recorrer distancias mayores (hasta 15 metros a velocidades moderadas).

Tabla 1.5: Comparativa de niveles de tensión: TTL vs. RS-232

Parámetro	UART/TTL	RS-232
Nivel lógico 0 (espacio)	0V	+3V a +15V
Nivel lógico 1 (marca)	3.3V o 5V	−3V a −15V
Inversión de señal	No	Sí (0 → +V, 1 → −V)
Distancia máxima	<1 metro	Hasta 15 metros
Inmunidad al ruido	Baja	Alta

¡Atención! En RS-232, los niveles están **invertidos** respecto a TTL: un “0” lógico se transmite como voltaje positivo, y un “1” lógico como voltaje negativo. Esto es crucial al interpretar señales en un osciloscopio.

1.9.3. Conectores y pines

RS-232 utiliza conectores tipo D-sub:

- **DB-9:** 9 pines, el más común hoy en día.
- **DB-25:** 25 pines, versión original del estándar.

De los 9 pines del DB-9, solo **tres** son imprescindibles para una comunicación básica:

1. **TXD (Transmit Data, pin 3):** salida de datos desde el DTE.
2. **RXD (Receive Data, pin 2):** entrada de datos al DTE.
3. **GND (Signal Ground, pin 5):** referencia de tierra común.

Los pines restantes se usan para **control de flujo por hardware** (RTS, CTS, DTR, DSR, DCD, RI), aunque en muchas aplicaciones modernas se omiten.

TODO: Añadir diagrama de los pines de un conector DB-9 macho y hembra, señalando TXD, RXD y GND.

Figura 1.17: Distribución de pines en un conector DB-9 para RS-232

1.9.4. Velocidad de subida y limitaciones

El estándar RS-232 especifica una **velocidad de subida (slew rate)** máxima de $30 \text{ V}/\mu\text{s}$. Esta limitación intencionada evita que las transiciones rápidas generen interferencias electromagnéticas en señales adyacentes. Como consecuencia, la velocidad máxima de datos está limitada a aproximadamente **115.2 kb/s** (aunque en la práctica moderna se alcanzan 230.4 kb/s con cables cortos).

1.9.5. Ventajas del RS-232 frente a otros protocolos

- **Mayor resistencia al ruido:** los voltajes de $\pm 12\text{V}$ son mucho más difíciles de alterar por interferencias que los 3.3V de TTL.
- **Distancias más largas:** hasta 15 metros frente a menos de 1 metro en TTL.
- **Compatibilidad universal:** existe hardware de conversión (TTL→RS-232) muy económico y estandarizado.

- **Simplicidad de cableado:** solo dos hilos de datos + tierra para una comunicación básica.

Limitaciones: solo permite comunicación **punto a punto** (un DTE y un DCE), no es adecuado para redes multipunto. Para ello existen RS-485 y CAN bus.

1.10 El protocolo USB: bus serie moderno

1.10.1. Introducción

El **USB (Universal Serial Bus)** es el estándar de comunicación serie más extendido en la actualidad para la interconexión de periféricos con ordenadores y sistemas embebidos. Fue diseñado para sustituir a las interfaces antiguas (como RS-232, puerto paralelo y PS/2) con un único conector versátil, capaz de transmitir datos y alimentar el dispositivo al mismo tiempo.

1.10.2. Características principales

Las principales características del bus USB son:

- **Topología punto a punto:** cada segmento conecta un host (o hub) con un único periférico o hub descendente.
- **Arquitectura maestro-esclavo:** el host (generalmente un PC) es el único maestro que inicia todas las comunicaciones; los periféricos responden únicamente cuando son interrogados.
- **Hasta 127 dispositivos:** mediante el uso de hubs en cascada, un único host puede gestionar hasta 127 periféricos simultáneamente.
- **Plug and Play:** los dispositivos pueden conectarse y desconectarse en caliente sin necesidad de apagar el equipo.
- **Alimentación integrada:** el cable proporciona una tensión nominal de 5 V, lo que permite alimentar periféricos de bajo consumo directamente desde el bus.
- **Velocidades escalables:** desde 1.5 Mb/s (Low Speed) hasta 20 Gb/s (USB4) en sus versiones más recientes.

1.10.3. Interfaz física y niveles eléctricos

El cable USB estándar dispone de **cuatro hilos**:

1. **VBUS** (+5 V, rojo): alimentación para el periférico.
2. **D-** (blanco): línea de datos diferencial negativa.

3. **D+** (verde): línea de datos diferencial positiva.
4. **GND** (negro): referencia de tierra común.

La señal de datos se transmite por un **par trenzado diferencial** (D+ y D-) con una impedancia característica de $90\ \Omega$. La sensibilidad del receptor es de al menos 200 mV y debe soportar un alto factor de rechazo de modo común. La velocidad de transferencia puede ser de 1.5 Mb/s, 12 Mb/s, 480 Mb/s, 5 Gb/s, 10 Gb/s o 20 Gb/s, dependiendo de la versión del estándar.

TODO: Añadir diagrama de un conector USB tipo A con la numeración de pines y colores de cable.

Figura 1.18: Conector USB estándar y asignación de pines

1.10.4. Codificación y sincronización

En USB, el reloj no se transmite por una línea separada, sino que se **recupera de los datos** mediante codificación NRZI. Para garantizar que haya suficientes transiciones y mantener la sincronización, el transmisor inserta **bits de relleno (bit stuffing)** cada vez que detecta seis unos consecutivos. Además, cada paquete comienza con un campo de sincronismo (SYNC) que permite al receptor ajustar su reloj interno.

1.10.5. Terminología USB

A continuación se definen los términos fundamentales que describen los elementos y roles de una arquitectura USB:

- **Host.** Dispositivo maestro que inicia la comunicación (generalmente la computadora).
- **Hub.** Dispositivo que contiene uno o más conectores o conexiones internas hacia otros dispositivos USB, el cual habilita la comunicación entre el host y diversos dispositivos. Cada conector representa un puerto USB.
- **Dispositivo compuesto.** Es aquel dispositivo con múltiples interfaces independientes. Cada una tiene una dirección sobre el bus, pero cada interfaz puede tener un diferente driver de dispositivo en el host.
- **Puerto USB.** Cada host soporta solo un bus; cada conector en el bus representa un puerto USB. Por lo tanto, sobre el bus puede haber varios conectores, pero solo existe una ruta y solo un dispositivo puede transmitir información a un tiempo.

- **Driver.** Es un programa que habilita las aplicaciones para poder comunicarse con el dispositivo. Cada dispositivo sobre el bus debe tener un driver; algunos periféricos utilizan los drivers que trae el sistema operativo.
- **Puntos terminales (Endpoints).** Es una localidad específica dentro del dispositivo. El endpoint es un *buffer* que almacena múltiples bytes, típicamente es un bloque de la memoria de datos o un registro dentro del microcontrolador. Todos los dispositivos deben soportar el punto terminal 0. Este punto terminal es el que recibe todo el control y las peticiones de estado durante la enumeración cuando el dispositivo está sobre el bus.

Tabla 1.6: Comparativa entre RS-232 y USB

Característica	RS-232	USB
Modo de acceso	Punto a punto	Maestro-esclavo (host)
Balanceada	No (single-ended)	Sí (diferencial)
Velocidad máxima	115.2 kb/s	Hasta 20 Gb/s (USB4)
Alimentación del bus	No	Sí (5 V, hasta 500 mA)
Conectores	DB-9, DB-25	Tipo A, Tipo C, Micro-USB
Hot-plug	No	Sí
Número de dispositivos	1 (P2P)	Hasta 127 (con hubs)

1.11 Práctica 1: Comunicación UART con ESP32-S3

1.11.1. Objetivos

La Práctica 1 tiene como objetivos:

1. Aprender a enviar y recibir datos mediante UART en modo asíncrono.
2. Configurar el periférico UART de un microcontrolador (ESP32-S3) mediante su API.
3. Examinar la señal UART física con un osciloscopio para comprender el formato de trama.
4. Implementar una comunicación bidireccional entre un PC y un microcontrolador.

1.11.2. Material utilizado

- **Hardware:** PC, placa de desarrollo ESP32-S3-DevKitC-1, cable USB a TTL, osciloscopio.
- **Software:** Arduino IDE, Python (pyserial), drivers USB-UART.

1.11.3. Marco teórico: protocolo UART

Llegamos al protocolo protagonista de esta unidad y de la Práctica 1. El **UART** (Universal Asynchronous Receiver-Transmitter) es el protocolo de comunicación serie asíncrona más utilizado en electrónica digital y sistemas embebidos.

Descripción del protocolo UART

*Un **UART (Universal Asynchronous Receiver-Transmitter)** es un circuito de hardware que implementa una comunicación serie asíncrona punto a punto, full-duplex, utilizando dos líneas de datos (TX y RX) y un nivel de tensión común de referencia (GND).*

Características principales:

- **Asíncrono:** no requiere línea de reloj compartida. La sincronización se logra mediante bits de inicio y parada.
- **Full-duplex:** ambos dispositivos pueden transmitir y recibir simultáneamente.
- **Punto a punto:** conecta exactamente dos dispositivos.
- **Pines mínimos:** TX (transmisión), RX (recepción) y GND (tierra común).
- **Niveles TTL:** típicamente 0V (lógica 0) y 3.3V o 5V (lógica 1). No confundir con RS-232, que usa voltajes mucho más altos ($\pm 12V$).

TODO: Añadir diagrama de conexión entre dos dispositivos UART: Dispositivo A (TX $\rightarrow$ RX B) y Dispositivo B (TX $\rightarrow$ RX A), con GND común. Indicar "conexión cruzada".

Figura 1.19: Conexión cruzada entre dos dispositivos UART: TX de A con RX de B, y viceversa

Formato de la trama UART

La transmisión de un byte mediante UART sigue un formato estricto:

1. **Estado de reposo (idle):** la línea se mantiene a nivel alto (1) cuando no hay transmisión.
2. **Bit de inicio (START):** una transición a nivel bajo (0) durante un tiempo de bit. Indica al receptor que comienza un nuevo byte.

3. **Bits de datos:** 5, 6, 7 u 8 bits (normalmente 8) que representan el carácter. El bit menos significativo (LSB) se envía primero.
4. **Bit de paridad** (opcional): paridad par (E), impar (O), o sin paridad (N).
5. **Bit(es) de parada (STOP):** nivel alto (1) durante 0.5, 1, 1.5 o 2 tiempos de bit. Marca el final del byte.

TODO: Añadir diagrama de tiempos detallado de una trama UART (ejemplo: 8N1). Mostrar: nivel alto idle, start bit a bajo, bits D0 a D7 (LSB primero), stop bit a alto. Anotar duración de cada segmento en tiempo de bit.

Figura 1.20: Formato de trama UART para la configuración 8N1 (8 bits, sin paridad, 1 bit de parada)

Configuración de una comunicación UART

Una comunicación UART se especifica mediante una cadena de parámetros:

`<velocidad><bits de datos><paridad><bits de parada>`

Ejemplos comunes:

- **9600 8N1:** 9600 baudios, 8 bits de datos, sin paridad, 1 bit de parada.
- **115200 8E1:** 115200 baudios, 8 bits de datos, paridad par (Even), 1 bit de parada.
- **9600 8O1:** 9600 baudios, 8 bits de datos, paridad impar (Odd), 1 bit de parada.

Velocidad de transmisión (baudios): indica el número de bits por segundo. Si el tiempo de bit es T_b , la velocidad en baudios es $1/T_b$. Por ejemplo, si $T_b = 0,1 \mu s$, la velocidad es 10 000 000 baudios (10 Mb/s).

TODO: Añadir fotografía de la pantalla de un osciloscopio mostrando la señal de un byte transmitido por UART. Señalar visualmente el start bit, los bits de datos, y el stop bit.

Figura 1.21: Captura de osciloscopio de una trama UART real (ejemplo del carácter 'A')

Control de flujo por hardware

Cuando dos dispositivos tienen velocidades de procesamiento muy diferentes, el receptor puede no ser capaz de leer los datos tan rápido como el transmisor los envía. Para evitar la pérdida de información, se utiliza el **control de flujo por hardware** mediante dos señales adicionales:

- **RTS (Request To Send)**: el transmisor activa esta señal para indicar que quiere enviar datos.
- **CTS (Clear To Send)**: el receptor activa esta señal para indicar que está listo para recibirlos.

El procedimiento es: Dispositivo A activa RTS → Dispositivo B responde con CTS → Dispositivo A transmite el byte → A desactiva RTS cuando termina → B desactiva CTS si no puede seguir.

TODO: Añadir diagrama de tiempos de las señales RTS y CTS en una comunicación UART con control de flujo.

Figura 1.22: Secuencia de control de flujo por hardware (RTS/CTS) en UART

Panorama de interfaces seriales

El UART es solo una de muchas interfaces serie disponibles. En la siguiente tabla se resumen las más relevantes para el ámbito de la Informática Industrial y Robótica:

1.11.4. Configuración de pines UART en ESP32-S3

El ESP32-S3 dispone de tres UARTs por hardware:

TODO: Añadir fotografía del ESP32-S3-DevKitC-1 señalando los pines UART1 (GPIO 17, 18, 19, 20).

Figura 1.23: Placa ESP32-S3-DevKitC-1 con indicación de los pines UART utilizados

Tabla 1.7: Interfaces seriales más comunes en sistemas embebidos e industriales

Interfaz	Modo	Balanceada	Máx. velocidad	Uso típico
UART	Asíncrono, P2P	No (TTL)	<1 Mb/s	Debug, sensores, GPS
SPI	Síncrono, maestro/esclavo	No	<50 Mb/s	Memoria, pantallas, SD
I2C	Síncrono, multi-esclavo	No	<3.4 Mb/s	Sensores, EEPROM
RS-232	Asíncrono, P2P	No	115.2 kb/s	Módems, periféricos antiguos
RS-485	Asíncrono, bus	Sí (diferencial)	10 Mb/s	Automatización, PLCs
CAN	Asíncrono, bus	Sí (diferencial)	1 Mb/s	Automoción, maquinaria
USB	Síncrono, maestro/esclavo	Sí (diferencial)	20 Gb/s (USB4)	Periféricos, carga

Tabla 1.8: Mapeo de pines UART en ESP32-S3

UART	GPIO RX	GPIO TX	GPIO RTS	GPIO CTS
UART0 (Serial)	44	43	—	—
UART1 (Serial1)	18	17	19	20
UART2 (Serial2)	16	15	—	—

Nota importante: UART0 está conectado internamente al conversor USB-UART de la propia placa, y se usa para programación y depuración (monitor serie del Arduino IDE). Para la comunicación con un PC externo o con otro dispositivo, se utiliza preferentemente UART1.

1.11.5. Desarrollo de la práctica

Primera parte: envío de datos desde PC y desde ESP32

Se desarrollaron dos versiones del programa:

1. **Programa en Python (PC):** abre el puerto COM asignado al cable USB-TTL con la velocidad configurada (ej. 9600 baudios) y envía un mensaje mediante `serial.write()`.
2. **Programa en ESP32 (Arduino IDE):** configura dos puertos serie:
 - `Serial` → UART0 (USB interno), para depuración y monitor.
 - `Serial1` → UART1 (GPIO 17 y 18), para la comunicación externa.

TODO: Añadir captura de pantalla del código Python para envío por serial, o del IDE Arduino con el código ESP32.

Figura 1.24: Código de ejemplo para envío de datos por UART (PC o ESP32)

Segunda parte: visualización con osciloscopio

Se conectó la sonda del osciloscopio al pin TX (GPIO 18 o 17, según el caso) para observar la señal física. Enviando el carácter 'A' (ASCII 0x41 = 01000001 en binario), se observó:

- **Bit de inicio:** bajada de nivel (0 lógico).
- **Bits de datos:** 1 (LSB), 0, 0, 0, 0, 0, 1, 0 (MSB) — enviados en orden LSB primero, por lo que en la línea se lee 10000010 (bit0 a bit7).
- **Bit de parada:** retorno a nivel alto (1 lógico).

TODO: Añadir fotografía de la pantalla del osciloscopio con la señal decodificada del carácter 'A'. Añadir anotaciones sobre los bits.

Figura 1.25: Señal UART del carácter 'A' capturada en osciloscopio

Decodificación manual: la configuración utilizada fue 9600 8N1. Cada bit ocupa aproximadamente $1/9600 \approx 104 \mu\text{s}$. En la pantalla del osciloscopio se puede medir la duración de cada pulso y confirmar que coincide con la velocidad configurada.

Ampliación: comunicación bidireccional PC ↔ ESP32

Para la comunicación entre PC y ESP32 mediante el cable USB-TTL, se realizó el siguiente montaje:

- Cable USB-TTL (verde = TX del cable) → GPIO 17 (RX de ESP32).
- GPIO 18 (TX de ESP32) → cable USB-TTL (blanco = RX del cable).

- GND común entre ambos dispositivos (esencial para la referencia de niveles).

TODO: Añadir fotografía del montaje físico: ESP32 conectada al cable USB-TTL, y este al PC. Mostrar los cables TX, RX y GND conectados.

Figura 1.26: Montaje físico para comunicación bidireccional PC ↔ ESP32 por UART

Problema detectado: en una primera versión, se incluyó un carácter `\n` (salto de línea) al final del mensaje. Esto generó una transición extra no deseada en la señal, visible en el osciloscopio. Al eliminarlo, la trama se ajustó exactamente al formato 8N1.

Problema de buffer de recepción: inicialmente, el código incluía una espera (delay) antes de leer. Esto causaba que varios bytes se acumularan en el buffer RX del ESP32 y se leyeran de golpe. Eliminando la espera, la lectura se hizo casi inmediata y se recibió correctamente un byte a la vez.

TODO: Añadir captura de pantalla del monitor serie del Arduino IDE o terminal Python mostrando el intercambio de datos entre PC y ESP32.

Figura 1.27: Resultado de la comunicación bidireccional PC ↔ ESP32

1.11.6. Relación teoría-práctica

1.12 Práctica 2: Comunicación RS-232 entre ESP32-S3

1.12.1. Objetivos

La Práctica 2 amplía los conocimientos de la Práctica 1 introduciendo el estándar RS-232:

1. Configurar la comunicación serie entre dos placas ESP32-S3 utilizando el protocolo RS-232.

Tabla 1.9: Conexión entre conceptos teóricos y elementos de la Práctica 1

Concepto teórico	Observación/aplicación en la práctica
Transmisión asíncrona	UART no usa reloj compartido; cada byte se sincroniza con su propio start bit
Formato de trama (start/stop)	Visualizado en osciloscopio: bajada (start), 8 bits, subida (stop)
Bit de paridad	Configuración 8N1: sin paridad; se podría cambiar a 8E1 o 8O1
Velocidad (baudios)	9600 baudios = $\sim 104 \mu\text{s}$ por bit; medible en osciloscopio
Conexión P2P	Solo un PC y un ESP32; no es multidrop
Niveles TTL	ESP32 trabaja a 3.3V; el cable USB-TTL adapta a USB
Control de flujo (CTS/RTS)	GPIO 19 y 20 disponibles en ESP32, aunque no usados en esta práctica básica
Conexión cruzada TX $\leftrightarrow$ RX	El TX del cable se conecta al RX del ESP32, y viceversa

2. Observar las diferencias entre señales TTL (UART) y señales RS-232 (niveles de tensión e inversión).
3. Verificar con osciloscopio la transmisión continua de un carácter y analizar su amplitud.
4. Utilizar un cable **null-modem** para la comunicación directa entre dos DTE.

1.12.2. Material utilizado

- **Hardware:** 2 placas ESP32-S3-DevKitC-1, PC, 2 cables USB-TTL, 2 adaptadores TTL a RS-232, 2 adaptadores TTL a RS-485, cable null-modem, osciloscopio.
- **Software:** Arduino IDE.

1.12.3. Desarrollo de la práctica

Primera parte: comunicación directa UART entre dos ESP32

Se estableció una comunicación punto a punto entre dos placas ESP32-S3 mediante el puerto Serial2 (GPIO 16 y 17), conectando TX $\leftrightarrow$ RX cruzado y GND común.

El código utilizado permite enviar o recibir simplemente comentando la parte no deseada:

```

1 #define RXD2 17
2 #define TXD2 18
3
4 void setup() {
5   Serial.begin(600);           // Debug USB
6   Serial2.begin(600, SERIAL_8N1, RXD2, TXD2);
7   Serial.println("ESP32 listo");
8 }
```

```
9
10 void loop() {
11     // Enviar datos
12     Serial2.print("S");
13     Serial.println("S");
14
15     // Leer si hay datos (comentar si solo se envia)
16     /*if (Serial2.available()) {
17         char data = Serial2.read('\n');
18         Serial.print("Recibido: ");
19         Serial.println(data);
20     }*/
21
22     delay(100);
23 }
```

Código 1.1: Código ESP32 para transmisión/recepción por Serial2

TODO: Añadir fotografía del montaje físico: dos ESP32-S3 conectadas entre sí, mostrando el cruce TX-RX y GND común.

Figura 1.28: Montaje físico de dos ESP32-S3 conectadas en comunicación serie directa

Segunda parte: verificación con osciloscopio

Se conectó la sonda del osciloscopio al pin TX para verificar la transmisión continua del carácter 'S' (ASCII 83 = 0x53 = 01010011 en binario).

Análisis de la trama observada:

- El carácter 'S' tiene valor binario: 01010011 (MSB → LSB).
- En UART, el **LSB se envía primero**, por lo que el orden de transmisión es: 11001010.
- La trama completa es: **Start bit** (0) + 11001010 + **Stop bit** (1).

La señal observada en el osciloscopio coincidió exactamente con esta secuencia teórica.

TODO: Añadir fotografía de la pantalla del osciloscopio mostrando la señal del carácter 'S' con los bits anotados.

Figura 1.29: Señal del carácter 'S' en osciloscopio (niveles TTL)

Tercera parte: análisis de amplitud de la señal

Se midieron los niveles de tensión de la señal TTL:

- **Nivel alto:** aproximadamente 2,90 V
- **Nivel bajo:** aproximadamente 0,70 V
- **Amplitud pico a pico:** $\approx 2,20$ V

En condiciones ideales, una ESP32-S3 debería producir niveles de 0 V (bajo) y 3,3 V (alto). Las pequeñas desviaciones observadas se explican por:

- Efecto de carga de la sonda del osciloscopio sobre el circuito.
- Caídas de tensión en el cableado.
- Referencia de masa no completamente estable.
- Ruido eléctrico ambiental, especialmente visible en los flancos.

TODO: Añadir fotografía de la pantalla del osciloscopio mostrando las medidas de tensión (V_{high} , V_{low} , V_{pp}).

Figura 1.30: Medida de amplitud de la señal TTL en osciloscopio

Cuarta parte: uso del cable null-modem

Un cable **null-modem** es un cable especial que cruza internamente las líneas TX y RX, permitiendo conectar dos equipos **DTE** (como dos ESP32) directamente sin necesidad de un DCE intermedio.

El montaje consiste en:

- Conectar los adaptadores TTL→RS-232 a cada ESP32.
- Unir ambos extremos mediante el cable null-modem.
- La conexión cruzada TX↔RX se realiza internamente en el cable.

Se comprobó que la comunicación funcionaba correctamente con el cable null-modem, y la señal observada en el osciloscopio coincidió con la obtenida anteriormente.

TODO: Añadir fotografía del montaje con adaptadores TTL-RS-232 y cable null-modem.

Figura 1.31: Montaje con adaptadores TTL-RS-232 y cable null-modem

1.12.4. Relación teoría-práctica: comparación UART vs. RS-232

Tabla 1.10: Comparación entre Práctica 1 (UART/TTL) y Práctica 2 (RS-232)

Aspecto	Práctica 1 (UART/TTL)	Práctica 2 (RS-232)
Nivel lógico 0	0V	+3V a +15V (típ. +12V)
Nivel lógico 1	3.3V	−3V a −15V (típ. −12V)
Inversión de señal	No	Sí
Distancia	<1 metro (USB-TTL)	Hasta 15 metros
Inmunidad al ruido	Baja	Alta
Hardware necesario	Cable USB-TTL	Adaptador TTL→RS-232
Conexión entre dos DTE	Cruzar TX-RX manualmente	Null-modem (cruce interno)

1.13 Resumen de la Unidad 1

- Las **comunicaciones en serie** han sustituido a las paralelas en la mayoría de aplicaciones por su menor coste, mayor fiabilidad y capacidad para largas distancias.
- La **conexión diferencial** (balanceada) cancela el ruido común y es el estándar en entornos industriales.
- La **sincronización** puede ser asíncrona (UART), síncrona con reloj separado (SPI) o mediante recuperación desde los datos (CDR en USB/Ethernet).
- Los **protocolos** empaquetan los datos en tramas con delimitadores, direcciones y códigos de detección de errores.

- La **detección de errores** evoluciona desde el simple bit de paridad hasta el robusto CRC, pasando por el BCC.
- El **UART** es el protocolo asíncrono básico más usado en sistemas embebidos; su formato 8N1 es la configuración estándar.
- El estándar **RS-232** extiende el UART a niveles de tensión industriales ($\pm 12V$), permitiendo distancias mayores y mayor inmunidad al ruido.
- La **Práctica 1** permite observar directamente la señal UART, confirmar el formato de trama, y establecer una comunicación real entre PC y microcontrolador.
- La **Práctica 2** demuestra la diferencia entre niveles TTL y RS-232, el uso de adaptadores, y la comunicación punto a punto entre dos microcontroladores.

TODO: Añadir esquema resumen/mapa conceptual de la Unidad 1: comunicación serie, medios, sincronización, UART, práctica.

Figura 1.32: Mapa conceptual resumen de la Unidad 1

CAPÍTULO 2

Redes Inalámbricas y Comunicaciones IoT

En los capítulos anteriores hemos estudiado cómo transmitir datos mediante cables: señales eléctricas que viajan por hilos de cobre siguiendo reglas estrictas como UART o RS-232. Sin embargo, en muchos entornos industriales y de automatización resulta inviable o poco práctico tender cables entre cada dispositivo. Imagina un almacén robotizado donde cientos de sensores de movimiento deben comunicarse con una central, o una fábrica de alimentos donde los cables dificultan la higiene.

En este capítulo abordamos las **comunicaciones inalámbricas**, que utilizan ondas de radio en lugar de conductores físicos para transmitir información. Nos centraremos en las dos tecnologías más utilizadas en sistemas embebidos: **Bluetooth Low Energy** (BLE) y **Wi-Fi**, y veremos cómo aplicarlas en la práctica para construir una red de sensores IoT (Internet de las Cosas).

2.1 Comunicaciones inalámbricas: ¿por qué eliminar los cables?

La tendencia en la industria 4.0 es hacia la **desconexión física**: dispositivos que se comunican sin cables, que se autoconfiguran y que pueden desplazarse. Algunas ventajas de las comunicaciones inalámbricas son:

- **Flexibilidad de instalación:** no es necesario diseñar canalizaciones ni prever la longitud exacta de los cables.
- **Movilidad:** robots, carretillas elevadoras o sensores portátiles pueden cambiar de ubicación sin restringirse por el cableado.
- **Reducción de costes de mantenimiento:** los cables físicos se degradan con el tiempo, especialmente en entornos con vibraciones, humedad o temperaturas extremas.
- **Escalabilidad:** es más sencillo añadir nuevos dispositivos a una red inalámbrica existente que tender nuevos cables.

Sin embargo, también existen inconvenientes:

- **Interferencias:** otras redes, microondas o motores eléctricos pueden degradar la señal.
- **Seguridad:** las ondas de radio pueden ser interceptadas si no se cifran adecuadamente.
- **Consumo energético:** transmitir por radio consume más energía que enviar señales por un hilo.
- **Alcance limitado:** la potencia de transmisión está regulada y, en interiores, las paredes atenúan la señal.

2.2 Bluetooth vs. Wi-Fi: dos filosofías, una misma banda

Tanto Bluetooth como Wi-Fi operan en la banda de radio de **2.4 GHz** (aunque Wi-Fi también puede usar 5 GHz y 6 GHz en versiones modernas). A pesar de compartir el mismo espectro, están diseñados para propósitos muy diferentes.

*Bluetooth y Wi-Fi **Bluetooth** es un estándar de comunicación inalámbrica de corto alcance y bajo consumo, pensado para crear redes de área personal (PAN) entre dispositivos próximos.*

***Wi-Fi** es un estándar de comunicación inalámbrica de mayor alcance y velocidad, diseñado para crear redes de área local (LAN) que permiten a múltiples dispositivos acceder a internet o compartir recursos de forma simultánea.*

Tabla 2.1: Comparación entre Bluetooth y Wi-Fi

Característica	Bluetooth (BLE)	Wi-Fi
Alcance típico	10–100 m (según clase)	30–100 m (interior)
Velocidad de datos	Hasta 2 Mb/s (BLE 5.0)	Hasta varios Gb/s (Wi-Fi 6)
Consumo de energía	Muy bajo (meses con pila)	Alto (necesita alimentación)
Topología típica	Punto a punto (1 maestro, 7 esclavos)	Estrella (AP + múltiples clientes)
Uso principal	Periféricos, sensores, wearables	Acceso a internet, streaming, redes corporativas
Complejidad de protocolo	Simple (GATT)	Compleja (TCP/IP, HTTP, etc.)
Interferencias en 2.4 GHz	Alta (comparte banda con Wi-Fi)	Alta (comparte banda con Bluetooth)

¿Cuándo usar cada uno?

- Usa **Bluetooth** cuando necesites conectar un sensor o un periférico a un dispositivo central, con bajo consumo y sin necesidad de acceso a internet.

Ejemplos: auriculares inalámbricos, pulsómetros, sensores de temperatura portátiles.

- Usa **Wi-Fi** cuando el dispositivo necesite enviar datos a un servidor en la nube, recibir comandos remotos o formar parte de una red corporativa. Ejemplos: cámaras de videovigilancia, termostatos inteligentes, paneles de control.

2.3 Bluetooth Low Energy (BLE)

2.3.1. ¿Qué es BLE?

Bluetooth Low Energy (BLE), también conocido como Bluetooth Smart, es una versión del estándar Bluetooth optimizada para un consumo de energía mínimo. No es compatible a nivel de protocolo con el Bluetooth clásico (el de los auriculares), pero comparte la misma banda de radio.

Bluetooth Low Energy (BLE) BLE es un protocolo de comunicación inalámbrica de corto alcance que permite la transmisión de pequeños paquetes de datos con un consumo de energía extremadamente bajo. Es ideal para dispositivos alimentados por baterías que deben operar durante meses o años sin recargarse.

En BLE, los dispositivos desempeñan dos roles principales:

- **Periférico (Peripheral)**: dispositivo que ofrece datos. Suele ser el sensor o el actuador.
- **Central (Central)**: dispositivo que solicita y recibe datos. Suele ser un teléfono, un PC o un gateway IoT.

2.3.2. Arquitectura GATT: servicios y características

BLE organiza la información mediante una estructura jerárquica conocida como **GATT** (Generic Attribute Profile):

1. **Perfil (Profile)**: conjunto de servicios predefinidos para un uso específico (por ejemplo, un perfil de pulsómetro).
2. **Servicio (Service)**: agrupación lógica de características. Por ejemplo, un “servicio de temperatura”.
3. **Característica (Characteristic)**: el dato concreto. Puede ser una temperatura, una cadena de texto o un valor de configuración.
4. **Descriptor (Descriptor)**: metadatos adicionales sobre una característica, como su formato o unidades.

Cada servicio y característica se identifica mediante un **UUID** (Universally Unique Identifier), un código de 128 bits que garantiza que no haya colisiones entre aplicaciones diferentes.

*UUID en BLE Un **UUID** (Universally Unique Identifier) es un identificador numérico de 128 bits que se utiliza para distinguir de forma única servicios, características y descriptores dentro de una aplicación BLE. Los UUIDs pueden ser estandarizados (16 o 32 bits) o customizados por el desarrollador (128 bits).*

2.3.3. Advertising y escaneo

Un periférico BLE no puede ser conectado directamente. Primero debe anunciar su presencia mediante un proceso llamado **advertising** (difusión): envía paquetes periódicos de anuncio que contienen su nombre, el UUID de sus servicios y otros datos.

Una central que desea conectarse realiza un **escaneo** (scanning): escucha los paquetes de anuncio hasta encontrar un dispositivo con el UUID que busca. Una vez encontrado, se establece la conexión.

2.3.4. Notificaciones y lecturas

Las características BLE pueden tener diferentes propiedades:

- **READ**: la central puede leer el valor explícitamente.
- **WRITE**: la central puede escribir un valor.
- **NOTIFY**: el periférico envía el valor automáticamente cada vez que cambia, sin que la central tenga que pedirlo.

Las notificaciones son especialmente útiles en IoT: el sensor envía una nueva lectura de temperatura cada vez que está disponible, sin que el receptor tenga que estar constantemente preguntando.

2.4 Wi-Fi en sistemas embebidos

2.4.1. Modos de operación

Un dispositivo Wi-Fi puede funcionar en dos modos principales:

- **STA (Station)**: el dispositivo se conecta a un punto de acceso (AP) existente, como un router. Es el modo más común para sensores IoT.
- **AP (Access Point)**: el dispositivo crea su propia red Wi-Fi a la que otros dispositivos pueden conectarse. Útil para configuración inicial.

2.4.2. Protocolo HTTP para IoT

En la práctica, cuando un dispositivo embebido con Wi-Fi necesita comunicarse, lo hace a través del protocolo **HTTP** (HyperText Transfer Protocol), el mismo que utiliza un navegador web para cargar páginas.

*HTTP en IoT HTTP es el protocolo de comunicación que permite a un cliente (por ejemplo, una ESP32 o un navegador) solicitar recursos a un servidor. En el contexto IoT, un sensor puede actuar como **servidor HTTP**, respondiendo a peticiones con datos en formato texto, JSON o XML.*

El flujo típico es:

1. El dispositivo sensor se conecta a la red Wi-Fi (modo STA).
2. Se le asigna una dirección IP por DHCP.
3. Inicia un servidor HTTP en un puerto (normalmente el 80).
4. Define rutas, como /temperatura, que devuelven el valor del sensor.
5. Un cliente (otra ESP32, un PC o un smartphone) realiza una petición GET a esa IP y ruta, y recibe el dato.

2.4.3. Redes corporativas y seguridad

En entornos universitarios o industriales, las redes Wi-Fi suelen estar protegidas mediante autenticación (WPA2-PSK, WPA2-Enterprise) o listas de control de acceso (MAC filtering). Esto puede complicar la conexión de dispositivos IoT, ya que no todos los sistemas embebidos soportan los métodos de autenticación más avanzados.

En la práctica, una solución común cuando la red corporativa es problemática es utilizar un **hotspot móvil** (punto de acceso creado por un teléfono), que permite controlar directamente qué dispositivos se conectan.

2.5 Sensor de temperatura y humedad DHT11

El **DHT11** es un sensor digital de bajo coste que mide temperatura y humedad relativa del aire. Es muy popular en proyectos educativos y de prototipado porque su conexión es simple (solo necesita un pin de datos y alimentación) y proporciona valores directamente en formato digital, sin necesidad de conversión analógica.

DHT11 El DHT11 es un sensor digital compuesto que incluye un elemento resistivo para medir humedad y un termistor para medir temperatura. Su resolución es de 1 grado Celsius y 1 % de humedad relativa, con un rango de 0 a 50 grados Celsius. El tiempo mínimo entre lecturas es de aproximadamente 1 segundo.

Aunque el DHT11 no es el sensor más preciso del mercado (el DHT22 o el SHT30 son más exactos), su simplicidad y precio lo hacen ideal para aprender los fundamentos de la adquisición de datos y la transmisión de sensores.

2.6 Práctica 3: Comunicaciones Inalámbricas

2.6.1. Objetivos

El objetivo de esta práctica es diseñar e implementar dos aplicaciones que transmitan la temperatura ambiente del laboratorio a una red:

1. Una aplicación basada en **Bluetooth BLE** que permita la comunicación directa entre dos placas ESP32-S3 sin cables.
2. Una aplicación basada en **Wi-Fi** que permita acceder a la temperatura mediante una petición HTTP desde cualquier dispositivo conectado a la misma red.

2.6.2. Material utilizado

- **Hardware:** Ordenador, dos placas ESP32-S3-DevKitC-1 N16R8, un sensor de temperatura DHT11.
- **Software:** Arduino IDE con las librerías DHT, BLEDevice y WebServer.
- **Red:** Conexión a la red inalámbrica UPV-PSK (o un hotspot móvil como alternativa).

2.6.3. Desarrollo de la práctica

Primera parte: lectura del sensor DHT11

Antes de enviar datos por el aire, es necesario obtenerlos de forma fiable. El primer programa tiene como objetivo leer la temperatura del sensor DHT11 conectado al pin 2 de la ESP32 y mostrarla por el puerto serie.

El código es sencillo: se inicializa el sensor, se realiza una lectura cada 2 segundos y se comprueba que el valor sea válido (el DHT11 puede devolver errores ocasionales si no se respeta el tiempo de muestreo).

```
1 #include <DHT.h>
2
3 #define DHTPIN 2
4 #define DHTTYPE DHT11
5
6 DHT dht(DHTPIN, DHTTYPE);
7
8 void setup() {
9     Serial.begin(115200);
10    dht.begin();
```

```
11 Serial.println("Sensor DHT11 inicializado.");
12 }
13
14 void loop() {
15     float temperatura = dht.readTemperature();
16
17     if (isnan(temperatura)) {
18         Serial.println("Error leyendo el DHT11");
19     } else {
20         Serial.print("Temperatura: ");
21         Serial.print(temperatura);
22         Serial.println(" C");
23     }
24     delay(2000);
25 }
```

Código 2.1: Lectura del sensor DHT11

TODO: Anadir fotografia del montaje fisico con la ESP32-S3 y el sensor DHT11 conectado.

Figura 2.1: Montaje físico del sensor DHT11 con la ESP32-S3

Segunda parte: servidor BLE

Una vez confirmado que el sensor funciona, se implementa un **servidor BLE** sobre una de las ESP32-S3. Este servidor crea un servicio de temperatura con dos características:

- **Característica 1:** envía la temperatura como valor numerico (float).
- **Característica 2:** envía la temperatura como cadena de texto descriptiva.

Ambas características tienen notificaciones activadas, lo que significa que cualquier cliente conectado recibirá los datos automáticamente sin tener que solicitarlos.

El servidor también gestiona la reconexión: si el cliente se desconecta, la ESP32 vuelve a anunciarse (advertising) para aceptar nuevas conexiones.

```
1 // UUIDs del servicio y características
2 #define SERVICE_UUID "8628b276-4bba-420a-ab99-dd69945408bf"
3 #define CHARACTERISTIC_UUID_1 "6cb1114c-4410-4757-85af-b064b3cc689a"
4 #define CHARACTERISTIC_UUID_2 "a1642e22-4f4e-41de-b173-b5bdb099f12c"
5
6 // Inicializar BLE
7 BLEDevice::init("ESP32_Temperatura");
8 pServer = BLEDevice::createServer();
```

```

9 pServer->setCallbacks(new MyServerCallbacks());
10
11 BLEService *pService = pServer->createService(SERVICE_UUID);
12
13 // Característica numerica con notificaciones
14 pCharacteristic_1 = pService->createCharacteristic(
15     CHARACTERISTIC_UUID_1,
16     BLECharacteristic::PROPERTY_NOTIFY
17 );
18 pBLE2902_1 = new BLE2902();
19 pBLE2902_1->setNotifications(true);
20 pCharacteristic_1->addDescriptor(pBLE2902_1);
21
22 pService->start();
23 BLEDevice::startAdvertising();

```

Código 2.2: Servidor BLE de temperatura (extracto)

En el bucle principal, si hay un cliente conectado, el servidor lee el DHT11 y envía el valor por ambas características:

```

1 if (deviceConnected) {
2     float temperatura = dht.readTemperature();
3     if (isnan(temperatura)) {
4         Serial.println("Error leyendo el DHT11");
5         delay(2000);
6         return;
7     }
8
9     // Enviar como valor numerico
10    pCharacteristic_1->setValue(temperatura);
11    pCharacteristic_1->notify();
12
13    // Enviar como texto descriptivo
14    String mensaje = "Temperatura actual: " + String(temperatura) + " C";
15    pCharacteristic_2->setValue(mensaje.c_str());
16    pCharacteristic_2->notify();
17
18    delay(2000);
19 }

```

Código 2.3: Envío de temperatura por BLE

Tercera parte: cliente BLE

La segunda ESP32-S3 actúa como **cliente BLE**. Su misión es escanear el entorno buscando un dispositivo que anuncie el SERVICE_UUID del servidor, conectarse a él, suscribirse a las notificaciones de una de las características y mostrar el dato recibido por el puerto serie.

```

1 // UUIDs (iguales que en el servidor)
2 #define SERVICE_UUID "8628b276-4bba-420a-ab99-dd69945408bf"
3 #define CHARACTERISTIC_UUID "a1642e22-4f4e-41de-b173-b5bdb099f12c"
4
5 // Callback cuando se detecta el servidor
6 class MyAdvertisedDeviceCallbacks: public BLEAdvertisedDeviceCallbacks
7 {

```



```

7  void onResult(BLEAdvertisedDevice advertisedDevice) {
8      if (advertisedDevice.haveServiceUUID() &&
9          advertisedDevice.isAdvertisingService(BLEUUID(SERVICE_UUID))) {
10         Serial.println("Servidor encontrado");
11         pServerAddress = new BLEAddress(advertisedDevice.getAddress());
12         doConnect = true;
13         advertisedDevice.getScan()->stop();
14     }
15 }
16 };
17
18 // Callback cuando llegan notificaciones
19 static void notifyCallback(
20     BLERemoteCharacteristic* pBLERemoteCharacteristic,
21     uint8_t* pData, size_t length, bool isNotify) {
22     String mensaje = "";
23     for (int i = 0; i < length; i++) {
24         mensaje += (char)pData[i];
25     }
26     Serial.print("Mensaje recibido: ");
27     Serial.println(mensaje);
28 }

```

Código 2.4: Cliente BLE de temperatura (extracto)

TODO: Anadir captura de pantalla del monitor serie del cliente BLE mostrando la temperatura recibida.

Figura 2.2: Monitor serie del cliente BLE recibiendo temperatura

Cuarta parte: servidor y cliente Wi-Fi

La alternativa a BLE es utilizar **Wi-Fi** para crear una red de sensores mas abierta. En este caso, la ESP32-S3 que lleva el DHT11 se conecta a la red UPV-PSK y actua como **servidor web HTTP** en el puerto 80.

Cada vez que un cliente visita la URL `http://IP_DEL_ESP32/temperatura`, el servidor lee el sensor y responde con el valor de temperatura como texto plano.

```

1  #include <WiFi.h>
2  #include <WebServer.h>
3  #include <DHT.h>
4
5  #define DHTPIN 2
6  #define DHTTYPE DHT11
7  DHT dht(DHTPIN, DHTTYPE);
8
9  const char* ssid = "UPV-PSK";
10 const char* password = "C0n3ct4nd0s3_m0l4!";

```

```

11 | WebServer server(80);
12 |
13 | void handleTemperatura() {
14 |     float temperatura = dht.readTemperature();
15 |     if (isnan(temperatura)) {
16 |         server.send(500, "text/plain", "Error leyendo sensor");
17 |         return;
18 |     }
19 |     server.send(200, "text/plain", String(temperatura));
20 | }
21 |
22 | void setup() {
23 |     Serial.begin(115200);
24 |     dht.begin();
25 |     WiFi.begin(ssid, password);
26 |     while (WiFi.status() != WL_CONNECTED) {
27 |         delay(500);
28 |         Serial.print(".");
29 |     }
30 |     Serial.println("\nConectado!");
31 |     Serial.print("IP del servidor: ");
32 |     Serial.println(WiFi.localIP());
33 |
34 |     server.on("/temperatura", handleTemperatura);
35 |     server.begin();
36 | }
37 |
38 | void loop() {
39 |     server.handleClient();
40 | }

```

Código 2.5: Servidor Wi-Fi de temperatura

Por otro lado, la segunda ESP32 (o cualquier ordenador) actúa como **cliente HTTP**. Se conecta a la misma red Wi-Fi, construye la URL a partir de la IP del servidor, realiza una petición GET cada 2 segundos y muestra la respuesta.

```

1 | #include <WiFi.h>
2 | #include <HTTPClient.h>
3 |
4 | const char* ssid = "UPV-PSK";
5 | const char* password = "C0n3ct4nd0s3_m0l4!";
6 | const char* serverIP = "192.168.1.42";
7 |
8 | void setup() {
9 |     Serial.begin(115200);
10 |    WiFi.begin(ssid, password);
11 |    while (WiFi.status() != WL_CONNECTED) {
12 |        delay(500);
13 |        Serial.print(".");
14 |    }
15 |    Serial.println("\nConectado a la red WiFi");
16 | }
17 |
18 | void loop() {
19 |     if (WiFi.status() == WL_CONNECTED) {
20 |         HTTPClient http;
21 |         String url = String("http://") + serverIP + "/temperatura";
22 |         http.begin(url);

```

```
23  int httpResponseCode = http.GET();
24
25  if (httpResponseCode > 0) {
26    String temperatura = http.getString();
27    Serial.print("Temperatura recibida: ");
28    Serial.println(temperatura + " C");
29  } else {
30    Serial.print("Error en la conexi3n: ");
31    Serial.println(httpResponseCode);
32  }
33  http.end();
34 }
35 delay(2000);
36 }
```

C3digo 2.6: Cliente Wi-Fi de temperatura

TODO: Anadir captura de pantalla del monitor serie del cliente Wi-Fi mostrando la temperatura recibida.

Figura 2.3: Monitor serie del cliente Wi-Fi recibiendo temperatura via HTTP

2.6.4. Problemas encontrados

Durante la practica, la conexi3n a la red UPV-PSK resulto en ocasiones bastante complicada. Frecuentemente aparecian **timeouts** que obligaban a reiniciar el proceso de conexi3n. Tras analizar la situaci3n, se concluyo que la red tiene un sistema de **control de acceso** que mantiene una lista de dispositivos admitidos. Algunos equipos quedaban en espera dentro de esta lista, lo que impedia que se conectaran correctamente.

Como **alternativa**, se opto por utilizar el **hotspot de un telefono movil**. Esto permitio controlar directamente que dispositivos podian conectarse, agilizando el proceso y asegurando que la practica pudiera realizarse sin depender de las limitaciones de la red de la universidad. Esta soluci3n demostro ser mas fiable y rapida para el desarrollo de los experimentos.

2.6.5. Relacion teoria-practica: BLE vs. Wi-Fi en la practica

2.7 Resumen de la Unidad 2

- Las **comunicaciones inalamblicas** eliminan la necesidad de cableado fisico, ofreciendo flexibilidad y movilidad, aunque introducen desafios como interferencias, seguridad y consumo energetico.

Tabla 2.2: Conexion entre conceptos teoricos y elementos de la Practica 3

Concepto teorico	Observacion/aplicacion en la practica
Bluetooth BLE	Se implemento un servidor/periférico BLE con servicio de temperatura y un cliente/central que recibe notificaciones
UUID (GATT)	Cada servicio y caracteristica tiene un UUID unico de 128 bits para identificarlos sin ambigüedad
Advertising	El servidor BLE envia paquetes de anuncio para que el cliente pueda descubrirlo
Notificaciones BLE	La temperatura se envia automaticamente al cliente sin que este tenga que solicitarla explicitamente
Wi-Fi (modo STA)	La ESP32 se conecta a la red UPV-PSK como estacion, obteniendo una IP por DHCP
Servidor HTTP	La ESP32 actua como servidor web en el puerto 80, respondiendo peticiones GET en la ruta /temperatura
Cliente HTTP	La segunda ESP32 realiza peticiones GET periodicas para obtener la temperatura del servidor
Red corporativa IoT	Se evidencio el problema del control de acceso en redes universitarias, resuelto con un hotspot alternativo
Sensor DHT11	Proporciona el dato de temperatura real que alimenta tanto al servidor BLE como al servidor Wi-Fi
Consumo energetico	BLE consume menos que Wi-Fi, aunque en esta practica ambas placas estaban alimentadas por USB

- **Bluetooth y Wi-Fi** comparten la banda de 2.4 GHz pero estan disenados para propósitos distintos: BLE es ideal para sensores de bajo consumo a corta distancia, mientras que Wi-Fi proporciona mayor alcance y velocidad para acceso a redes y a internet.
- **BLE** organiza los datos mediante el perfil GATT, utilizando servicios y caracteristicas identificados por UUIDs. Las notificaciones permiten enviar datos de forma automatica sin polling.
- **Wi-Fi** en sistemas embebidos permite crear servidores HTTP que exponen datos de sensores a traves de peticiones web estandar, facilitando la integracion con aplicaciones y plataformas en la nube.
- La **Practica 3** demuestra la viabilidad de ambas tecnologias para transmitir datos de temperatura en tiempo real, comparando el enfoque punto a punto de BLE con el enfoque cliente-servidor web de Wi-Fi.
- Las **redes corporativas IoT** pueden presentar barreras de acceso (control de dispositivos, timeouts) que requieren soluciones alternativas como hotspots moviles para el desarrollo de prototipos.

CAPÍTULO 3

Redes de Area Local y Encaminamiento

Hasta ahora hemos estudiado como comunicar dos dispositivos mediante cables (UART, RS-232) y mediante ondas de radio (Bluetooth, Wi-Fi). Sin embargo, en un entorno industrial real no hay solo dos dispositivos: hay cientos de ordenadores, PLCs, sensores, servidores y camaras, todos conectados entre si y con la necesidad de compartir informacion.

En este capitulo abordamos las **redes de computadores** desde una perspectiva arquitectonica: como se organizan fisicamente, como se identifican los dispositivos, como se toman las decisiones de donde enviar los paquetes de datos y que dispositivos permiten que una red se conecte con otra.

3.1 Redes de computadores: mas alla de dos dispositivos

Una **red de computadores** es un conjunto de dispositivos (hosts o sistemas terminales) interconectados mediante enlaces de comunicacion y dispositivos de conmutacion, que siguen unos protocolos comunes para intercambiar informacion.

*Red de computadores Una **red de computadores** es un conjunto de sistemas terminales (hosts) interconectados mediante enlaces de comunicacion y dispositivos de conmutacion (switches, routers), que intercambian datos siguiendo unos protocolos predefinidos.*

Los componentes esenciales son:

- **Hosts:** ordenadores, servidores, PLCs, smartphones... cualquier dispositivo que genera o consume datos.
- **Enlaces de comunicacion:** cables (par trenzado, coaxial, fibra optica) o medios inalambricos (ondas de radio, Wi-Fi).
- **Dispositivos de conmutacion:** switches y routers que reenvian los paquetes hacia su destino.

3.1.1. Medios de transmision

Los enlaces de comunicacion pueden clasificarse en:

- **Medios guiados:** la senal viaja confinada en un conductor fisico.
 - **Par trenzado:** dos cables de cobre aislados y trenzados. Barato, muy usado en Ethernet. Categorias: 5e, 6, 6a, 7...
 - **Cable coaxial:** dos conductores concentricos. Usado en redes cableadas antiguas y en television por cable.
 - **Fibra optica:** transmision de luz a traves de vidrio o plastico. Alta velocidad, baja tasa de error, inmune al ruido electromagnetico. Ideal para redes industriales y troncales de larga distancia.
- **Medios no guiados:** la senal se propaga por el aire o el vacio.
 - **Ondas de radio:** Wi-Fi, Bluetooth, 4G/5G. Sensible a interferencias, absorcion por objetos y efectos ambientales.

3.2 El modelo TCP/IP

Para que todos los dispositivos de una red se entiendan, es necesario un modelo de capas que organice las funciones de comunicacion. El modelo **TCP/IP** es el estandar de Internet y consta de cuatro capas:

Tabla 3.1: Las cuatro capas del modelo TCP/IP

Capa	Funcion
Aplicacion	Interfaz con el usuario. Protocolos: HTTP, FTP, DNS, SMTP...
Transporte	Entrega fiable o rapida de datos. Protocolos: TCP (fiable), UDP (rapido).
Internet (Red)	Direccionamiento y encaminamiento de paquetes. Protocolo: IP.
Enlace de datos	Transmision fisica por el medio (cable, fibra, radio). Ethernet, Wi-Fi...

3.2.1. TCP frente a UDP

En la capa de transporte, los dos protocolos mas importantes son:

*TCP y UDP **TCP** (Transmission Control Protocol) es un protocolo **orientado a conexion** que garantiza la entrega fiable de datos, controlando errores, reordenando paquetes y solicitando retransmisiones si es necesario.*

***UDP** (User Datagram Protocol) es un protocolo **sin conexion** que envia datos sin garantizar su entrega ni orden. Es mas rapido que TCP y se usa en aplicaciones donde la velocidad es prioritaria (streaming, juegos en linea, DNS).*

Tabla 3.2: Comparacion entre TCP y UDP

Caracteristica	TCP	UDP
Conexion	Orientado a conexion	Sin conexion
Fiabilidad	Garantiza entrega y orden	No garantiza nada
Velocidad	Mas lento (overhead de control)	Mas rapido
Control de errores	Si (checksum, ACK, retransmission)	No
Uso tipico	Web, correo, transferencia de archivos	Streaming, juegos, DNS, VoIP

3.3 Direccionamiento IP

Cada dispositivo conectado a una red necesita un identificador unico: la **direccion IP** (Internet Protocol). En IPv4, es un numero de 32 bits representado en notacion decimal con puntos (por ejemplo, 192.168.1.10).

*Direccion IP Una **direccion IP** es un identificador numerico de 32 bits (IPv4) o 128 bits (IPv6) asignado a cada dispositivo conectado a una red. Permite localizar el dispositivo dentro de la red y enrutar los paquetes hacia el.*

3.3.1. Estructura de una direccion IPv4

Una direccion IPv4 se divide en dos partes:

- **Parte de red:** identifica la red a la que pertenece el dispositivo.
- **Parte de host:** identifica el dispositivo concreto dentro de esa red.

La separacion entre red y host viene dada por la **mascara de subred** (subnet mask), tambien de 32 bits. Los bits a 1 en la mascara indican la parte de red; los bits a 0, la parte de host.

Por ejemplo, con la mascara 255.255.255.0 (/24), los primeros 24 bits (3 octetos) son la red y el ultimo octeto es el host.

3.3.2. Clases de direcciones IP

Historicamente, las direcciones IP se dividieron en clases para facilitar su asignacion:

Direcciones privadas (no enrutables en Internet):

- Clase A: 10.0.0.0 – 10.255.255.255
- Clase B: 172.16.0.0 – 172.31.255.255
- Clase C: 192.168.0.0 – 192.168.255.255

Tabla 3.3: Clases de direcciones IPv4

Clase	Rango del primer octeto	Uso
A	1 – 126	Redes muy grandes (16 millones de hosts)
B	128 – 191	Redes medianas (65.000 hosts)
C	192 – 223	Redes pequenas (254 hosts)
D	224 – 239	Multicast (grupos de receptores)
E	240 – 255	Reservado para investigacion

3.3.3. Gateway (puerta de enlace)

El **gateway** o puerta de enlace es la direccion IP del dispositivo que conecta una red local con el exterior (otra red o Internet). Cuando un host quiere enviar un paquete a una direccion que no pertenece a su propia red, lo envia al gateway, que se encarga de encaminarlo.

*Gateway (puerta de enlace) El **gateway** es la direccion IP del dispositivo de interconexion (router) que actua como salida de una red local. Todos los hosts de la red deben conocer la direccion del gateway para poder comunicarse con dispositivos externos.*

3.4 Dispositivos de interconexion: switch y router

3.4.1. Switch (conmutador)

Un **switch** es un dispositivo que opera en la capa de enlace de datos (Capa 2 del modelo OSI). Su funcion es conectar multiples dispositivos dentro de una misma red local (LAN), reenviando los paquetes de datos solo al puerto al que esta conectado el destinatario.

*Switch Un **switch** es un dispositivo de interconexion de red que opera en la capa de enlace de datos. Conecta multiples dispositivos dentro de una misma red local y reenvia los paquetes unicamente al puerto del destinatario, mejorando la eficiencia respecto a un hub.*

El switch aprende las direcciones MAC de los dispositivos conectados y construye una tabla de conmutacion. Cuando recibe un paquete, consulta esta tabla para saber por que puerto debe reenviarlo.

3.4.2. Router (encaminador)

Un **router** es un dispositivo que opera en la capa de red (Capa 3). Su funcion es interconectar diferentes redes, tomando decisiones sobre el camino que deben seguir los paquetes para llegar a su destino.

*Router Un **router** o encaminador es un dispositivo de interconexion de red que opera en la capa de red. Conecta diferentes redes (por ejemplo, dos LAN con diferentes rangos de IP) y encamina los paquetes entre ellas mediante tablas de rutas.*

Un router tiene al menos dos interfaces de red (tarjetas de red), cada una conectada a una red diferente y con una direccion IP correspondiente a esa red. Cuando un paquete llega, el router consulta su **tabla de encaminamiento** para decidir por que interfaz debe reenviarlo.

3.4.3. Tabla de encaminamiento

La **tabla de encaminamiento** (o tabla de rutas) es la estructura de datos que utiliza un router para decidir como reenviar un paquete. Cada entrada contiene:

- **Destino:** direccion de red de destino.
- **Mascara:** mascara de subred del destino.
- **Siguiente salto (next hop):** direccion IP del siguiente router o dispositivo al que hay que enviar el paquete.
- **Coste:** metrica que indica la “distancia” o preferencia de la ruta.

*Tabla de encaminamiento Una **tabla de encaminamiento** es la estructura de datos que utiliza un router para determinar la mejor ruta hacia un destino. Contiene entradas con el destino, la mascara, el siguiente salto (next hop) y el coste asociado.*

3.5 Encaminamiento: como los paquetes encuentran su camino

El **encaminamiento** (routing) es el proceso de seleccionar el camino optimo para enviar un paquete desde su origen hasta su destino a traves de una red interconectada.

3.5.1. Tipos de encaminamiento

- **Encaminamiento estatico:** las rutas se configuran manualmente en los routers. Es sencillo pero no se adapta a cambios en la red.
- **Encaminamiento dinamico:** los routers intercambian informacion periodicamente y construyen sus tablas automaticamente. Se adapta a fallos y cambios de topologia. Protocolos: RIP (vector distancia), OSPF (estado de enlace).
- **Encaminamiento por difusion (broadcasting):** envia el paquete a todos los nodos. Muy ineficiente pero robusto (flooding).

- **Encaminamiento por camino mas corto:** se minimiza el numero de saltos entre origen y destino (o algun otro coste, como retardo o ancho de banda).

3.5.2. Encaminamiento en la practica: de una LAN a otra

Cuando un ordenador de la red 192.168.10.0 quiere enviar un paquete a otro ordenador de la misma red, el switch lo entrega directamente (consultando su tabla MAC).

Cuando quiere enviar un paquete a un ordenador de la red 192.168.20.0, el ordenador detecta que la direccion destino no pertenece a su red (comparando su propia IP y mascara con la IP destino). Entonces envia el paquete a su **gateway** (192.168.10.254), que es el router. El router recibe el paquete por su interfaz conectada a 192.168.10.0, consulta su tabla de rutas, ve que la red 192.168.20.0 es alcanzable a traves de su otra interfaz (192.168.20.254), y reenvia el paquete.

Este proceso es la base de la comunicacion entre redes (**inter-networking**), que es precisamente lo que hace posible Internet.

3.6 Practica 4: Cisco Packet Tracer ---

3.6.1. Objetivos

El objetivo de esta practica es entrar en contacto con el simulador de redes Cisco Packet Tracer y comprender los conceptos fundamentales de una red de area local:

1. Configurar una red de area local (LAN) con al menos 3 ordenadores y un switch, asignando direcciones IP, mascara de subred y gateway.
2. Verificar la comunicacion entre los ordenadores de la misma LAN mediante herramientas como ping.
3. Crear una segunda LAN con un rango de direcciones diferente.
4. Interconectar ambas LAN mediante un router, configurando sus interfaces de red para que actuen como gateway de cada LAN.
5. Verificar que los ordenadores de una red pueden comunicarse con los de la otra a traves del router.

3.6.2. Material utilizado

- **Software:** Cisco Packet Tracer (simulador de redes Cisco).
- **Hardware simulado:** PCs personales, switches (conmutadores) y router (encaminador).
- **Redes:**

- Red 1: 192.168.10.0 / 255.255.255.0
- Red 2: 192.168.20.0 / 255.255.255.0

3.6.3. Desarrollo de la practica

Primera parte: configuracion de la LAN 192.168.10.0

Se diseno una red local con 3 ordenadores personales y un switch. Cada ordenador se configuro con los siguientes parametros:

- **IP:** en el rango 192.168.10.0 (por ejemplo, 192.168.10.1, 192.168.10.2, 192.168.10.3).
- **Mascara de subred:** 255.255.255.0 (/24).
- **Gateway:** 192.168.10.254 (direccion que ocupara el router).

Se anadieron etiquetas a los ordenadores para identificar su nombre y su IP. A continuacion, se verifico la comunicacion entre los 3 ordenadores mediante el comando ping. El resultado confirmo que todos los ordenadores de la misma red local pueden comunicarse entre si a traves del switch.

TODO: Anadir captura de pantalla de Packet Tracer mostrando la LAN 192.168.10.0 con 3 PCs, el switch y las etiquetas.

Figura 3.1: Topologia de la primera LAN (192.168.10.0)

Comando de verificacion:

```
1 C:\> ping 192.168.10.2
2
3 Respuesta desde 192.168.10.2: bytes=32 tiempo<1ms TTL=128
4 Respuesta desde 192.168.10.2: bytes=32 tiempo<1ms TTL=128
5 Respuesta desde 192.168.10.2: bytes=32 tiempo<1ms TTL=128
```

Código 3.1: Verificacion de conectividad en la LAN 1

Segunda parte: configuracion de la LAN 192.168.20.0 e interconexion

Se creo una segunda red local con 3 ordenadores mas y otro switch, esta vez en el rango 192.168.20.0. La configuracion de cada ordenador fue:

- **IP:** en el rango 192.168.20.0 (por ejemplo, 192.168.20.1, 192.168.20.2, 192.168.20.3).

- **Mascara de subred:** 255.255.255.0.
- **Gateway:** 192.168.20.254.

Se verifico que los ordenadores de esta segunda LAN se comunicaban entre si.

TODO: Anadir captura de pantalla de Packet Tracer mostrando la LAN 192.168.20.0 con 3 PCs, el switch y las etiquetas.

Figura 3.2: Topologia de la segunda LAN (192.168.20.0)

Tercera parte: conexion de ambas LAN mediante un router

Para que los ordenadores de la red 192.168.10.0 puedan comunicarse con los de la red 192.168.20.0, se anadio un **router** entre ambas redes. El router cuenta con dos interfaces de red (tarjetas), una conectada a cada switch:

- **Interfaz 1:** IP 192.168.10.254 / Mascara 255.255.255.0 (conectada al switch de la LAN 1).
- **Interfaz 2:** IP 192.168.20.254 / Mascara 255.255.255.0 (conectada al switch de la LAN 2).

Estas dos direcciones son las **puertas de enlace** (gateway) de cada red.

Se etiquetaron las conexiones del router indicando la red a la que se conecta cada interfaz y la IP configurada. Finalmente, se verifico la comunicacion entre ordenadores de ambas redes:

```
1 C:\> ping 192.168.20.2
2
3 Respuesta desde 192.168.20.2: bytes=32 tiempo=1ms TTL=127
4 Respuesta desde 192.168.20.2: bytes=32 tiempo=1ms TTL=127
5 Respuesta desde 192.168.20.2: bytes=32 tiempo=1ms TTL=127
```

Código 3.2: Verificacion de conectividad entre redes

TODO: Anadir captura de pantalla de la topologia completa con ambas LAN, switches, router y las conexiones etiquetadas.

Figura 3.3: Topologia completa: interconexion de dos LAN mediante un router

3.6.4. Analisis de la comunicacion entre redes

Cuando el PC 192.168.10.1 envia un ping al PC 192.168.20.2, ocurre lo siguiente:

1. El PC 192.168.10.1 calcula la red destino aplicando su mascara a la IP 192.168.20.2. Detecta que la red destino (192.168.20.0) no coincide con su propia red (192.168.10.0).
2. Por tanto, no puede enviar el paquete directamente. Lo envia a su **gateway**: 192.168.10.254.
3. El router recibe el paquete por su interfaz 192.168.10.254. Consulta su tabla de encaminamiento y detecta que la red 192.168.20.0 es **directamente conectada** a traves de su otra interfaz.
4. El router reenvia el paquete por la interfaz 192.168.20.254 hacia el switch de la LAN 2.
5. El switch entrega el paquete al PC 192.168.20.2.
6. El PC 192.168.20.2 responde. Como la IP 192.168.10.1 pertenece a otra red, el PC 192.168.20.2 envia la respuesta a su gateway (192.168.20.254).
7. El router reenvia la respuesta por la interfaz 192.168.10.254.

Este proceso ilustra el concepto fundamental de **inter-networking**: la interconexion de redes distintas mediante un dispositivo de capa 3 (router).

3.6.5. Relacion teoria-practica

3.7 Resumen de la Unidad 3

- Una **red de computadores** esta compuesta por hosts, enlaces de comunicacion (guiados o no guiados) y dispositivos de conmutacion.
- El **modelo TCP/IP** organiza la comunicacion en cuatro capas: aplicacion, transporte, internet y enlace. TCP garantiza fiabilidad; UDP prioriza la velocidad.

Tabla 3.4: Conexion entre conceptos teoricos y elementos de la Practica 4

Concepto teorico	Observacion/aplicacion en la practica
Red de computadores	Se disenaron dos redes locales con 3 ordenadores cada una, switch y enlaces fisicos
Switch (Capa 2)	Conecta los 3 ordenadores de cada LAN y reenvia paquetes al destinatario correcto
Router (Capa 3)	Interconecta las dos LAN distintas, encaminando paquetes entre 192.168.10.0 y 192.168.20.0
Direccion IP	Cada ordenador tiene una IP unica dentro de su rango (192.168.10.x o 192.168.20.x)
Mascara de subred	255.255.255.0 indica que los primeros 3 octetos son red y el ultimo es host
Gateway	192.168.10.254 y 192.168.20.254 son las interfaces del router que actuan como puerta de enlace
Comunicacion intra-red	Los PCs de la misma LAN se comunican directamente a traves del switch (ping funciona)
Comunicacion inter-red	Los PCs de distintas redes necesitan el router; el ping inter-red confirma el encaminamiento
Tabla de encaminamiento	El router internamente tiene una entrada indicando que ambas redes son directamente conectadas
Modelo TCP/IP (Capa 3)	El ICMP (ping) opera sobre IP, demostrando la funcion de la capa de red

- El **direccionamiento IP** permite identificar cada dispositivo en una red. Una direccion IPv4 de 32 bits se divide en parte de red y parte de host, separadas por la **mascara de subred**.
- Las **clases de direcciones IP** (A, B, C, D, E) facilitan la asignacion de rangos a redes de diferentes tamanos. Las direcciones privadas permiten redes locales sin consumir IPs publicas.
- El **gateway** es la direccion del router que permite salir de una red local hacia otra red o hacia Internet.
- Un **switch** opera en la capa de enlace, conectando dispositivos dentro de una misma LAN mediante reenvio basado en direcciones MAC.
- Un **router** opera en la capa de red, interconectando diferentes redes y tomando decisiones de encaminamiento mediante su **tabla de rutas**.
- El **encaminamiento** puede ser estatico (configurado manualmente) o dinamico (protocolos como RIP u OSPF). El proceso de enviar un paquete de una red a otra es la base de Internet.
- La **Practica 4** demuestra todos estos conceptos en un entorno simulado: dos LAN conectadas mediante un router, configuracion de IPs, mascarar, gateways y verificacion de conectividad con ping.

CAPÍTULO 4

Servicios de Red

Hasta ahora hemos aprendido como los dispositivos se conectan físicamente mediante cables y ondas de radio, como se organizan en redes, como se identifican mediante direcciones IP y como los routers encaminan los paquetes. Pero una red no es solo cableado y protocolos de bajo nivel: necesita **servicios** que permitan a los usuarios y aplicaciones compartir recursos, comunicarse y acceder a internet.

En este capítulo estudiaremos los principales servicios de red, con especial atención a dos de los más fundamentales: el **DNS** (sistema de nombres de dominio) y el **DHCP** (protocolo de configuración dinámica de host), sin los cuales una red moderna no podría funcionar.

4.1 Definición de servicios de red

*Servicios de red Los **servicios de red** son protocolos de software ejecutados en servidores, que permiten a los dispositivos conectados compartir recursos, comunicarse, acceder a internet y gestionar la infraestructura tecnológica.*

Los servicios de red operan bajo un modelo **cliente-servidor**: el servidor ofrece un servicio y el cliente lo consume. Son esenciales tanto en redes locales (LAN) para compartir impresoras o archivos, como en redes globales para el acceso a internet.

4.2 Clasificación de los servicios de red

Los servicios de red se pueden clasificar en cuatro categorías principales:

4.2.1. Servicios de infraestructura

Son los servicios fundamentales para la conectividad y el funcionamiento de la red:

- **DNS (Domain Name System):** traduce nombres de dominio (direcciones web como `www.upv.es`) a direcciones IP numericas (como `158.42.4.23`). Es el “agenda telefonica” de Internet.
- **DHCP (Dynamic Host Configuration Protocol):** asigna automaticamente direcciones IP y otros parametros de configuracion (mascara, gateway, DNS) a los dispositivos que se conectan a la red.

4.2.2. Servicios de internet y comunicacion

Permiten la comunicacion entre personas y el acceso a la informacion:

- **WWW (World Wide Web):** usa los protocolos HTTP/HTTPS para navegar por paginas web. Es el servicio mas visible de Internet.
- **Correo electronico:** usa SMTP (envio), POP3 e IMAP (recepcion) para el intercambio de mensajes.
- **FTP (File Transfer Protocol):** permite transferir ficheros entre ordenadores de forma rapida y fiable.

4.2.3. Servicios de acceso y gestion

Facilitan la administracion remota y la organizacion de recursos:

- **Acceso remoto:** SSH (Secure Shell) y Telnet permiten gestionar equipos a distancia. SSH es el estandar actual porque cifra la comunicacion.
- **Servicios de directorio:** gestionan usuarios, permisos y recursos en una red. Ejemplo: Active Directory de Microsoft.

4.2.4. Servicios de aplicacion

Son los servicios que usan directamente los usuarios finales:

- **Aplicaciones en la nube (SaaS):** software ejecutado en servidores remotos (Google Workspace, Microsoft 365).
- **Mensajeria:** WhatsApp, Slack, Teams.
- **Bases de datos:** servidores que almacenan y gestionan datos.
- **Streaming:** Netflix, Spotify, YouTube.

4.3 DNS (Domain Name System)

4.3.1. El problema: los humanos usan nombres, las maquinas usan numeros

Los seres humanos recordamos nombres mucho mejor que numeros. Sin embargo, los dispositivos de red se identifican mediante direcciones IP (numeros). El **DNS** resuelve esta disparidad actuando como un traductor entre nombres y numeros.

*DNS (Domain Name System) El **DNS** (Domain Name System) es un sistema distribuido y jerarquico que traduce nombres de dominio legibles por humanos (como `www.upv.es`) a direcciones IP numericas (como `158.42.4.23`) y viceversa. Esta definido en los RFC 1034 y 1035.*

4.3.2. El nombre en una red: FQDN

Un nombre de dominio completo se llama **FQDN** (Full Qualified Domain Name) y tiene una estructura jerarquica:

```
www.upv.es
|   |   |
|   |   +-- Dominio de nivel superior (TLD): pais o categoria
|   +----- Organizacion a la que pertenece
+----- Nombre del equipo o rol que desempeña
```

Por ejemplo, en `www.upv.es`:

- `www`: nombre del equipo o servicio (world wide web)
- `upv`: organizacion (Universitat Politecnica de Valencia)
- `es`: codigo de pais (Espana)

4.3.3. Estructura jerarquica del DNS

El DNS no es una base de datos centralizada. Es **distribuido** en una estructura de arbol invertido:

1. **Root Servers** (servidores raiz): son los 13 servidores raiz etiquetados de la A a la M. La mayoría estan en Estados Unidos. Son el punto de entrada de cualquier consulta DNS.
2. **TLD Servers** (Top Level Domain): gestionan los dominios de primer nivel (`.com`, `.es`, `.org`, `.edu`, `.net`).
3. **Authoritative DNS Servers**: servidores autoritativos de cada dominio concreto. Contienen los registros reales (A, MX, CNAME...) de los equipos de esa organizacion.

Esta descentralizacion evita un punto unico de fallo y distribuye el trafico.

4.3.4. Tipos de consultas DNS

Existen dos formas de resolver un nombre:

- **Consulta recursiva:** el cliente pide al servidor DNS que resuelva el nombre completo. Si el servidor no lo sabe, el propio servidor consulta a otros servidores en nombre del cliente hasta obtener la respuesta.
- **Consulta iterativa:** el servidor responde con la mejor informacion que tiene. Si no conoce la respuesta, indica al cliente a que otro servidor debe preguntar.

El **caching (cache) es fundamental:** los servidores y los clientes almacenan temporalmente las respuestas para acelerar futuras consultas y reducir la carga en la red.

4.3.5. Tipos de registros DNS

Un servidor DNS autoritativo almacena registros de distintos tipos:

Tabla 4.1: Tipos de registros DNS mas comunes

Registro	Funcion
A (Address)	Asocia un nombre de dominio a una direccion IPv4
AAAA	Asocia un nombre a una direccion IPv6
CNAME (Canonical Name)	Alias: asocia un nombre alternativo a un nombre canonico
MX (Mail Exchange)	Indica el servidor de correo del dominio
TXT	Texto arbitrario, usado para verificaciones y politicas (SPF, DKIM)
NS (Name Server)	Indica que servidores DNS son autoritativos para el dominio
PTR (Pointer)	Resolucion inversa: de IP a nombre de dominio

4.3.6. DNS internos vs. DNS externos (autoritativos)

- **DNS interno:** resuelve nombres dentro de la red local de una empresa. Por ejemplo, `servidor-interno.empresa.local` puede apuntar a `192.168.1.10`. Solo es visible desde dentro de la red.
- **DNS externo / autoritativo:** resuelve nombres publicos en Internet. Por ejemplo, `www.empresa.com` apunta a la IP publica del servidor web. Es visible desde cualquier parte de Internet.

Muchas empresas tienen ambos: un DNS interno para sus equipos y un DNS externo gestionado por su proveedor o registrador de dominio.

4.3.7. Consulta manual con nslookup

Para diagnosticar problemas DNS, se puede usar el comando nslookup:

```
1 C:\> nslookup www.upv.es
2 Servidor:  dns.google
3 Address:  8.8.8.8
4
5 Respuesta no autoritativa:
6 Nombre:  www.upv.es
7 Address:  158.42.4.23
```

Código 4.1: Consulta DNS con nslookup

4.4 DHCP (Dynamic Host Configuration Protocol)

4.4.1. Definicion

*DHCP (Dynamic Host Configuration Protocol) El **DHCP** es un protocolo de red que permite a los dispositivos obtener automaticamente una direccion IP y otros parametros de configuracion (mascara de subred, gateway, servidores DNS) de un servidor, sin intervencion manual.*

Sin DHCP, cada dispositivo de una red tendria que configurarse manualmente con una IP, mascara, gateway y DNS. En una red con cientos de dispositivos, esto seria impracticable y propenso a errores (IPs duplicadas, mascaras incorrectas...).

4.4.2. Beneficios clave de DHCP

- **Administracion centralizada:** un unico servidor gestiona toda la configuracion IP de la red.
- **Consistencia:** todos los dispositivos reciben la configuracion correcta y homogenea.
- **Flexibilidad:** cambiar la red (por ejemplo, el gateway) solo requiere modificar el servidor, no cada dispositivo.
- **Escalabilidad:** anadir nuevos dispositivos es tan simple como conectarlos; reciben IP automaticamente.

4.4.3. Como funciona: el proceso DORA

Cuando un dispositivo se conecta a una red, el proceso DHCP sigue cuatro pasos:

1. **DHCPDISCOVER:** el cliente envia un mensaje de difusion (broadcast) preguntando: "Hay algun servidor DHCP? Necesito una IP".

2. **DHCPOFFER**: el servidor responde ofreciendo una dirección IP disponible, junto con la máscara, el gateway, los DNS y el tiempo de concesión (lease time).
3. **DHCPREQUEST**: el cliente acepta la oferta y solicita formalmente esa IP.
4. **DHCPACK**: el servidor confirma la concesión y el cliente puede empezar a usar la IP.

Este proceso se conoce como **DORA** (Discover, Offer, Request, Acknowledge).

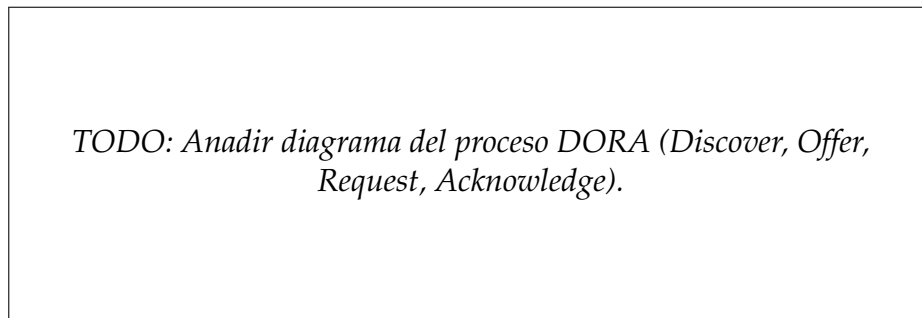


Figura 4.1: Proceso DORA de DHCP

4.4.4. Mensajes DHCP adicionales

Además de los cuatro principales, existen otros mensajes:

- **DHCPDECLINE**: el cliente rechaza la IP ofrecida porque detecta que ya está en uso (por ejemplo, mediante ARP).
- **DHCPNAK**: el servidor deniega una concesión (por ejemplo, si la IP ya no está disponible o el cliente ha cambiado de red).
- **DHCPRELEASE**: el cliente libera voluntariamente la IP antes de desconectarse (útil para dispositivos móviles).
- **DHCPINFORM**: el cliente ya tiene IP (por ejemplo, configurada manualmente) pero solicita otros parámetros como DNS o gateway.

4.4.5. Renovación de la concesión

Las IPs asignadas por DHCP no son permanentes: se conceden por un tiempo limitado (**lease time**). Cuando la mitad del tiempo ha transcurrido, el cliente intenta renovar la concesión enviando un nuevo DHCPREQUEST. Si el servidor responde con DHCPACK, la concesión se extiende. Si no, el cliente puede seguir usando la IP hasta que expire, momento en el que debe iniciar un nuevo proceso DORA.

4.4.6. Ambitos (scopes) y exclusiones

Un servidor DHCP organiza las direcciones en **ambitos** (scopes):

- **Ambito:** un rango continuo de direcciones IP disponibles para asignar. Por ejemplo, de 192.168.1.100 a 192.168.1.200.
- **Exclusiones:** direcciones dentro del ambito que no se asignan automaticamente. Por ejemplo, la IP del router (192.168.1.1), del servidor de impresion (192.168.1.10), etc.
- **Reservas:** direcciones asignadas permanentemente a un dispositivo especifico (identificado por su direccion MAC). Util para servidores, impresoras o PLCs que necesitan una IP fija.

4.4.7. Agentes de retransmision (DHCP Relay)

Si el servidor DHCP esta en una red diferente a la de los clientes, los mensajes DHCP (que son broadcasts) no llegan a traves de los routers. La solucion son los **agentes de retransmision** (DHCP relay agents): dispositivos que reciben los mensajes DHCP de broadcast en una red y los reenvian como unicast al servidor DHCP en otra red.

4.4.8. BOOTP: el predecesor de DHCP

BOOTP (Bootstrap Protocol) fue el predecesor de DHCP. A diferencia de DHCP, BOOTP asignaba direcciones IP de forma estatica (manualmente configuradas en el servidor) y no admitia concesiones temporales. DHCP es retrocompatible con BOOTP y puede atender a clientes BOOTP.

4.5 Otros servicios de red relevantes

4.5.1. HTTP/HTTPS (World Wide Web)

El protocolo **HTTP** (HyperText Transfer Protocol) es el fundamento de la Web. Permite a los navegadores (clientes) solicitar paginas web a los servidores. **HTTPS** es la version segura, que cifra la comunicacion mediante TLS/SSL.

4.5.2. SSH (Secure Shell)

SSH permite acceder y gestionar equipos remotos mediante una conexion segura y cifrada. Es el reemplazo seguro de Telnet (que enviaba datos en texto plano). Se usa ampliamente para administrar servidores, routers, switches y dispositivos IoT.

4.5.3. Correo electronico: SMTP, POP3, IMAP

- **SMTP** (Simple Mail Transfer Protocol): envia correos desde el cliente al servidor, y entre servidores.
- **POP3** (Post Office Protocol): descarga los correos del servidor al cliente (los elimina del servidor).
- **IMAP** (Internet Message Access Protocol): accede a los correos en el servidor sin descargarlos, permitiendo sincronizacion entre multiples dispositivos.

4.5.4. FTP/SFTP

FTP (File Transfer Protocol) permite transferir ficheros entre ordenadores. Aunque es rapido, no cifra la comunicacion. **SFTP** (SSH File Transfer Protocol) utiliza SSH para transferir ficheros de forma segura.

4.6 Resumen de la Unidad 4

- Los **servicios de red** son protocolos de software que permiten a los dispositivos compartir recursos, comunicarse y gestionar la infraestructura.
- Los principales tipos de servicios son: infraestructura (DNS, DHCP), internet y comunicacion (WWW, correo, FTP), acceso y gestion (SSH, Active Directory), y aplicacion (nube, bases de datos, streaming).
- El **DNS** traduce nombres de dominio a direcciones IP. Es un sistema jerarquico y distribuido con servidores raiz, TLD y autoritativos. Utiliza registros como A, CNAME, MX, TXT, NS.
- El **DHCP** asigna automaticamente direcciones IP y parametros de configuracion. El proceso sigue cuatro pasos: Discover, Offer, Request, Acknowledge (DORA). Gestiona ambitos, exclusiones, reservas y renovaciones.
- La **convergencia de servicios** (DNS + DHCP + Active Directory) permite una gestion centralizada y automatizada de las redes modernas.

CAPÍTULO 5

Introduccion a las Redes Industriales

Hasta ahora hemos estudiado como comunican dispositivos mediante cables, ondas de radio, y como se organizan las redes de computadores mediante direcciones IP, switches y routers. Sin embargo, en un entorno de fabricacion o produccion, las redes no solo conectan ordenadores: conectan sensores, actuadores, robots, PLCs y sistemas de supervision, todo trabajando en tiempo real para controlar procesos industriales.

En este capitulo introducimos el concepto de **redes industriales**, tambien conocidas como redes de comunicacion para la automatizacion y control de procesos. Veremos que son, como funcionan, que tipos existen y por que son fundamentales en la industria moderna.

5.1 Sistemas industriales distribuidos

5.1.1. Definicion

Un **sistema industrial distribuido** es una arquitectura de control y automatizacion donde multiples dispositivos, sensores, actuadores y sistemas informaticos estan interconectados para operar de manera descentralizada.

*Sistema industrial distribuido Un **sistema industrial distribuido** es una arquitectura de control donde multiples dispositivos (PLCs, sensores, actuadores, robots) comparten la responsabilidad de la operacion, interconectados mediante una red de comunicacion industrial. Ofrece mayor flexibilidad, escalabilidad y eficiencia que los sistemas centralizados tradicionales.*

En un sistema centralizado clasico, un unico controlador (PLC o computador) toma todas las decisiones y envia ordenes a los dispositivos de campo. En un sistema distribuido, la inteligencia se reparte: cada dispositivo puede tomar decisiones locales y compartir informacion con el resto.

5.1.2. Características principales

Los sistemas industriales distribuidos se caracterizan por:

- **Descentralización:** múltiples dispositivos comparten la responsabilidad de la operación. No hay un único punto de control.
- **Interconectividad:** se basan en redes de comunicación industrial como Ethernet/IP, Modbus, Profibus o IoT industrial.
- **Escalabilidad:** se pueden agregar nuevos dispositivos sin afectar la operación global.
- **Tolerancia a fallos:** al no depender de un único nodo, el sistema puede seguir funcionando ante fallos parciales.
- **Optimización de recursos:** permite un uso eficiente de energía, maquinaria y mano de obra.
- **Automatización avanzada:** utiliza PLCs, SCADA, IIoT e inteligencia artificial.

5.1.3. Aplicaciones y ventajas

Las aplicaciones más comunes incluyen:

- **Automatización de fábricas:** producción y ensamblaje de productos.
- **Redes de energía inteligente (Smart Grid):** gestión eficiente de energía eléctrica.
- **Transporte y logística:** control distribuido en almacenes y vehículos autónomos.
- **Industria 4.0:** implementación de IoT y Big Data para optimización.
- **Robotica industrial:** control distribuido en líneas de producción.

Las ventajas principales son: mayor eficiencia operativa, reducción de costes de mantenimiento, mayor adaptabilidad a cambios en la producción, y mejora en la recopilación y análisis de datos.

5.2 Redes industriales: conceptos fundamentales

5.2.1. Definición

*Red industrial Una **red industrial** es una estructura de comunicación automatizada que gestiona procesos industriales, involucrando actuadores, computadoras, máquinas, sensores, interfaces, medios de comunicación (fibra óptica, cables, redes inalámbricas) y*

robots. Su objetivo es transmitir y compartir datos para garantizar un funcionamiento eficiente de los procesos productivos.

5.2.2. Como funcionan

Las redes industriales reemplazan el modelo tradicional de control cableado a mano (donde cada dispositivo necesitaba su propio cable de control) por un modelo de comunicación digital donde múltiples dispositivos comparten un único medio de transmisión.

Este modelo reduce drásticamente la cantidad de cableado necesario, simplifica la organización del sistema y permite una flexibilidad mucho mayor para reconfigurar la producción.

5.2.3. Convergencia de TI y TO (OT)

En la industria moderna se distinguen dos tecnologías:

- **TI (Tecnología de la Información):** se ocupa de la entrega de información cuando un profesional la solicita. Sistemas de datos, ERP, CRM.
- **TO (Tecnología Operacional) o OT:** trabaja con la entrega de información en un momento específico bajo una condición determinada. Controla equipos industriales en tiempo real.

La convergencia de TI y TO está relacionada con el auge del **edge computing** (computación en el borde): trasladar los recursos informáticos a la ubicación física de la fuente de datos (por ejemplo, en la propia fábrica), permitiendo unificar sistemas que tradicionalmente estaban aislados.

5.2.4. Desafíos

Las redes industriales enfrentan desafíos específicos:

- **Complejidad en la integración:** sistemas heterogéneos de diferentes fabricantes deben comunicarse.
- **Ciberseguridad:** las redes industriales son objetivos críticos; un ataque puede parar la producción.
- **Capacitación especializada:** requiere personal con conocimientos en automatización y redes.

5.3 Clasificación de las redes industriales

Las redes industriales se clasifican según el nivel de la pirámide de automatización:

Tabla 5.1: Niveles de las redes industriales

Nivel	Funcion	Ejemplos
Nivel de informacion (gestion)	Planificacion, ERP, MES	Redes Ethernet, TCP/IP, bases de datos
Nivel de control	Controladores, PLCs, SCADA	ControlNet, Ethernet/IP
Nivel de campo y proceso	Sensores, actuadores, transmisores	Profibus, Modbus, CAN
Nivel de entrada/salida	Dispositivos simples, fotocelulas	AS-i, Sensor Bus

5.3.1. Clasificacion por funcionalidad

Segun sus capacidades y el tipo de datos que transportan:

1. **Sensor Bus (buses de sensores):** conectan sensores digitales y actuadores simples. Transmite pocos datos, distancias cortas, bajo costo. Ejemplos: AS-i, CAN, Interbus.
2. **Device Bus (buses de dispositivos):** intermedian entre sensores y PLCs. Manejan variables de proceso (presion, temperatura, flujo). Alcance hasta 500 metros. Ejemplos: Modbus, Profibus DP, DeviceNet.
3. **Field Bus (buses de campo):** permiten que varios instrumentos se comuniquen en red para control y monitoreo. Conectan mas equipos a mayor distancia. Ejemplos: Foundation Fieldbus, Profibus PA.
4. **Control Bus / Ethernet Industrial:** redes de alta velocidad para control en tiempo real. Ejemplos: Ethernet/IP, PROFINET, EtherCAT, ControlNet.

5.3.2. Tabla comparativa de tipos de redes industriales

Tabla 5.2: Comparacion de tipos de redes industriales

Tipo	Velocidad	Funcionalidad	Uso tipico
Sensor Bus	Baja	Simple	Sensores, actuadores binarios
Device Bus	Media	Media	Transmisores, servomotores
Field Bus	Media	Alta	Control de procesos continuos
Ethernet Industrial	Alta	Muy alta	Control en tiempo real, integracion TI/TO

5.4 Principales protocolos de redes industriales

5.4.1. PROFINET

PROFINET es una solución de Ethernet Industrial abierta basada en estándares internacionales. Es el protocolo de comunicación desarrollado para intercambiar datos entre controladores y dispositivos en un sector automatizado.

Utiliza Ethernet estándar como medio de comunicación, lo que permite que otros protocolos Ethernet (SNMP, MQTT, HTTP) coexistan en la misma infraestructura. Además, define la comunicación cíclica y acíclica entre componentes, incluyendo diagnósticos, seguridad funcional, alarmas e información adicional.

5.4.2. PROFIBUS

PROFIBUS es una red digital que proporciona comunicación entre sensores de campo y controladores. Surgió inicialmente como PROFIBUS FMS (Fieldbus Message Specification), pero hoy en día los más utilizados son:

- **PROFIBUS DP** (Decentralised Peripherals): para automatización de fábricas, comunicación rápida entre PLCs y dispositivos de campo.
- **PROFIBUS PA** (Process Automation): para procesos continuos como industrias petroquímica y minera.

5.4.3. EtherCAT

EtherCAT (Ethernet for Control Automation Technology) es un protocolo que lleva la potencia y flexibilidad de Ethernet a la automatización industrial, al control de movimientos y a los sistemas de control en tiempo real.

Fue desarrollado por Beckhoff Automation y estandarizado bajo IEC 61158. Es mantenido por el EtherCAT Technology Group (ETG). Su principal ventaja es la extrema velocidad de transmisión y la sincronización precisa de los dispositivos.

5.4.4. ControlNet

ControlNet utiliza el Protocolo Industrial Común (CIP) para las capas superiores del modelo OSI. Se desarrolló para ofrecer control confiable y de alta velocidad, con transferencia de datos I/O programada en momentos específicos.

Garantiza que los mensajes críticos que no dependen del tiempo se ejecuten sin interferir en la transferencia de datos de control. Se utiliza normalmente para aplicaciones redundantes.

5.4.5. Ethernet en la industria: importancia en la Industria 4.0

Ethernet se ha convertido en el estandar mundial para la intercomunicacion de datos en red porque:

- Tiene una implementacion simple y costos reducidos.
- Permite el uso de varios protocolos en un entorno industrial.
- Es interoperable y escalable para las demandas.
- Sigue en constante evolucion y mejora.

En la **Industria 4.0**, la informacion fluye horizontal y verticalmente a traves de toda la organizacion. Esto requiere una infraestructura de comunicacion que abarque todas las maquinas, computadoras, robots y equipos, lo cual solo es posible con Ethernet e Internet unidos al IIoT (Internet Industrial de las Cosas).

5.5 Resumen de la Unidad 5

- Los **sistemas industriales distribuidos** descentralizan el control, permitiendo que multiples dispositivos compartan la responsabilidad de la operacion y mejoren la tolerancia a fallos.
- Una **red industrial** es una estructura de comunicacion que gestiona procesos industriales, conectando sensores, actuadores, PLCs, robots y sistemas de supervision.
- La **convergencia de TI y TO (OT)** unifica los sistemas de informacion empresarial con los sistemas de control en tiempo real, facilitando el **edge computing** y la Industria 4.0.
- Las redes industriales se clasifican en niveles: informacion, control, campo y E/S. Por funcionalidad: Sensor Bus, Device Bus, Field Bus y Ethernet Industrial.
- Los principales protocolos son: **PROFINET, PROFIBUS, EtherCAT, ControlNet, Modbus, Ethernet/IP**.
- **Ethernet** es el estandar dominante en la industria moderna por su simplicidad, bajo costo, interoperabilidad y capacidad para soportar multiples protocolos simultaneamente.

CAPÍTULO 6

Encaminadores y Protocolos de Routing

En los capítulos anteriores hemos visto como los dispositivos se conectan en redes, como se comunican mediante direcciones IP y como los switches operan en la capa de enlace. Sin embargo, cuando una red crece y se divide en múltiples subredes, surge un problema fundamental: como hacer que los paquetes de datos lleguen desde una red a otra, eligiendo el mejor camino posible.

En este capítulo estudiamos el **encaminamiento** (routing): el proceso de tomar decisiones sobre por donde enviar los paquetes en una red interconectada. Veremos los fundamentos teóricos, los algoritmos que permiten calcular caminos óptimos, y los protocolos que permiten a los routers compartir información de rutas automáticamente.

6.1 Conceptos fundamentales de encaminamiento

*Encaminamiento y conmutación El **encaminamiento** (routing) es el proceso de elección del camino óptimo que debe seguir un paquete desde su origen hasta su destino a través de una red interconectada. Es una decisión de control.*

*La **conmutación** (switching) es el acto de retransmitir los datos por uno de los caminos posibles, una vez tomada la decisión. Es una acción de reenvío.*

El encaminamiento toma la decisión sobre el camino a seguir; la conmutación ejecuta esa decisión reenviando el paquete por la interfaz correspondiente.

6.1.1. Tipos de encaminamiento

Existen varias estrategias para encaminar paquetes:

- **Difusión (Broadcasting)**: envío de información a todos los nodos de la red. Se implementa mediante **inundación** (flooding): cada nodo reenvía el paquete a todos sus vecinos excepto al que se lo envió.

- Ventaja: muy robusta, garantiza que se prueban todos los caminos posibles.
 - Desventaja: mucha sobrecarga, transmisiones y recepciones innecesarias.
- **Camino mas corto (Shortest Path):** una de las estrategias mas empleadas. Se minimiza el numero de saltos entre origen y destino. De manera generica, se puede hablar de **coste** o **metrica**, estableciendo alguna medida como distancia, retardo, ancho de banda, etc.
 - **Encaminamiento optimo:** el camino mas corto no siempre ofrece el mejor comportamiento (por ejemplo, puede estar congestionado). El encaminamiento optimo busca la mejor ruta segun multiples criterios mediante optimizacion matematica.
 - **Patata caliente (Hot Potato):** un nodo manda cada paquete por la interfaz menos cargada, deshaciendose del mismo lo antes posible. Es una tecnica muy simple que no considera la distancia global.

6.2 Grafos y encaminamiento

Los **grafos** son una herramienta matematica que se emplea para formular problemas de encaminamiento.

*Grafo Un **grafo** es una estructura matematica definida como $G = (N, d)$, donde:*

- *N es un conjunto de nodos (vertices).*
- *d es una coleccion de enlaces (edges), donde cada enlace consta de un par de nodos de N.*

Al trasladar un grafo a un problema de encaminamiento:

- Los **nodos** N son los encaminadores (routers) de la red.
- Los **enlaces** d se corresponden con los enlaces fisicos entre ellos.

6.2.1. Tipos de grafos

- **Grafos dirigidos:** los enlaces son pares ordenados (u,v) donde el orden importa. $(u,v) \neq (v,u)$.
- **Grafos no dirigidos:** los enlaces no tienen direccion. $(u,v) = (v,u)$.

6.2.2. Concatenaciones de enlaces

- **Paseo (Walk):** secuencia de nodos (n1, n2, ..., nl) tal que cada pareja (ni-1, ni) es un enlace del grafo.

- **Camino (Path):** es un paseo en el que no hay nodos repetidos.
- **Ciclo o bucle (Cycle):** camino con mas de un enlace en el que el nodo inicial coincide con el final ($n_1 = n_l$).

6.2.3. Grafo conectado

Se dice que un grafo esta **conectado** si cualquier par de nodos esta conectado por un camino. En redes de comunicaciones, esto significa que cualquier router puede comunicarse con cualquier otro.

6.2.4. Representacion de grafos

Los grafos se pueden representar de dos formas principales:

Tabla 6.1: Representacion de grafos

Aspecto	Lista de adyacencia	Matriz de adyacencia
Estructura	Array de N listas, una por nodo, con punteros a vecinos	Matriz F de dimension $N \times N$
Memoria	Menor memoria, apropiada para grafos dispersos (sparse)	Mayor memoria, N^2 elementos
Ventajas	Eficiente en memoria para grafos con pocos enlaces	Busqueda muy rapida, facil asignar costes
Desventajas	Busqueda puede ser lenta	Requiere mas memoria, se usa en grafos pequenos
Grafos no dirigidos	Numero de punteros = $2 d $	Matriz simetrica: $F^T = F$
Grafos dirigidos	Numero de punteros = $ d $	Matriz no simetrica

6.2.5. Costes en los enlaces

En algunas ocasiones se asignan **costes** $c(u,v)$ a los enlaces, representando metricas como:

- Distancia fisica (kilometros).
- Retardo (milisegundos).
- Ancho de banda (Mbps).
- Congestion actual.

El objetivo del encaminamiento es entonces encontrar el camino con menor coste acumulado.

6.3 Algoritmos de camino mas corto

La idea principal es encontrar el camino con un coste minimo entre una fuente (S) y un destino (D). Si $c(u,v)$ se mantiene constante para todos los enlaces, la solucion es la ruta de menor numero de saltos.

6.3.1. Algoritmos con una unica fuente

Encuentran el camino mas corto entre un nodo origen S y el resto de nodos:

- **Dijkstra**: funciona con grafos donde los costes son positivos. Es el algoritmo mas utilizado en protocolos como OSPF.
- **Bellman-Ford**: permite costes negativos (si no hay ciclos negativos). Es la base del encaminamiento por vectores de distancia (RIP).

6.3.2. Algoritmos para toda la red

Encuentran el camino mas corto entre todas las posibles parejas de nodos:

- **Floyd-Warshall**: algoritmo de programacion dinamica.
- **Johnson**: combina Dijkstra y Bellman-Ford para grafos dispersos.

6.3.3. Arbol de expansion minimo (Minimum Spanning Tree)

Un **arbol** es un grafo no dirigido, conectado y sin ciclos. El numero de enlaces es igual al numero de nodos menos 1: $|d| = |N| - 1$.

Un **Spanning Tree** cubre (se expande por) todos los nodos de un grafo. El **Minimum Spanning Tree (MST)** es aquel que tiene el menor coste total.

Aplicaciones:

- Procesos de difusion (broadcast) de informacion.
- Redes de area local: bridges (IEEE 802.1D) emplean el MST para eliminar enlaces no necesarios y evitar bucles.

Algoritmos: Kruskal y Prim.

6.4 Protocolos de encaminamiento

Los nodos (encaminadores) toman decisiones en base a cierta informacion. Es necesario disponer de un **protocolo de senalizacion** que proporcione un metodo para transportar dicha informacion por la red.

6.4.1. Clasificación de protocolos

Tabla 6.2: Clasificación de protocolos de encaminamiento

Tipo	Descripción
Vector de distancia (DV)	Cada router intercambia periódicamente su tabla con los vecinos. Usa Bellman-Ford. Ejemplo: RIP.
Estado de enlace (LS)	Cada router difunde el estado de sus enlaces a toda la red. Usa Dijkstra. Ejemplo: OSPF.

6.4.2. Encaminamiento por vector de distancia

Se basa en el algoritmo de Bellman-Ford. Cada nodo de la red mantiene una tabla de encaminamiento con una entrada por cada uno del resto de nodos:

- **Distancia:** métrica o coste del camino hacia dicho nodo.
- **Interfaz de salida:** necesaria para alcanzarle.

Periodicamente, cada nodo intercambia la información de su tabla con sus vecinos.

Problemas:

- **Convergencia lenta:** los cambios en la topología tardan en propagarse.
- **Propagación rápida de “buenas noticias” y lenta de las “malas”:** cuando un enlace mejora, la noticia se propaga rápido; cuando empeora, tarda.
- **Count-to-infinity:** si un enlace cae, los routers pueden incrementar sus métricas hasta el infinito de forma cíclica.

Soluciones al count-to-infinity:

- **Limitar el infinito:** en RIP, el límite es 16 saltos (una red a 16 saltos se considera inalcanzable).
- **Horizonte dividido (Split Horizon):** un router no anuncia una ruta de regreso a la interfaz por la que la aprendió.
- **Horizonte dividido con respuesta envenenada (Poison Reverse):** si se anuncia la ruta de regreso, se hace con distancia infinita.
- **Actualizaciones forzadas (Triggered Updates):** cuando un router detecta un cambio, difunde inmediatamente la información sin esperar al temporizador.

6.4.3. Encaminamiento por estado de enlace

Desventajas:

- Necesidad de informacion global sobre la red.
- Envio de un numero mayor de mensajes: sobrecarga.
- Un evento de cambio en la topologia debe notificarse a todos los nodos.

Protocolo OSPF (Open Shortest Path First):

- Definido en el RFC 2328 (version 2).
- Es un protocolo Intra-AS (dentro de un Sistema Autonomo).
- Utiliza el algoritmo de Dijkstra.
- Cada router conoce la topologia completa de la red.

6.4.4. Encaminamiento jerarquico: Sistemas Autonomos (AS)

Internet esta organizada en **Sistemas Autonomos (AS)**:

*Sistema Autonomo (AS) Un **Sistema Autonomo (AS)** es una coleccion de redes y encaminadores gestionadas y administradas por una misma autoridad (por ejemplo, un ISP, una universidad, una empresa).*

- **Encaminadores internos del AS:** interconectan redes dentro del propio AS. Solo conocen en detalle la organizacion local. Utilizan protocolos **IGP** (Interior Gateway Protocol).
- **Encaminadores externos o frontera (border router):** interconectan varios sistemas autonomos. Conocen el camino al resto de AS, pero no en detalle. Utilizan protocolos **EGP** (Exterior Gateway Protocol).

Protocolos IGP (dentro de un AS):

- **RIP:** Routing Information Protocol.
- **OSPF:** Open Shortest Path First.
- **IGRP:** Internal Gateway Routing Protocol (de Cisco).

Protocolos EGP (entre AS):

- **BGP:** Border Gateway Protocol. Es el protocolo que sostiene Internet, permitiendo que los diferentes AS intercambien informacion de rutas.

6.5 Protocolo RIP (Routing Information Protocol)

6.5.1. Características generales

RIP es un protocolo de encaminamiento interior (IGP) basado en vectores de distancia (algoritmo Bellman-Ford).

Tabla 6.3: Versiones de RIP

Version	Características
RIP v1 (RFC 1058, 1993)	No soporta VLSM/CIDR, envía broadcast
RIP v2 (RFC 2453, 1998)	Soporta VLSM/CIDR, envía multicast, incluye máscara
RIPng (RFC 2080, 1997)	Version para IPv6, puerto UDP 521, multicast FF02::9

Los vectores de distancia incluyen:

- La lista de destinos (redes) alcanzables por cada router.
- La distancia (numero de saltos) a dichos destinos.

La métrica de RIP es el **numero de saltos**. El límite de infinito es 16 saltos. RIP utiliza los puertos UDP 520 (RIPv2) y 521 (RIPng).

6.5.2. Temporizadores de RIP

RIP utiliza tres temporizadores principales:

- **Temporizador periódico** (25-35 s): intervalo de envío de mensajes RESPONSE para anunciar vectores de distancia. El valor oficial es 30 s, pero en la práctica se usa un valor aleatorio entre 25 y 35 s para evitar sincronización.
- **Temporizador de expiración** (180 s): controla el periodo de validez de una entrada. Si no se recibe actualización durante 180 s (equivalente a 3 intervalos periódicos), la entrada deja de considerarse válida.
- **Temporizador de “colección de basura”** (120 s): cuando una entrada expira, no se elimina inmediatamente. Se sigue anunciando con métrica 16 (destino inalcanzable) durante 120 s adicionales para notificar a los vecinos.

6.5.3. Mensajes RIP

- **REQUEST**: enviados por un router cuando se conecta a la red, o cuando caduca una entrada.
- **RESPONSE**: enviados periódicamente con los vectores de distancia, en respuesta a una solicitud, o como actualización forzada cuando cambia la distancia a una red.

6.5.4. RIPng para IPv6

RIPng (RIP new generation) es la adaptacion de RIP-2 para IPv6:

- Los mensajes se encapsulan en datagramas UDP dirigidos al puerto 521.
- Se difunden a la direccion IPv6 multicast FF02::9.
- Los vectores de distancia anuncian prefijos de red IPv6 en lugar de direcciones IPv4.
- No incluye el campo “Next Hop” en cada entrada (se usa un mecanismo separado).
- No utiliza autentificacion propia; delega a los mecanismos de IPv6.

6.6 Practica 5: Encaminamiento en Internet con RIP

6.6.1. Objetivos

El objetivo de esta practica es profundizar en el concepto de encaminamiento en redes de computadores, configurando routers con el protocolo de encaminamiento **RIP** (Routing Information Protocol) en el simulador Cisco Packet Tracer.

1. Comprender la diferencia entre encaminamiento estatico y dinamico.
2. Configurar el protocolo RIP version 2 en multiples routers.
3. Verificar que los routers intercambian informacion de rutas y convergen a la topologia de red.
4. Comprobar la conectividad entre diferentes redes mediante traceroute y ping.

6.6.2. Material utilizado

- **Software:** Cisco Packet Tracer.
- **Hardware simulado:** 4 routers interconectados mediante enlaces seriales.
- **Redes involucradas:**
 - 192.168.10.0/24 (LAN conectada a Router 0)
 - 192.168.20.0/24 (LAN conectada a Router 3)
 - 10.10.10.0/24 (enlace entre Router 0 y Router 2)
 - 10.10.20.0/24 (enlace entre Router 1 y Router 2)
 - 10.10.30.0/24 (enlace entre Router 0 y Router 1)
 - 10.10.40.0/24 (enlace entre Router 2 y Router 3)

6.6.3. Desarrollo de la practica

Topologia de la red

Se diseno una red con 4 routers interconectados mediante enlaces seriales, formando una topologia mallada. Cada router tiene al menos una LAN conectada:

- **Router 0:** conectado a la LAN 192.168.10.0/24 y a los enlaces 10.10.10.0/24 (hacia Router 2) y 10.10.30.0/24 (hacia Router 1).
- **Router 1:** conectado al enlace 10.10.20.0/24 (hacia Router 2) y 10.10.30.0/24 (hacia Router 0).
- **Router 2:** conectado a los enlaces 10.10.10.0/24 (hacia Router 0), 10.10.20.0/24 (hacia Router 1) y 10.10.40.0/24 (hacia Router 3).
- **Router 3:** conectado a la LAN 192.168.20.0/24 y al enlace 10.10.40.0/24 (hacia Router 2).

TODO: Anadir captura de pantalla de Packet Tracer mostrando la topologia con 4 routers, las LANs y los enlaces seriales.

Figura 6.1: Topologia de la red con 4 routers y encaminamiento RIP

Configuracion de RIP en los routers

El protocolo **RIP** (Routing Information Protocol) es un protocolo de encaminamiento por **vectores de distancia** que utiliza el numero de saltos como metrica. En esta practica se configuro RIP version 2, que permite el uso de subredes con mascara variable (VLSM) y envia actualizaciones por multicast.

```
1 Router> enable
2 Router# configure terminal
3 Router(config)# router rip
4 Router(config-router)# version 2
5 Router(config-router)# no auto-summary
6 Router(config-router)# network 192.168.10.0
7 Router(config-router)# network 10.10.10.0
8 Router(config-router)# network 10.10.30.0
9 Router(config-router)# passive-interface g0/0
10 Router(config-router)# default-information originate
11 Router(config-router)# end
12 Router# wr
13 Router# show ip protocols
```

Código 6.1: Configuracion de RIP en Router 0

Explicacion de los comandos:

- `router rip`: activa el proceso de encaminamiento RIP.
- `version 2`: fuerza el uso de RIPv2 (soporta CIDR/VLSM).
- `no auto-summary`: desactiva el resumen automatico de rutas.
- `network X.X.X.X`: anuncia las redes directamente conectadas.
- `passive-interface g0/0`: evita enviar actualizaciones RIP por la interfaz hacia la LAN (innecesario y mas seguro).
- `default-information originate`: propaga una ruta por defecto si existe.
- `show ip protocols`: verifica que RIP esta activo y muestra las redes configuradas.

Las configuraciones de los otros routers siguen la misma logica, anunciando las redes que tienen directamente conectadas:

```
1 Router(config)# router rip
2 Router(config-router)# version 2
3 Router(config-router)# no auto-summary
4 Router(config-router)# network 10.10.20.0
5 Router(config-router)# network 10.10.30.0
6 Router(config-router)# default-information originate
7 Router(config-router)# end
8 Router# wr
```

Código 6.2: Configuracion de RIP en Router 1

```
1 Router(config)# router rip
2 Router(config-router)# version 2
3 Router(config-router)# no auto-summary
4 Router(config-router)# network 10.10.10.0
5 Router(config-router)# network 10.10.20.0
6 Router(config-router)# network 10.10.40.0
7 Router(config-router)# default-information originate
8 Router(config-router)# end
9 Router# wr
```

Código 6.3: Configuracion de RIP en Router 2

```
1 Router(config)# router rip
2 Router(config-router)# version 2
3 Router(config-router)# no auto-summary
4 Router(config-router)# network 192.168.20.0
5 Router(config-router)# network 10.10.40.0
6 Router(config-router)# passive-interface g0/1
7 Router(config-router)# default-information originate
8 Router(config-router)# end
9 Router# wr
```

Código 6.4: Configuracion de RIP en Router 3

Verificacion del encaminamiento

Una vez configurado RIP en todos los routers, se espera unos minutos para que las tablas converjan (proceso de convergencia de RIP). Luego, se verifica que cada router conoce las redes remotas:

```

1 Router# show ip route
2
3 C    192.168.10.0/24 is directly connected, GigabitEthernet0/0
4 C    10.10.10.0/24 is directly connected, Serial0/0/0
5 C    10.10.30.0/24 is directly connected, Serial0/0/1
6 R    10.10.20.0/24 [120/1] via 10.10.30.2, 00:00:15, Serial0/0/1
7      [120/1] via 10.10.10.2, 00:00:15, Serial0/0/0
8 R    10.10.40.0/24 [120/1] via 10.10.10.2, 00:00:15, Serial0/0/0
9 R    192.168.20.0/24 [120/2] via 10.10.10.2, 00:00:15, Serial0/0/0

```

Código 6.5: Verificacion de la tabla de encaminamiento en Router 0

Analisis de la tabla:

- Las rutas marcadas con C son **directamente conectadas**.
- Las rutas marcadas con R fueron aprendidas mediante **RIP**.
- El formato [120/1] indica: distancia administrativa 120 (RIP) y **1 salto** hasta esa red.
- La red 192.168.20.0/24 esta a [120/2]: dos saltos desde Router 0.

Verificacion de conectividad

Se realizo una prueba de conectividad desde un PC en la LAN 192.168.10.0 hacia un PC en la LAN 192.168.20.0:

```

1 C:\> ping 192.168.20.10
2
3 Reply from 192.168.20.10: bytes=32 time=12ms TTL=253
4 Reply from 192.168.20.10: bytes=32 time=10ms TTL=253
5 Reply from 192.168.20.10: bytes=32 time=11ms TTL=253
6 Reply from 192.168.20.10: bytes=32 time=10ms TTL=253

```

Código 6.6: Ping desde LAN 192.168.10.0 hacia LAN 192.168.20.0

El ping exitoso con TTL=253 confirma que los paquetes atravesaron 3 routers (TTL inicial 255 menos 3 saltos = 252, ajustado por el sistema operativo).

```

1 Router# traceroute 192.168.20.10
2
3 Tracing the route to 192.168.20.10
4  1  10.10.10.2      2 msec      2 msec      2 msec
5  2  10.10.40.2      3 msec      3 msec      3 msec
6  3  192.168.20.10   4 msec      4 msec      4 msec

```

Código 6.7: Traceroute desde Router 0 hacia 192.168.20.10

El traceroute confirma que el camino es: Router 0 → Router 2 → Router 3 → PC destino.

TODO: Anadir captura de pantalla del resultado de traceroute o ping en Packet Tracer.

Figura 6.2: Resultado de traceroute mostrando el camino de 3 saltos

6.6.4. Relacion teoria-practica

Tabla 6.4: Conexion entre conceptos teoricos y elementos de la Practica 5

Concepto teorico	Observacion/aplicacion en la practica
Encaminamiento estatico	Configuracion manual previa de rutas (alternativa mostrada en el enunciado)
Encaminamiento dinamico	RIP permite que los routers aprendan rutas automaticamente sin intervencion manual
Protocolo RIP (vector distancia)	Cada router intercambia periodicamente su tabla con los vecinos, usando saltos como metrica
Convergencia	Tras configurar RIP, los routers necesitan un tiempo para intercambiar tablas y alcanzar rutas estables
Tabla de encaminamiento	show ip route muestra rutas directamente conectadas (C) y aprendidas por RIP (R)
Distancia administrativa	120 es la distancia administrativa de RIP; menor valor implica mayor confianza
Metrica de saltos	[120/2] indica 2 saltos hasta la red remota; RIP maximo 15 saltos
Next-hop	via 10.10.10.2 indica la direccion IP del siguiente router al que reenviar el paquete
TTL (Time To Live)	El TTL decrece en cada salto; un valor de 253 indica 3 routers atravesados
Passive-interface	Evita enviar actualizaciones RIP por la interfaz LAN, reduciendo trafico innecesario

6.7 Resumen de la Unidad 6

- El **encaminamiento** es la eleccion del camino optimo para un paquete; la **conmutacion** es el reenvio por ese camino.
- Los **grafos** modelan las redes: nodos son routers, enlaces son conexiones. Se representan mediante listas de adyacencia (eficientes en memoria) o matrices de adyacencia (busqueda rapida).

- Los **algoritmos de camino mas corto** como Dijkstra y Bellman-Ford permiten calcular rutas optimas. El **Minimum Spanning Tree** es util para difusion y para evitar bucles en bridges.
- Los **protocolos de encaminamiento** se clasifican en: vector de distancia (RIP, Bellman-Ford) y estado de enlace (OSPF, Dijkstra).
- El **encaminamiento jerarquico** organiza Internet en Sistemas Autonomos (AS), usando protocolos IGP (RIP, OSPF) internamente y EGP (BGP) entre AS.
- **RIP** es un protocolo IGP por vectores de distancia con metrica de saltos, limite de 16, temporizadores periodicos (30s), de expiracion (180s) y de basura (120s).
- La **Practica 5** demuestra la configuracion de RIPv2 en una red de 4 routers, la convergencia de tablas, la verificacion con `show ip route`, ping y traceroute, confirmando la comunicacion entre redes distantes.

CAPÍTULO 7

Protocolos de Comunicacion y Redes CAN

Hasta ahora hemos visto como se comunican dispositivos mediante UART, RS-232, Wi-Fi, Bluetooth y redes TCP/IP. En el entorno industrial, sin embargo, existen protocolos especificos diseñados para conectar sensores, actuadores, PLCs y sistemas de control en tiempo real. Estos protocolos deben ser robustos, rapidos y capaces de operar en entornos con ruido electrico y grandes distancias.

En este capitulo estudiamos los protocolos de comunicacion mas relevantes en la industria: I2C, SPI, RS-485, Modbus y CAN. Cada uno tiene sus ventajas y se emplea segun las necesidades de velocidad, distancia, numero de nodos y fiabilidad del sistema.

7.1 I2C: Inter-Integrated Circuit

7.1.1. Que es I2C

El bus I2C (abreviatura de *Inter-Integrated Circuits*) es un protocolo de comunicacion serie sincrona desarrollado por Philips en 1982. Se diseño para conectar circuitos integrados de baja velocidad dentro de un mismo dispositivo o placa, como microcontroladores, sensores, memorias EEPROM y pantallas.

*El **bus I2C** es un protocolo de comunicacion serie sincrono que utiliza dos lineas para transmitir datos entre dispositivos: una linea de reloj (SCL) y una linea de datos (SDA). Permite la conexion de multiples dispositivos en una arquitectura de bus con multiples maestros y multiples esclavos.*

La principal ventaja del I2C es su sencillez: solo necesita dos lineas de senal mas masa (GND), lo que reduce el numero de pines necesarios en los microcontroladores. Ademias, al ser un bus multi-maestro, cualquier dispositivo conectado puede iniciar una transferencia si el bus esta libre.

7.1.2. Senalizacion

El bus I2C utiliza tres conexiones basicas:

- **SCL** (*System Clock*): linea de reloj que sincroniza la comunicacion entre todos los dispositivos. El maestro genera los pulsos de reloj.
- **SDA** (*System Data*): linea de datos por la que se transfieren los bits de informacion entre maestro y esclavo.
- **GND** (Masa): referencia de tension comun para todos los dispositivos conectados al bus.

Ambas lineas (SCL y SDA) son *open-drain* o *open-collector*, lo que significa que cada dispositivo solo puede poner la linea a nivel bajo (0) o liberarla (1). Por eso se necesitan resistencias de pull-up externas conectadas a la alimentacion (tipicamente 4.7 k Ω a 3.3 V o 5 V).

TODO: Esquema del bus I2C con maestro, dos esclavos y resistencias de pull-up.

Figura 7.1: Esquema basico del bus I2C con resistencias de pull-up.

7.1.3. Transferencia de datos

La comunicacion en I2C siempre la inicia un **maestro**. Cada esclavo tiene una direccion unica de 7 bits (o 10 bits en modo extendido), que el maestro envia al principio de cada transaccion.

Escritura en un dispositivo esclavo:

1. El maestro envia una **condicion de inicio** (START): SDA pasa de alto a bajo mientras SCL permanece en alto.
2. Envio de la **direccion del esclavo** (7 bits) mas el bit de lectura/escritura (0 = escritura).
3. El esclavo responde con un bit de **reconocimiento** (ACK).
4. El maestro envia la **direccion del registro interno** donde quiere escribir.
5. El esclavo responde con ACK.
6. El maestro envia el **byte de dato**.
7. El esclavo responde con ACK.

Opcionalmente se pueden enviar mas bytes de datos.

8. El maestro envia una **condicion de parada** (STOP): SDA pasa de bajo a alto mientras SCL esta en alto.

Lectura desde un dispositivo esclavo:

1. Condicion de inicio (START).
2. Direccion del esclavo con bit de escritura (0).
3. ACK del esclavo.
4. Envio de la direccion del registro interno a leer.
5. ACK del esclavo.
6. **Condicion de inicio reiterada** (Repeated START).
7. Direccion del esclavo con bit de lectura (1).
8. ACK del esclavo.
9. El esclavo envia el **byte de dato**.
10. El maestro responde con ACK (si quiere leer mas) o NACK (fin de lectura).
11. Condicion de parada (STOP).

7.1.4. Ejemplo practico: sensor brujula digital CMPS03

El sensor CMPS03 es una brujula digital que mide el campo magnetico terrestre para determinar la orientacion. Una vez calibrado, ofrece una precision de 3 a 4 grados con resolucion de decimas.

Este sensor dispone de dos interfaces de salida:

- **PWM** (modulacion en anchura de pulso): un pin genera pulsos cuya duracion es proporcional al angulo.
- **I2C**: permite leer el angulo directamente como un byte (0–255) o como dos bytes (0–3599, resolucion de 0.1 grados).

Para leer el angulo por I2C, el procedimiento es:

1. Enviar START y la direccion del CMPS03 (0xC0) con bit de escritura.
2. Enviar la direccion del registro interno 0x01 (angulo en 0–255).
3. Enviar Repeated START.
4. Enviar la direccion del CMPS03 (0xC1) con bit de lectura.
5. Leer el byte de dato que contiene el angulo.
6. Enviar STOP.

7.2 SPI: Serial Peripheral Interface

7.2.1. Que es SPI

SPI es un protocolo de comunicacion serie sincrona desarrollado por Motorola en 1982. A diferencia de I2C, SPI esta pensado para alcanzar velocidades muy superiores y opera en modo **full-duplex**, lo que significa que puede enviar y recibir datos simultaneamente.

*El **bus SPI** (Serial Peripheral Interface) es un protocolo de comunicacion serie sincrona de cuatro hilos que permite la transmision full-duplex entre un dispositivo maestro y uno o varios esclavos. Utiliza lineas separadas para reloj (SCK), datos de salida (MOSI), datos de entrada (MISO) y seleccion de esclavo (SS).*

SPI se utiliza habitualmente para comunicar un microcontrolador con perifericos de alta velocidad como memorias Flash, pantallas TFT, convertidores ADC/DAC y modulos de comunicacion.

7.2.2. Senales del bus SPI

Un bus SPI completo utiliza cuatro lineas:

- **SCK** (*Serial Clock*): senal de reloj generada por el maestro. Sincroniza la transmision de datos.
- **MOSI** (*Master Out Slave In*): linea de datos del maestro al esclavo. El maestro envia informacion por esta linea.
- **MISO** (*Master In Slave Out*): linea de datos del esclavo al maestro. El esclavo responde por esta linea.
- **SS** o **CS** (*Slave Select / Chip Select*): linea de control que el maestro utiliza para seleccionar a que esclavo se dirige. Normalmente activa en nivel bajo.

TODO: Esquema SPI con maestro, dos esclavos y lineas SCK, MOSI, MISO, SS.

Figura 7.2: Esquema del bus SPI con un maestro y dos esclavos.

7.2.3. Funcionamiento

En SPI, el maestro siempre genera el reloj (SCK). En cada pulso de reloj, se transfiere un bit en ambas direcciones simultaneamente: un bit sale por MOSI y otro entra por MISO. Esto permite el modo full-duplex.

El maestro debe conocer de antemano cuantos bits espera recibir del esclavo, porque debe mantener el reloj activo durante toda la transferencia. Si el esclavo necesita tiempo para preparar la respuesta, el maestro puede introducir pequenas pausas entre bytes.

Selección de esclavo:

Existen dos formas de conectar multiples esclavos:

1. **SS independiente:** cada esclavo tiene su propia linea SS conectada al maestro. El maestro activa (baja) solo la linea del esclavo con el que quiere comunicarse.
2. **Conexion en cascada (daisy chain):** los esclavos se conectan en serie. El maestro envia datos a traves de todos los esclavos consecutivamente, como una cadena de registros de desplazamiento.

7.2.4. Comparación: SPI vs I2C vs UART

Tabla 7.1: Comparativa de protocolos de comunicacion serie

Característica	SPI	I2C	UART
Tipo de señal	Sincrona	Sincrona	Asincrona
Modo duplex	Full-duplex	Half-duplex	Full-duplex
Numero de lineas	4 (o mas)	2	2 (TX/RX)
Velocidad típica	Hasta 10+ Mbps	Hasta 3.4 Mbps	Hasta 1 Mbps
Maestros	Uno (típico)	Multiples	Ninguno (peer-to-peer)
Esclavos	Multiples (por SS)	Multiples (por direccion)	1
Direccionamiento	Por linea SS	Por direccion 7/10 bits	Ninguno
Distancia	Corta (placa)	Corta (placa)	Media (15 m RS-232)

Ventajas de SPI:

- Mayor velocidad que I2C y UART.
- Full-duplex: envia y recibe simultaneamente.
- El tamaño de los mensajes puede ser arbitrario.
- Hardware sencillo y de bajo coste.
- Los esclavos no necesitan oscilador propio (el reloj lo genera el maestro).

Desventajas de SPI:

- Necesita mas pines que I2C (especialmente si hay muchos esclavos).
- No hay confirmación de recepción por parte del esclavo (no hay ACK).
- Solo admite un maestro en la mayoría de implementaciones.
- Distancias muy cortas (dentro de la misma placa).
- El maestro debe conocer de antemano la longitud de la respuesta.

7.3 RS-485: Comunicacion diferencial robusta

7.3.1. Introduccion

RS-485 es un estandar de comunicacion serie muy utilizado en aplicaciones de adquisicion de datos y control industrial. Su principal ventaja es la capacidad de conectar multiples dispositivos en el mismo bus (hasta 32 transmisores y 32 receptores tipicamente) sobre distancias de hasta 1200 metros.

*El estandar **RS-485** (tambien conocido como EIA-485) define una interfaz de comunicacion serie multipunto con transmision diferencial balanceada. Permite la conexion de multiples transmisores y receptores en un bus semiduplex, siendo altamente resistente al ruido electrico y apto para largas distancias.*

7.3.2. Caracteristicas principales

Las caracteristicas mas destacadas de RS-485 son:

- **Transmision diferencial:** los datos se envian mediante un par trenzado de hilos (A y B). Un hilo transmite la senal original y el otro su copia invertida. El receptor detecta la **diferencia de voltaje** entre ambos, lo que elimina el ruido comun que afecta por igual a ambos hilos.
- **Multipunto:** permite conectar hasta 32 dispositivos (transmisores y receptores) en el mismo bus. Con repetidores o transceivers avanzados se puede extender a mas nodos.
- **Semiduplex:** los dispositivos pueden enviar y recibir, pero no simultaneamente. En un instante dado, solo un equipo puede transmitir, mientras que todos los demas pueden recibir.
- **Inmunidad al ruido:** la transmision diferencial y el uso de par trenzado (a veces blindado) garantizan una alta resistencia a interferencias electromagneticas.

7.3.3. Niveles de senal y cableado

En RS-485, el estado logico se determina por la diferencia de voltaje entre las lineas A y B:

- **Bit 1 (Marca / recessive):** $V_A - V_B < -200 \text{ mV}$.
- **Bit 0 (Espacio / dominant):** $V_A - V_B > +200 \text{ mV}$.

El cableado basico consiste en un par trenzado para los datos (A y B), mas lineas opcionales de alimentacion (0 V y 5 V) para alimentar dispositivos del bus. Es recomendable colocar **terminaciones de linea** (resistencias de 120Ω) al principio y al final del bus para evitar reflexiones de senal.

TODO: Esquema de bus RS-485 con par trenzado, terminaciones de 120 Ohm y varios nodos conectados.

Figura 7.3: Topologia tipica de un bus RS-485 con terminaciones.

7.3.4. Distancia vs velocidad

Una regla practica en RS-485 es que el producto de la longitud del cable (en metros) por la velocidad de datos (en bits por segundo) no debe superar 10^8 :

$$\text{Longitud (m)} \times \text{Velocidad (bps)} \leq 10^8 \quad (7.1)$$

Tabla 7.2: Relacion entre distancia y velocidad en RS-485

Distancia	Velocidad maxima aproximada
15 m	10 Mbps
100 m	1 Mbps
500 m	200 kbps
1200 m	100 kbps

7.3.5. RS-232 vs RS-485

Tabla 7.3: Comparativa entre RS-232 y RS-485

Caracteristica	RS-232	RS-485
Tipo de comunicacion	Punto a punto	Multipunto
Duplex	Full-duplex	Semiduplex
Tipo de senal	Desbalanceada (unipolar)	Balanceada (diferencial)
Niveles de voltaje	$\pm 3 \text{ V}$ a $\pm 15 \text{ V}$	Diferencial 200 mV
Max. dispositivos	1 TX + 1 RX	32 TX + 32 RX (estandar)
Velocidad maxima	1 Mbps a 15 m	10 Mbps a 15 m
Distancia maxima	$\sim 15 \text{ m}$	$\sim 1200 \text{ m}$ a 100 kbps
Inmunidad al ruido	Baja	Alta
Lineas de control	DTR, DSR, RTS, CTS	Ninguna (solo datos)

7.4 Modbus: protocolo de comunicacion industrial

7.4.1. Que es Modbus

Modbus es uno de los protocolos de comunicacion mas extendidos en la automatizacion industrial. Fue desarrollado en 1979 por Modicon (hoy Schneider

Electric) para su gama de PLCs. Hoy en dia es un estandar abierto que permite la comunicacion entre controladores, sensores, actuadores y sistemas SCADA.

*El protocolo **Modbus** es un protocolo de comunicacion abierto para la transmision de datos entre dispositivos electronicos en entornos industriales. Se situa en los niveles 1, 2 y 7 del modelo OSI. Funciona sobre arquitectura maestro/esclavo (en RTU) o cliente/servidor (en TCP/IP).*

Modbus es independiente del medio fisico: puede funcionar sobre RS-232, RS-485, Ethernet TCP/IP, o incluso por radio. Esto lo hace extremadamente versatil.

7.4.2. Variantes de Modbus

A lo largo de los años han aparecido diferentes variantes de Modbus adaptadas a diversos medios fisicos:

Tabla 7.4: Variantes del protocolo Modbus

Variante	Descripcion
Modbus RTU	Implementacion mas comun. Usa comunicacion serie (RS-232/RS-485) con representacion binaria compacta de los datos. Requiere CRC para verificacion.
Modbus ASCII	Usa comunicacion serie con caracteres ASCII. Mas lento que RTU. Utiliza checksum LRC.
Modbus TCP/IP	Funciona sobre Ethernet TCP/IP, puerto 502. No requiere checksum adicional porque TCP ya lo proporciona.
Modbus RTU sobre IP	Similar a Modbus TCP pero incluye CRC en la carga util, como en RTU.
Modbus UDP	Variante experimental sobre UDP para reducir overhead.
Modbus Plus (MB+)	Version propietaria de Schneider Electric. Soporta comunicacion peer-to-peer entre multiples maestros. Requiere hardware especial (co-procesador HDLC).

7.4.3. Modbus RTU en detalle

Modbus RTU (*Remote Terminal Unit*) es la variante mas utilizada en el entorno industrial. Se basa en una arquitectura **maestro-esclavo** sobre un bus serie (generalmente RS-485).

Estructura de la trama RTU:

Tabla 7.5: Formato de trama Modbus RTU

Campo	Tamano	Descripcion
Direccion del esclavo	1 byte	Identifica el dispositivo destino (1–247)
Codigo de funcion	1 byte	Indica la operacion a realizar (leer, escribir, etc.)
Datos	N bytes	Parametros de la funcion y valores
CRC	2 bytes	Codigo de redundancia ciclica para deteccion de errores

El dispositivo **maestro** (por ejemplo un PLC o sistema SCADA) inicia la comunicacion enviando una trama al esclavo. Solo el esclavo con la direccion indicada responde. Los demas escuchan pero ignoran el mensaje.

Codigos de funcion mas comunes:

- **01:** Leer bobinas (coils) – salidas digitales.
- **02:** Leer entradas digitales.
- **03:** Leer registros de retencion (holding registers) – 16 bits.
- **04:** Leer registros de entrada (input registers).
- **05:** Escribir una bobina.
- **06:** Escribir un registro de retencion.
- **15:** Escribir multiples bobinas.
- **16:** Escribir multiples registros.

7.4.4. RS-232 vs RS-485 en Modbus

La eleccion entre RS-232 y RS-485 depende de las necesidades de la aplicacion:

- **RS-232:** adecuado para conexiones punto a punto a corta distancia (hasta 15 m). Usa conectores DB9 o DB25. Ideal para conectar un PLC directamente a un panel HMI o a un PC de programacion.
- **RS-485:** emplea un esquema de bus multidrop que permite conectar multiples dispositivos en la misma linea. Ideal para distancias largas (hasta 1200 m) y entornos con ruido electrico. Es el estandar de facto en redes de campo industriales.

7.5 CAN: Controller Area Network

7.5.1. Introduccion

CAN (*Controller Area Network*) es un bus de comunicaciones serie diseñado originalmente por Bosch en 1986 para la industria automotriz. Hoy en dia es un estandar internacional (ISO 11898) y se emplea ampliamente en automocion, maquinaria industrial, robots, sistemas medicos y avionica.

*El **bus CAN** (*Controller Area Network*) es un protocolo de comunicacion de datos en serie de alta integridad para aplicaciones en tiempo real. Utiliza un bus de dos hilos con arbitraje basado en prioridad, difusion de mensajes, y excelentes capacidades de deteccion de errores y confinamiento de fallos.*

CAN se distingue de otros protocolos porque no utiliza direcciones de nodo. En su lugar, cada mensaje lleva un **identificador** que define tanto la prioridad como el contenido del mensaje. Cualquier nodo puede enviar un mensaje cuando el bus esta libre, y todos los demas nodos reciben ese mensaje, decidiendo individualmente si les interesa o no.

7.5.2. Caracteristicas principales

Las caracteristicas mas destacadas de CAN son:

- **Acceso multi-maestro:** cualquier nodo puede iniciar la transmision cuando el bus esta libre.
- **Difusion de mensajes** (broadcast): todos los nodos reciben cada mensaje. No hay comunicacion punto a punto.
- **Prioridad de mensajes:** el identificador determina la prioridad. Los mensajes con identificador mas bajo tienen mayor prioridad.
- **Longitud limitada:** hasta 8 bytes de datos por mensaje.
- **Velocidad:** hasta 1 Mbit/s en distancias cortas (hasta 40 m a 1 Mbps; 125 kbps a 500 m aproximadamente).
- **Deteccion de errores:** mecanismos de supervision de bits, CRC, reconocimiento y relleno de bits garantizan una tasa de error residual extremadamente baja.

7.5.3. Tipos de tramas CAN

En CAN existen cuatro tipos de tramas:

1. **Trama de datos:** transmite un mensaje al bus. Contiene el identificador, los datos (0–8 bytes) y campos de control y CRC.

2. **Trama remota:** solicita la transmision de un mensaje con un identificador especifico. Un nodo que produzca ese mensaje respondera enviando la trama de datos correspondiente.
3. **Trama de error:** cualquier nodo que detecte una condicion de error envia una trama de error para notificar a todos los demas nodos.
4. **Trama de sobrecarga:** similar a la trama de error, pero indica que un nodo no esta listo para recibir mas datos y necesita un retraso.

7.5.4. Formato de trama de datos

Una trama de datos CAN (formato estandar con identificador de 11 bits) contiene los siguientes campos:

Tabla 7.6: Campos de una trama de datos CAN (formato estandar)

Campo	Bits	Descripcion
Inicio de trama (SOF)	1	Bit dominante que marca el inicio
Identificador + RTR	12	11 bits de ID + 1 bit RTR (Remote Transmission Request)
Control	6	Indica tamano de datos y formato (estandar/extendido)
Datos	0–64	De 0 a 8 bytes de informacion util
CRC	15 + 1	Codigo de redundancia ciclica + delimitador
ACK	2	Ventana de reconocimiento por parte de los receptores
Fin de trama (EOF)	7	Bits recesivos que marcan el final
Intermission (IFS)	3	Tiempo minimo entre tramas

Formato estandar vs extendido:

- **CAN estandar** (CAN 2.0A): identificador de 11 bits (2048 mensajes posibles, aunque algunos IDs estan reservados).
- **CAN extendido** (CAN 2.0B): identificador de 29 bits (mas de 500 millones de mensajes posibles).

Ambos formatos pueden coexistir en el mismo bus. La distincion se realiza mediante el bit **IDE** (Identifier Extension).

7.5.5. Arbitraje de bus: dominante y recesivo

CAN utiliza un mecanismo de arbitraje **no destructivo** basado en los niveles logicos del bus:

- **Nivel dominante** (0 logico): el estado que impone su valor cuando un nodo lo transmite.
- **Nivel recesivo** (1 logico): el estado que solo se mantiene si *ningun* nodo transmite un dominante.

El bus CAN es como una **AND cableada**: si un nodo pone dominante y otro recesivo, el resultado es dominante. Esto permite que varios nodos comiencen a transmitir simultaneamente, pero solo el que envíe el mensaje con el **identificador mas bajo** (mas dominantes en los primeros bits) gane el arbitraje.

Proceso de arbitraje:

1. Dos o mas nodos detectan que el bus esta libre y comienzan a transmitir al mismo tiempo.
2. Cada nodo envia su identificador bit a bit.
3. Cada nodo transmisor supervisa el nivel real del bus.
4. Si un nodo envia un recesivo (1) pero ve un dominante (0) en el bus, significa que otro nodo tiene un identificador con mayor prioridad (ID mas bajo). Este nodo se retira y pasa a modo receptor.
5. El nodo con el ID mas bajo gana el arbitraje y continua transmitiendo sin perdida de tiempo ni datos.
6. Los nodos que perdieron reintentaran automaticamente cuando el bus vuelva a estar libre.

TODO: Diagrama temporal de arbitraje CAN con dos nodos compitiendo. Nodo A (ID 0x123) gana sobre Nodo B (ID 0x456).

Figura 7.4: Arbitraje no destructivo en el bus CAN.

7.5.6. Supervision de bits y relleno de bits

Supervision de bits (bit monitoring): Cada nodo transmisor supervisa el nivel del bus bit a bit y lo compara con lo que el mismo esta enviando. Si detecta una discrepancia, sabe que ha ocurrido un error (o que perdio el arbitraje).

Relleno de bits (bit stuffing): Para garantizar suficientes transiciones en el bus (necesarias para la sincronizacion), CAN inserta un bit de polaridad opuesta despues de 5 bits consecutivos del mismo valor. Este bit de relleno es transparente para la capa superior y se elimina en recepcion.

La longitud de la trama varia debido al relleno. En el peor caso (identificador de 11 bits, datos maximos):

Tabla 7.7: Longitud de trama CAN con relleno de bits

Datos	Sin relleno	Promedio	Peor caso
0 bytes	47 bits	49 bits	55 bits
8 bytes	111 bits	114 bits	135 bits

7.5.7. Errores y confinamiento de fallos

CAN dispone de sofisticados mecanismos para detectar y gestionar errores:

Tipos de errores:

1. **Error de bit:** un transmisor envia un bit pero lee el nivel opuesto en el bus.
2. **Error de relleno:** se detectan 6 bits consecutivos del mismo valor.
3. **Error CRC:** el calculo CRC del receptor no coincide con el campo CRC recibido.
4. **Error de formato:** bits de formato fijo en posiciones incorrectas.
5. **Error de reconocimiento (ACK):** ningun receptor envio un ACK dominante.

Confinamiento de fallos (fault confinement): Cada nodo CAN mantiene dos contadores de errores:

- **TEC** (*Transmit Error Counter*): cuenta errores en transmision.
- **REC** (*Receive Error Counter*): cuenta errores en recepcion.

Segun el valor de estos contadores, un nodo puede estar en tres estados:

1. **Activo** (Error Active): estado normal. El nodo puede enviar tramas de error activas (6 bits dominantes).
2. **Pasivo** (Error Passive): cuando un contador supera 127. El nodo solo puede enviar tramas de error pasivas (6 bits recesivos) y debe esperar 8 bits adicionales antes de retransmitir.
3. **Bus Off:** cuando TEC supera 255. El nodo se desconecta del bus y no puede transmitir ni recibir hasta que el problema se resuelva.

Este sistema de **voto mayoritario** asegura que un nodo defectuoso no sature el bus con tramas de error, protegiendo al resto de la red.

7.5.8. Topologia y cableado

CAN utiliza una **topologia de bus** con dos lineas principales:

- **CAN_H:** linea de bus en estado dominante alto.

- **CAN_L**: linea de bus en estado dominante bajo.

El bus debe terminarse en ambos extremos con resistencias de $120\ \Omega$ para evitar reflexiones. En topologias cortas, una sola terminacion puede bastar.

TODO: Topologia de bus CAN con terminaciones, nodos conectados y conector DB9 con pines CAN_H, CAN_L, CAN_GND y CAN_SHLD.

Figura 7.5: Topologia de bus CAN segun la norma ISO 11898.

Conector DB9 comun en CAN:

Tabla 7.8: Asignacion de pines en conector DB9 para CAN

Pin	Senal	Descripcion
2	CAN_L	Linea de bus CAN (bajo dominante)
3	CAN_GND	Masa de referencia CAN
5	CAN_SHLD	Blindaje opcional
7	CAN_H	Linea de bus CAN (alto dominante)
9	CAN_V+	Alimentacion externa opcional

7.5.9. Controladores CAN: Basic vs Full CAN

Los chips controladores CAN se clasifican en dos categorias:

- **Basic CAN**: ofrece un buffer de recepcion FIFO y un buffer de transmision. Es una solucion de bajo coste, pero requiere que el CPU atienda rapidamente los mensajes para no perder datos en redes con mucho trafico.
- **Full CAN**: proporciona multiples buffers de mensaje programables (tanto para recepcion como para transmision). El controlador puede filtrar mensajes por identificador y almacenarlos en buffers dedicados sin intervencion del CPU. Permite gestionar alto trafico de CAN con bajo rendimiento de CPU.

La mayoría de los microcontroladores modernos integran un controlador CAN Full o al menos Basic CAN, lo que facilita su uso en aplicaciones industriales sin necesidad de hardware externo.

7.5.10. Resumen: SPI, I2C, RS-485, Modbus y CAN

Tabla 7.9: Resumen comparativo de protocolos industriales

Característica	I2C	SPI	RS-485	Modbus	CAN
Tipo	Sincrono	Sincrono	Diferencial	Protocolo	Bus
Duplex	Half	Full	Half	Half/Full	Full
Nodos	127 (7 bits)	Limitado por SS	32+	247 (RTU)	1112
Distancia	Corta (placa)	Corta (placa)	1200 m	1200 m	5000 m
Velocidad	3.4 Mbps	10+ Mbps	10 Mbps	115.2 kbps	1 Mbps
Arbitraje	Software	SS hardware	Maestro unico	Master/slave	No de
Deteccion errores	ACK	Ninguna	CRC/paridad	CRC (RTU)	CRC
Uso tipico	Sensores, EEPROM	Pantallas, Flash	Redes de campo	PLCs, SCADA	Automoc

7.6 Resumen de la Unidad 7

En este capitulo hemos estudiado los protocolos de comunicacion mas relevantes para la interconexion de dispositivos en entornos industriales:

1. **I2C** es un bus sincrono de dos hilos (SDA, SCL) que permite conectar multiples maestros y esclavos en cortas distancias. Es ideal para sensores y circuitos integrados dentro de una placa.
2. **SPI** es un bus sincrono de cuatro hilos (SCK, MOSI, MISO, SS) que opera en full-duplex a muy alta velocidad. Perfecto para perifericos rapidos como memorias Flash o pantallas.
3. **RS-485** es un estandar electrico de transmision diferencial balanceada que permite conectar hasta 32 dispositivos en un bus de hasta 1200 m. Es la base fisica de muchas redes industriales.
4. **Modbus** es un protocolo de aplicacion abierto muy utilizado en automatizacion. Funciona sobre RS-232, RS-485 o TCP/IP, con arquitectura maestro/esclavo en RTU y cliente/servidor en TCP.
5. **CAN** es un bus de comunicacion de alta fiabilidad con arbitraje no destructivo por prioridad, difusion de mensajes, excelente deteccion de errores y confinamiento de fallos. Se usa en automocion, maquinaria industrial, robots y sistemas medicos.

La eleccion del protocolo adecuado depende del entorno: para sensores dentro de una placa, I2C o SPI; para redes de campo en una fabrica, RS-485 con Modbus o CAN; para sistemas criticos en tiempo real, CAN es la opcion mas robusta.

Bibliografía

- [1] Nombre Apellido. Título de un artículo de ejemplo. *Nombre de la revista*, 12(3):45–67, julio de 2024.
- [2] Nombre Apellido. *Título de un libro de ejemplo*. Editorial de ejemplo, Madrid, 2.^a edición, 2024.
- [3] Nombre de la organización. Título de un recurso web de ejemplo. Consultado en <https://ejemplo.com/recurso>.
- [4] USB Implementers Forum. *Universal Serial Bus 3.2 Specification*. Consultado en <https://www.usb.org/document-library>.
- [5] Lemus Zúñiga, G. L. “Protocolo USB”. En: *Comunicación Serie Parte 2 (T1)*, Redes Industriales, UPV, 2026.

APÉNDICE A

Manual de Instalación

A.1 Requisitos previos

- **TODO: Requisito 1:** descripción.
- **TODO: Requisito 2:** descripción.
- **TODO: Requisito 3:** descripción.

A.2 Procedimiento de instalación

1. **TODO: Paso 1:** descripción.
2. **TODO: Paso 2:** descripción.
3. **TODO: Paso 3:** descripción.

A.3 Resolución de problemas frecuentes

- **TODO: Problema 1:** descripción y solución.
- **TODO: Problema 2:** descripción y solución.
- **TODO: Problema 3:** descripción y solución.
- **TODO: Problema 4:** descripción y solución.

APÉNDICE B

Manual de Funcionamiento

B.1 Puesta en marcha

- TODO: Configuración 1: descripción.
- TODO: Configuración 2: descripción.

B.2 Operación básica

B.3 Mantenimiento y buenas prácticas

- TODO: Buena práctica 1: descripción.
- TODO: Buena práctica 2: descripción.
- TODO: Buena práctica 3: descripción.

APÉNDICE C

Otra documentación adicional

C.1 Especificaciones técnicas detalladas

C.2 Manuales y referencias complementarias

Nota para futuras ampliaciones. A continuación se sugieren elementos que podrían incorporarse en futuras revisiones del proyecto:

- **TODO: Elemento 1:** descripción.
- **TODO: Elemento 2:** descripción.
- **TODO: Elemento 3:** descripción.
- **TODO: Elemento 4:** descripción.
- **TODO: Elemento 5:** descripción.